

BloodConnect

COMPLETE VIVA GUIDE

Basics se Advanced tak - Sab Kuch Hinglish me

- 450+ Questions aur Answers - Complete Coverage
- Backend Flow - Har Controller, Model, View ka Detail
- Project Deep Dive - Har Feature Kaise Kaam Karta Hai
- Scenario-Based - Real Situations, Real Answers
- Viva Tips - Examiner Ko Kaise Impress Karein

ASP.NET Core 10 MVC | EF Core 10 | SQL Server | Identity | MemoryCache

Every drop counts. Every question counts too.

Welcome, Future Topper!

Bhai, yeh guide tumhare liye ek dost ki tarah hai. BloodConnect project tumne banaya hai, aur ab viva me usko confidently defend karna hai. Yeh PDF me tumhe **450+ questions aur answers** milenge - basics se advanced tak, aur har feature ka backend flow detail me.

Yeh Guide ka Structure

Sections 1-4: *Programming, Web, .NET, MVC basics - foundation*
Section 5: *EF Core - database ka dil*
Sections 6-8: *Auth, Blood compatibility, Workflow - project ka core*
Sections 9-11: *API, Caching, Security - engineering depth*
Sections 12-14: *Medium + Advanced topics*
Sections 15-17: *Project deep dive + scenarios + comparisons*
Section 18: *Viva tips - ready-made strategies*

Reading strategy: *Pehle 1-11 padho. Phir 14-15 (project). Last me 16-17 (tricky). Section 18 viva se ek raat pehle.*

Table of Contents

1 **Programming Fundamentals**
OOP, C# basics, types, exceptions

2 **Web Fundamentals**
HTTP, HTTPS, cookies, CORS, REST

3 **.NET Basics**
CLR, JIT, GC, middleware, config

4 **ASP.NET Core MVC**
MVC pattern, routing, Razor, DI, filters

5 **Entity Framework Core**
ORM, DbContext, migrations, LINQ, concurrency

6 **Authentication & Authorization**
Identity, cookies, CSRF, XSS, IDOR, role hierarchy

7 **BloodConnect Project Overview**
Entities, structure, seeder, controllers

8 **Blood Compatibility Domain**
Universal donor, receiver, medical rules

9 **Donation Workflow Deep Dive**
Pledge state machine, auto-reject, contact reveal

10 **REST API Deep Dive**
Endpoints, pagination, error handling, versioning

11 **Caching Deep Dive**
IMemoryCache, TTL, invalidation, distributed

12 **Security Deep Dive**
OWASP, SQL injection, XSS, HTTPS, headers

13 **Medium Level Questions**
DI lifetimes, async/await, filters, logging

14 **Advanced / Tough Questions**
SOLID, CQRS, Docker, Kubernetes, Redis, Saga

15 **Project-Specific Deep Dive**
Har controller, view, feature ka step-by-step flow

16	**Comparison Questions**	X vs Y - 30+ comparisons
17	**Scenario-Based Questions**	Real situations, kya hoga, kaise handle
18	**Viva Preparation Tips**	Strategies, must-prepare, mock viva
19	**MonsterASP.NET & Deployment**	Hosting, FTP, SSL, migrations, common errors
20	**Bilkul Basic Questions**	Computer, RAM, OS, variables, loops - beginner level
21	**IIS, Windows Hosting & Practical**	IIS, app pool, web.config, backup, health checks

SECTION 1

Programming Fundamentals

OOP, C# basics, types, exceptions, collections

Q1.1. Programming kya hai?

Ans. Programming ek process hai jisme hum computer ko step-by-step instructions dete hain kaam karne ke liye. Yeh instructions ek specific language me likhi jaati hain jaise C#, Python, JavaScript. Computer in instructions ko follow karke output deta hai. Programming ka core hai - problem ko todna, logic banana, aur solution implement karna.

Q1.2. Compiler aur Interpreter me difference?

Ans. Compiler: Poora code ek saath machine code me convert karta hai, phir execute karta hai. Fast execution, but compile step zaroori. Jaise C#, C++. Interpreter: Line by line code ko convert aur execute karta hai. Slow, but flexible. Jaise Python, JavaScript. C# compiled language hai (IL me compile hoti hai, phir JIT runtime pe machine code banata hai).

Q1.3. Statically typed vs Dynamically typed languages?

Ans. Statically typed: Variable ka type compile-time pe decide hota hai. Type mismatch pe compile error. Jaise C#, Java, C++. Dynamically typed: Type runtime pe decide hota hai. Flexible but errors runtime pe aate hain. Jaise Python, JavaScript, Ruby. C# statically typed hai - yeh safety deta hai ki galat type ka data galat jagah nahi jaayega.

Q1.4. OOP kya hai?

Ans. OOP (Object-Oriented Programming) ek programming paradigm hai jisme hum 'objects' ke through sochte hain. Har object ka state (data/properties) aur behavior (methods) hota hai. Yeh real-world entities ko code me model karne ka natural tareeka hai. 4 pillars: Encapsulation, Inheritance, Polymorphism, Abstraction.

Q1.5. OOP ke 4 pillars kya hain?

Ans. • Encapsulation - Data aur methods ko bundle karna, internal details hide karna (private fields, public methods). Jaise humari Donor class me fields private hain, properties public. • Inheritance - Ek class doosri class ki properties/methods inherit kare. Jaise AppUser IdentityUser ko inherit karta hai. • Polymorphism - Same name, different behavior. Method overloading (compile-time) aur overriding (runtime). • Abstraction - Complex details hide karke sirf essential expose karna. Interface ya abstract class se.

Q1.6. Class aur Object me difference?

Ans. Class ek blueprint ya template hai. Object us blueprint ka actual instance hai. Jaise 'Donor' ek class hai - jisme Name, BloodGroup, Age properties define hain. 'Ravi, O+, 27' ek object hai us class ka. Ek class se multiple objects bana sakte hain - har object apni state rakhta hai.

Q1.7. Constructor kya hai?

Ans. Constructor ek special method hai jo object create hone pe automatically call hoti hai. Iska naam class ke naam jaisa hota hai aur koi return type nahi hota. Kaam: object ko initialize karna, dependencies inject karna. Types: default constructor, parameterized constructor, copy constructor. Humare controllers me parameterized constructor DI ke liye use karte hain - IMessageCache jaise service inject karne ke liye.

Q1.8. Properties aur Fields me difference?

Ans. Fields - Class ke andar direct variables. Usually private hote hain. Properties - Fields ko expose karne ka safe tareeka. Getter aur setter ke through access hota hai. Validation, logic embed kar sakte hain. Jaise: private string _name; public string Name { get => _name; set => _name = value?.Trim(); } ASP.NET Core me model binding properties pe hi kaam karti hai, fields pe nahi.

Q1.9. Interface kya hai?

Ans. Interface ek contract hai - sirf method signatures define karta hai, implementation nahi. Jo class ise implement karti hai, usse saare methods define karne padte hain. Fayde: Loose coupling, testability, multiple inheritance ka alternative. Jaise IMessageCache interface hai, MessageCache uski implementation. Controller IMessageCache pe depend karta hai, actual class pe nahi.

Q1.10. Abstract class vs Interface?

Ans. Abstract class: Partial implementation de sakta hai, constructors, fields, access modifiers ho sakte hain. Single inheritance. Interface: Sirf contract (method signatures), no state, no constructor. Multiple interfaces implement kar sakte hain. Default methods bhi (C# 8+) ho sakte hain. Jab 'is-a' relationship ho to abstract class, jab 'can-do' capability ho to interface.

Q1.11. Static class kya hai?

Ans. Static class ka object nahi ban sakta. Saare members static hote hain. Direct class name se access karte hain. Jaise humari BloodCompatibility.cs static class hai - sirf utility methods hain, state nahi. Isliye static rakhi. Use: Utility/helper classes, extension methods, constants.

Q1.12. Virtual, Override, New keywords?

Ans. virtual - Method ko derived class me override karne ki permission deta hai. override - Base class ka virtual method override karta hai. new - Base class ka method hide karta hai (different from override - yahan polymorphism nahi hoti, hiding hoti hai). Jaise ToString() method virtual hai, hum override kar sakte hain.

Q1.13. Sealed class kya hai?

Ans. Sealed class ko inherit nahi kar sakte. Yeh inheritance ko rok deti hai. Use: Jab class ko extend nahi karna chahte, performance optimization ke liye bhi (JIT compiler optimizations kar sakta hai). Jaise string class sealed hai.

Q1.14. Exception kya hai?

Ans. Exception ek runtime error hai jo normal program flow ko interrupt karta hai. Jaise NullReferenceException, ArgumentException, DbUpdateException. Handle karne ke liye try-catch-finally use karte hain. Best practice: Specific exceptions catch karo, generic Exception sabse end me. Humare project me DbUpdateException catch karte hain pledge duplicate case me.

Q1.15. try-catch-finally kaise kaam karta hai?

Ans. try block me risky code likhte hain. catch block me exception handle karte hain. finally block always execute hota hai - chahe exception aaye ya na aaye. Use finally: Resources cleanup ke liye (file close, connection dispose). Best practice: Sirf wahi exceptions catch karo jinhe handle kar sakte ho. Baaki ko bubble up karne do.

Q1.16. Collections kya hain C# me?

Ans. Data structures jo multiple items hold karte hain: • List<T> - Ordered, dynamic size, index access. Jaise List<Donor>. • Dictionary<K,V> - Key-value pairs, fast lookup. Jaise cache. • HashSet<T> - Unique items, fast contains check. • Queue<T> - FIFO. • Stack<T> - LIFO. • IQueryable<T> - EF Core queries ke liye. • IEnumerable<T> - Base interface for iteration.

Q1.17. List<T> vs Array me difference?

Ans. Array: Fixed size, fastest access, zero-based index. Type-safe. List<T>: Dynamic size (auto-grow), methods (Add, Remove, Find), generics. Thoda overhead. Use List jab size change ho sakti ho, Array jab fixed size pata ho aur performance critical ho.

Q1.18. IEnumerable vs ICollection vs IList vs IQueryable?

Ans. IEnumerable<T> - Read-only iteration. Forward-only. LINQ-to-Objects. ICollection<T> - IEnumerable + Count, Add, Remove. IList<T> - ICollection + index access (this[int]). IQueryable<T> - LINQ-to-SQL/EF. Deferred execution, translated to SQL. Best practice: Parameter me sabse generic rakho (IEnumerable), return me specific (List).

Q1.19. Deferred execution kya hai?

Ans. LINQ queries tab tak execute nahi hoti jab tak unhe enumerate na karo (foreach, ToList, Count). Jaise: var q = _context.Donors.Where(d => d.Age > 25); - yahan koi SQL nahi jaata. Jab foreach ya ToListAsync karo, tab SQL execute hoti hai. Fayde: Multiple filters combine karke ek query me bhej sakte hain. Nuksan: Confusing ho sakta hai - query define karne pe kuch nahi hota.

Q1.20. yield return kya karta hai?

Ans. Iterator method banata hai jo lazy evaluation karta hai. Har baar foreach next value maangta hai, tab execute hota hai. Memory efficient hai - poori collection memory me load nahi hoti. Use: Large datasets, infinite sequences, streaming.

Q1.21. Extension method kya hai?

Ans. Existing class me method add karna bina us class ko modify kiye. Static class me static method, first parameter pe 'this' keyword. Jaise: public static string ToTitleCase(this string s) { ... } Use: Utility functions, LINQ operators, chaining. .NET me bahut common hai.

Q1.22. ref, out, in keywords?

Ans. ref - Pass by reference (read + write). Variable initialized hona chahiye. out - Pass by reference (write only). Method ko value assign karni padti hai. in - Pass by reference but read-only. Use: Performance ke liye large structs, multiple return values.

Q1.23. Tuple kya hai?

Ans. Multiple values ek saath return karne ke liye lightweight structure. C# 7+ me ValueTuple hai. Jaise: (int id, string name) = GetUser(); Return: return (1, "Ravi"); Use: Chhote grouped returns, multiple return values ke liye (out parameters se better).

Q1.24. Nullable types kya hain?

Ans. Value types jo null ho sakti hain - int?, DateTime?, bool?. Isme .HasValue aur .Value properties hain. Reference types me C# 8+ se nullable reference types - string? (null allowed), string (null not allowed by convention). Use: Optional database fields, missing values represent karne ke liye.

Q1.25. String vs StringBuilder?

Ans. String: Immutable hai - har concatenation naya string banata hai. Repeated concatenation slow (memory allocations). StringBuilder: Mutable, efficient for multiple appends. Internal buffer grow karta hai. Use StringBuilder: Loops me string concatenate kar rahe ho (100+ iterations). Use String: Simple, few concatenations.

Q1.26. String interpolation vs concatenation?

Ans. Concatenation: "Hello " + name + "!" Interpolation: $"Hello {name}!" - readable, type-safe. String.Format: string.Format("Hello {0}!", name) - older approach. Interpolation modern C# me recommended hai. Compiler internally String.Format ya efficient alternatives me convert karta hai.

Q1.27. C# me async aur await kya karte hain?

Ans. async - Method ko asynchronous banata hai. Return type Task ya Task<T> hoti hai. await - Task complete hone tak wait karta hai, but thread block nahi karta. Control caller ko return hota hai. Use: I/O bound operations (DB, HTTP, file) ke liye - server zyada requests handle kar sakta hai. Humare saare DB calls async hain - ToListAsync, SaveChangesAsync.

Q1.28. Deadlock kya hai?

Ans. Do ya zyada threads ek doosre ka resource wait karte hain circularly - koi aage nahi badh sakta. Async code me common cause: .Result ya .Wait() blocking call, sync context ke saath. Solution: End-to-end async karo, ConfigureAwait(false) use karo library code me, .Result avoid karo.

SECTION 2

Web Fundamentals

HTTP, HTTPS, cookies, CORS, REST, JSON

Q2.1. Internet kaise kaam karta hai?

Ans. Client (browser) server ko request bhejta hai - IP address ke through. Server response bhejta hai. DNS naam ko IP me convert karta hai. HTTP/HTTPS protocol communication karta hai. Steps: (1) DNS lookup, (2) TCP handshake, (3) HTTP request, (4) Server processing, (5) Response, (6) Rendering.

Q2.2. Web browser kya karta hai?

Ans. HTML parse karke DOM banata hai, CSS apply karta hai, JavaScript execute karta hai, aur page render karta hai. Resources (images, CSS, JS) fetch karta hai. Cookies manage karta hai. Requests bhejta hai HTTP/HTTPS se.

Q2.3. Web server kya hai?

Ans. Ek software jo HTTP requests receive karta hai aur responses bhejta hai. Static files serve karta hai, dynamic content generate karta hai (applications ke through). Examples: IIS (Windows), Kestrel (ASP.NET Core default), Nginx, Apache. Production me commonly: Nginx reverse proxy → Kestrel.

Q2.4. DNS kya hai?

Ans. Domain Name System - internet ki phonebook. Domain names (google.com) ko IP addresses (142.250.192.46) me convert karta hai. Hierarchical system - root servers, TLD servers, authoritative servers. Kaam: Jab aap browser me URL dalte ho, DNS resolve karta hai IP address, phir connection establish hota hai.

Q2.5. TCP/IP kya hai?

Ans. Internet ka core protocol suite. TCP (Transmission Control Protocol) - reliable, ordered delivery. IP (Internet Protocol) - addressing aur routing. TCP 3-way handshake: SYN → SYN-ACK → ACK. Data packets sequence me deliver hote hain, loss pe retransmit hote hain. HTTP TCP ke upar chalta hai (HTTP/2, HTTP/3 me variations hain).

Q2.6. HTTP vs HTTPS me difference?

Ans. HTTP: Plain text, port 80, no encryption. Data intercept ho sakta hai. HTTPS: HTTP + TLS/SSL encryption, port 443. Data encrypted, server identity verified (certificate). SEO ranking better, browser warning nahi. Modern web me HTTPS mandatory hai - especially login, payment, personal data ke liye.

Q2.7. TLS/SSL kya hai?

Ans. TLS (Transport Layer Security) - SSL ka successor. Encrypted channel establish karta hai client aur server ke beech. Steps: (1) Client hello, (2) Server hello + certificate, (3) Certificate verify, (4) Key exchange, (5) Encrypted communication. Isse: confidentiality (encryption), integrity (tampering detection), authentication (server identity).

Q2.8. URL kya hai?

Ans. Uniform Resource Locator - web resource ka address. Structure: scheme://host:port/path?query#fragment Jaise: https://bloodconnect.com:443/Donor/Search?bloodGroup=A%2B Components: • Scheme - protocol (https) • Host - domain (bloodconnect.com) • Port - 443 • Path - /Donor/Search • Query - bloodGroup=A+ • Fragment - #section1

Q2.9. HTTP request ke components kya hain?

Ans. • Request line: METHOD /path HTTP/1.1 • Headers: Host, User-Agent, Accept, Cookie, Authorization • Body: POST/PUT data (JSON, form data) Jaise: POST /Donor/Create HTTP/1.1 Host: localhost:5006 Content-Type: application/x-www-form-urlencoded Cookie: .AspNetCore.Identity=... name=Ravi&bloodGroup=O%2B&age=27

Q2.10. HTTP response ke components kya hain?

Ans. • Status line: HTTP/1.1 200 OK • Headers: Content-Type, Content-Length, Set-Cookie, Cache-Control • Body: HTML, JSON, XML, file content Jaise: HTTP/1.1 200 OK Content-Type: text/html; charset=utf-8 Content-Length: 1234 <html>...</html>

Q2.11. HTTP methods kaunse hain?

Ans. • GET - Data fetch (safe, idempotent) • POST - New resource create • PUT - Full update/replace (idempotent) • PATCH - Partial update • DELETE - Remove (idempotent) • HEAD - GET without body (headers only) • OPTIONS - Supported methods check (CORS preflight) • TRACE, CONNECT - Rarely used

Q2.12. Safe vs Idempotent methods?

Ans. Safe: Read-only, koi side effect nahi. GET, HEAD, OPTIONS. Idempotent: Multiple calls same result. Safe methods + PUT + DELETE. POST idempotent nahi. Example: DELETE /donor/5 - pehli baar delete, doosri baar 404. But state same rahi - isliye idempotent. POST /donor - do baar bhejo, do donors ban jaayenge - isliye non-idempotent.

Q2.13. Query string vs Request body?

Ans. Query string: URL me ?key=value&key2=value2. GET requests me, small data, visible, bookmarkable. Request body: POST/PUT/PATCH me. Large data, JSON/form, hidden, not bookmarkable. Rule: Data fetch ke liye GET + query. Data create/update ke liye POST/PUT + body.

Q2.14. Content-Type header kya batata hai?

Ans. Request/response body ka format. • application/json - JSON data • application/x-www-form-urlencoded - form data (default) • multipart/form-data - file upload • text/html - HTML page • text/plain - plain text • application/xml - XML • application/octet-stream - binary Browser/Postman is header se body parse karte hain.

Q2.15. Accept header kya karta hai?

Ans. Client batata hai kaunsa response format accept kar sakta hai. Accept: application/json - JSON chahiye Accept: text/html - HTML chahiye Accept: */* - kuch bhi chalega Content negotiation me use hota hai. ASP.NET Core me formatters is header ke hisaab se response format dete hain.

Q2.16. Cookies kaise kaam karti hain?

Ans. Server Set-Cookie header se cookie bhejta hai. Browser store karta hai. Har subsequent request me Cookie header me automatically bhejta hai (same domain, matching path). Properties: name, value, domain, path, expiry, secure, httponly, samesite. Use: Session tracking, authentication, preferences. Hum Identity.Application cookie use karte hain authentication ke liye.

Q2.17. Session vs Cookie vs Local Storage?

Ans. Session: Server-side storage, only session ID client ke paas (cookie me). Secure, unlimited size. Cookie: Client-side, ~4KB limit, automatic bhejti hai server ko. Local Storage: Client-side, ~5-10MB, JavaScript se access, cookie ki tarah automatic nahi bhejti. Use case: Auth = cookie (HttpOnly). Cart = session ya DB. Preferences = localStorage.

Q2.18. Stateless protocol kya hai?

Ans. Server har request ko independent treat karta hai - previous request yaad nahi rakhta. HTTP stateless hai. Challenge: Login session kaise maintain karo? Solutions: Cookies, session, JWT tokens. Har request me identity credentials bhejo.

Q2.19. CORS kya hai?

Ans. Cross-Origin Resource Sharing - browser security. Default me browser ek origin ka JS doosre origin ke API ko call nahi kar sakta (unless server allow kare). Server CORS headers return karta hai: Access-Control-Allow-Origin: https://example.com Access-Control-Allow-Methods: GET, POST Access-Control-Allow-Credentials: true Preflight: Complex requests ke liye browser pehle OPTIONS bhejta hai.

Q2.20. JSON kya hai?

Ans. JavaScript Object Notation - lightweight data format. Human-readable, language-independent. Types: object {}, array [], string "", number, boolean, null. Jaise: {"name": "Ravi", "age": 27, "available": true, "groups": ["O+", "O-"]} Modern APIs JSON use karti hain XML se zyada. ASP.NET Core me System.Text.Json default serializer.

Q2.21. XML vs JSON me difference?

Ans. XML: Verbose, tags <tag></tag>, schema validation (XSD), namespaces, comments. JSON: Compact, {key: value}, no schema (usually), no comments, faster parse. Use: JSON modern web/mobile APIs me. XML enterprise, SOAP, config files me.

Q2.22. REST API kya hai?

Ans. REST (Representational State Transfer) - architectural style for APIs. Principles: • Client-server separation • Stateless • Cacheable • Uniform interface (HTTP methods + URIs) • Layered system • Resources identified by URLs Humara /api/donor/search ek REST endpoint hai.

Q2.23. SOAP vs REST?

Ans. SOAP: XML-only, WSDL contract, strict protocol, built-in security (WS-Security), stateful options. REST: Multiple formats (JSON/XML), no strict contract, HTTP-based, lightweight, stateless. Modern apps REST prefer karti hain. SOAP enterprise/legacy me (banking, insurance).

Q2.24. WebSocket kya hai?

Ans. Full-duplex communication protocol over single TCP connection. Client aur server dono ek saath messages bhej sakte hain. Handshake HTTP upgrade se hota hai, phir protocol switch ho jaata hai (ws:// ya wss://). Use: Chat, live notifications, real-time gaming, trading apps. ASP.NET Core me SignalR se abstract karte hain.

Q2.25. HTTP/2 vs HTTP/1.1?

Ans. HTTP/1.1: Ek connection pe ek request at a time (pipelining limited). Multiple connections zaroori parallel ke liye. HTTP/2: Multiplexing - ek connection pe multiple parallel requests. Header compression (HPACK). Server push. Binary protocol. HTTP/3: QUIC (UDP-based), faster handshake, better mobile performance.

Q2.26. Progressive Web App (PWA) kya hai?

Ans. Web app jo native app jaisi feel deti hai - offline support, push notifications, install prompt, home screen icon. Components: Service Workers (offline + caching), Web App Manifest, HTTPS. BloodConnect PWA nahi hai - traditional MVC web app hai. PWA roadmap me ho sakta hai.

SECTION 3

.NET Framework Basics

CLR, JIT, GC, middleware, configuration

Q3.1. .NET kya hai?

Ans. .NET ek free, open-source, cross-platform developer platform hai Microsoft ka. Isme runtime (CLR), libraries (BCL), SDK, aur tools hote hain. Use: Web apps, desktop, mobile, games, IoT, cloud, machine learning. Languages: C#, F#, VB.NET. C# sabse popular. Versions: .NET Framework (legacy), .NET Core (2016-2019), .NET 5+ (unified).

Q3.2. .NET Framework vs .NET Core vs .NET 5+?

Ans. .NET Framework: Windows-only, 2002 se. Legacy but still supported. .NET Core: Cross-platform, open-source, 2016 se. High performance. .NET 5+: Unified platform (2020 se). .NET 5, 6, 7, 8, 9, 10 sab isi line me. Har saal November me naya version. Humara BloodConnect .NET 10 use karta hai.

Q3.3. .NET versions ka support cycle?

Ans. LTS (Long Term Support): 3 saal support. Even versions - 6, 8, 10. STS (Standard Term Support): 18 months support. Odd versions - 5, 7, 9. Production me LTS recommended. Hum .NET 10 (LTS) use kar rahe hain.

Q3.4. CLR kya hai?

Ans. Common Language Runtime - .NET ka execution engine. Kaam: • Code compile karta hai (IL → machine code, JIT) • Memory manage karta hai (Garbage Collection) • Type safety enforce karta hai • Exception handling • Security (Code Access Security) • Threading manage karta hai Java ka JVM jaisa concept hai.

Q3.5. BCL kya hai?

Ans. Base Class Library - .NET ki standard libraries ka collection. Thousands of types. Jaise: System (core types), System.Collections, System.Linq, System.IO (file), System.Net (network), System.Data (database), System.Threading.Tasks (async). ASP.NET Core, EF Core BCL ke upar build hote hain.

Q3.6. IL/MSIL kya hai?

Ans. Intermediate Language - C# compiler ka output. Platform-independent, stack-based. Jaise C#: int x = 5 + 3; IL: ldc.i4.5 / ldc.i4.3 / add / stloc.0 JIT compiler runtime pe IL ko machine code me convert karta hai. Isliye same IL different platforms pe chalti hai.

Q3.7. JIT compiler kya karta hai?

Ans. Just-In-Time - runtime pe IL ko native machine code me convert karta hai, method by method. Process: IL → JIT → x86/x64/ARM machine code → CPU execute. Optimizations: Method inlining, loop unrolling, dead code elimination. Types: Normal JIT (most common), Eco JIT (mobile), ReadyToRun (pre-JIT).

Q3.8. AOT compilation kya hai?

Ans. Ahead-Of-Time - IL ko build time pe machine code me convert karta hai (JIT ke bajaye). .NET 7+ me Native AOT - startup time kam, memory kam, no JIT dependency. Use: Microservices, CLI tools, mobile apps, serverless. Trade-off: Larger binary, limited reflection support.

Q3.9. Garbage Collection kya hai?

Ans. Automatic memory management. Jab object reachable nahi hota, GC memory reclaim karta hai. Generations: • Gen 0 - Short-lived objects, fast collection • Gen 1 - Medium-lived (buffer) • Gen 2 - Long-lived, slow collection • LOH (Large Object Heap) - 85KB+ objects Types: Workstation (client), Server (multi-threaded, server apps).

Q3.10. GC me dispose vs finalizer?

Ans. Dispose(): Deterministic cleanup. IDisposable interface. 'using' statement ke saath automatically call hota hai. Unmanaged resources (file, DB connection) ke liye. Finalizer: Non-deterministic, GC ke saath call hota hai. Slow. Sirf tab jab Dispose miss hua. Best: Both implement karo - Dispose() se finalizer suppress karo.

Q3.11. IDisposable kya hai?

Ans. Interface hai jisme Dispose() method hai. Unmanaged resources release karne ke liye. 'using' statement: using (var conn = new SqlConnection(...)) { /* ... */ } // dispose automatically Humara DbContext bhi IDisposable hai - DI container dispose kar deta hai.

Q3.12. Value types vs Reference types?

Ans. Value: Stack pe store, copy by value. Jaise int, bool, struct, enum, DateTime. Reference: Heap pe store, copy by reference. Jaise class, string, array, delegate. Example: int a = 5; int b = a; b = 10; // a still 5 Donor d1 = new(); Donor d2 = d1; d2.Name = "X"; // d1.Name bhi X

Q3.13. Stack vs Heap?

Ans. Stack: LIFO, per-thread, small (1MB), fast. Value types + method call frames. Heap: Shared, large, slower. Reference types (objects). GC manage karta hai. Escape analysis se compiler decide karta hai object stack pe rakhna ya heap pe.

Q3.14. Boxing aur Unboxing?

Ans. Boxing: Value type ko reference type me convert karna (object ya interface). int x = 5; object o = x; // boxing Unboxing: Reference type se value type nikalna. int y = (int)o; // unboxing Cost: Heap allocation + copy. Performance hit. Avoid karo loops me.

Q3.15. CTS aur CLS?

Ans. CTS (Common Type System): Common data types jo saari .NET languages share karti hain. int, string, bool - C# me bhi, VB.NET me bhi same. CLS (Common Language Specification): Rules jo language interop ke liye follow karo. Jaise C# me public method chhota naam, VB.NET me use ho sake.

Q3.16. Managed vs Unmanaged code?

Ans. Managed: CLR ke control me. C#, VB.NET, F#. GC memory manage karta hai, type safety, exception handling. Unmanaged: Direct OS ke saath. C, C++ native. Manual memory management. P/Invoke se managed se call ho sakta hai. Jaise SQL Server native unmanaged hai, but .NET se SqlClient ke through connect hote hain.

Q3.17. Assembly kya hai?

Ans. .NET me deployment ka unit. .dll ya .exe file. Isme: • IL code • Metadata (types, methods) • Manifest (version, dependencies) Types: Private (local folder me), Shared (GAC - Global Assembly Cache). Strong naming: Signed assembly, unique identity (public key).

Q3.18. Namespace kya hai?

Ans. Logical grouping of types. Naming conflicts avoid karta hai. Jaise: System.Collections.Generic me List<T> hai. System.Web.Mvc me Controller hai. using directive se import: using Microsoft.AspNetCore.Mvc; File-scoped namespace (C# 10+): namespace BloodConnect.Models;

Q3.19. NuGet kya hai?

Ans. .NET ka package manager. NuGet.org pe millions of packages hain. Commands: dotnet add package Microsoft.EntityFrameworkCore.SqlServer dotnet restore dotnet list package Project file (.csproj) me PackageReference is format me hota hai: <PackageReference Include='PackageName' Version='10.0.0' />

Q3.20. Middleware kya hai ASP.NET Core me?

Ans. Request pipeline me ek software component. Chain of middlewares. Flow: Request → Middleware1 → Middleware2 → ... → Endpoint ← Response ← Response ← ... ← Har middleware: (1) Request process, (2) Next call ya short-circuit, (3) Response process. Examples: UseAuthentication, UseAuthorization, UseRouting, UseStaticFiles.

Q3.21. Request pipeline ka order?

Ans. Order important hai: 1. ExceptionHandler 2. HSTS 3. HttpsRedirection 4. StaticFiles 5. Routing 6. CORS 7. Authentication 8. Authorization 9. Custom middleware 10. Endpoints Program.cs me order preserve karo - galat order pe issues.

Q3.22. Kestrel kya hai?

Ans. ASP.NET Core ka default cross-platform web server. Production me IIS/Nginx reverse proxy ke peeche use hota hai. Features: HTTP/1.1, HTTP/2, HTTPS, WebSockets. Kestrel direct internet pe expose nahi karte usually - reverse proxy ke through.

Q3.23. appsettings.json kya hai?

Ans. Configuration file - JSON format. Sections me settings. Jaise: { "ConnectionStrings": { "DefaultConnection": "Server=..." }, "Logging": { ... } } Environment-specific: appsettings.Development.json, appsettings.Production.json. Access: IConfiguration inject karke, ya IOptions<T> pattern se.

Q3.24. Environment variables configuration me kaise use karte hain?

Ans. Double underscore (__) se nested keys. Jaise: ConnectionStrings__DefaultConnection=Server=... ASP.NET Core me higher priority than appsettings.json. Use: Production secrets (connection strings, API keys) - repo me commit nahi karne.

Q3.25. ILogger kya hai?

Ans. Built-in logging interface. DI se inject. _logger.LogInformation("User {UserId} searched {BloodGroup}", userId, bg); Levels: Trace, Debug, Information, Warning, Error, Critical. Providers: Console, File, EventLog, Application Insights, Serilog, NLog.

SECTION 4

ASP.NET Core MVC

MVC pattern, routing, Razor, DI, filters, model binding

Q4.1. MVC kya hai?

Ans. Model-View-Controller - architectural pattern jo application ko teen parts me todta hai: • Model - Data + business logic • View - UI (presentation) • Controller - Request handle, model manipulate, view choose Fayde: Separation of concerns, testability, maintainability. ASP.NET Core MVC iska popular implementation hai.

Q4.2. MVC me request flow kya hai?

Ans. 1. Browser URL hit karta hai. 2. Kestrel request receive karta hai. 3. Routing middleware URL ko controller+action me map karta hai. 4. Model binding form/query data ko object me convert karta hai. 5. Controller action execute. 6. Action data fetch/modify karta hai (via Model/Service). 7. View result return hota hai. 8. Razor view HTML generate karta hai. 9. Response browser ko jaata hai.

Q4.3. MVC vs MVP vs MVVM?

Ans. MVC: Controller request handle karta hai, View ko model bhejta hai. MVP (Model-View-Presenter): Presenter view ke saath 1:1, bi-directional. Desktop apps. MVVM (Model-View-ViewModel): ViewModel data binding se View ko update karta hai. WPF, Xamarin, Angular. ASP.NET Core MVC - classic MVC pattern.

Q4.4. MVC vs Web Forms?

Ans. Web Forms: Event-driven, ViewState, server controls, complex page lifecycle, difficult testing. MVC: Request-driven, stateless, clean URLs, HTML control, testable, separation. MVC modern standard hai - .NET Core me recommended.

Q4.5. MVC vs Web API?

Ans. MVC: Full-stack - HTML views return karta hai (Razor). Users ke liye web pages. Web API: Data (JSON/XML) return karta hai. Mobile apps, SPA, third-party. .NET Core me dono ek hi framework me - ControllerBase (API) vs Controller (MVC).

Q4.6. MVC vs Razor Pages?

Ans. MVC: Controller-based, ek controller multiple actions. Complex apps. Razor Pages: Page-based, har page ka apna .cshtml + .cshtml.cs. Simple CRUD. Humara BloodConnect MVC pattern pe hai - multiple related views, shared logic, complex workflows.

Q4.7. Controller kya hai?

Ans. Ek class jo ControllerBase (ya Controller) inherit karti hai. Request handle karti hai, action methods (public methods) define karti hai. Jaise: public class DonorController : Controller { public async Task<IActionResult> Index() { ... } public IActionResult Create() { ... } [HttpPost] public async Task<IActionResult> Create(Donor donor) { ... } } Convention: Controller suffix, plural name.

Q4.8. Action method kya hai?

Ans. Controller ka public method jo request handle karta hai. Return type usually IActionResult ya Task<IActionResult>. Action ka naam URL me aata hai. [HttpGet], [HttpPost] attributes HTTP method restrict karte hain. Humare: Index, Create, Edit, Delete, Details, Search, Offer, Accept, Complete.

Q4.9. IActionResult types kya hain?

Ans. • ViewResult - View() return • PartialViewResult - PartialView() • JsonResult - Json(data) • RedirectResult - Redirect(url) • RedirectToActionResult - RedirectToAction("Index") • NotFoundResult - NotFound() • BadRequestResult - BadRequest() • ForbidResult - Forbid() • StatusCodeResult - StatusCode(500) • FileResult - File(bytes, type) • EmptyResult - kuch nahi

Q4.10. ViewResult kya hai?

Ans. Action ka result jo Razor view render karta hai. View() method return karta hai. return View(); // action ke naam ka view (Index.cshtml) return View(model); // model pass karta hai return View("~/Views/Shared/Custom.cshtml"); View engine Views/{Controller}/{Action}.cshtml dhunda hai.

Q4.11. Model binding kaise kaam karti hai?

Ans. HTTP request data ko (form fields, query string, route values, JSON body) C# parameters me map karta hai. Automatically. Order: Form → Route → Query Jaise: public IActionResult Search(string bloodGroup, string location, int page = 1) Direct object binding: public IActionResult Create(Donor donor) Attributes: [FromBody], [FromQuery], [FromRoute], [FromForm].

Q4.12. Model validation kaise hoti hai?

Ans. Validation attributes model pe lagate hain: [Required], [StringLength(50)], [Range(18, 65)], [RegularExpression(@"...")], [EmailAddress], [Compare("Password")] Controller me check: if (!ModelState.IsValid) return View(model); Error display: Custom attribute: MustBeTrueAttribute (humar■ Terms acceptance).

Q4.13. Routing kya hai?

Ans. URL ko controller action se map karta hai. Convention-based: app.MapControllerRoute(name: "default", pattern: "{controller=Home}/{action=Index}/{id?}"); Attribute-based: [Route("api/donor")] [HttpGet("search")] Route values: {controller}, {action}, {id?}. Constraints: {id:int}, {slug:alpha}.

Q4.14. Razor kya hai?

Ans. View engine - C# aur HTML mix karke views likhne deta hai. @ symbol se C# code. @model Donor <h1>@Model.Name</h1> @if (Model.IsAvailable) { Available } @foreach (var d in Model.Donors) { @d.Name } Compile-time pe errors. Clean syntax. Tag helpers ke saath modern.

Q4.15. Layout view kya hai?

Ans. Master template - common structure (header, footer, nav, CSS/JS). _Layout.cshtml: <html> <body> <nav>...</nav> @RenderBody() <footer>...</footer> </body> </html> Views: @{ Layout = "_Layout"; } _ViewStart.cshtml se default layout set hota hai.

Q4.16. Partial view kya hai?

Ans. Reusable Razor chunk - doosre views me include. <partial name="_DonorForm" model="Model" /> @await Html.PartialAsync("_LoginPartial") Humare: _Form.cshtml (donor create/edit form), _LoginPartial.cshtml (login badge with unread count). View Component se different - View Component me logic hoti hai (C# class), partial view sirf presentation.

Q4.17. Section kya hai Razor me?

Ans. Named placeholder jo view fill karta hai. Layout me render hota hai. View: @section Scripts { <script src="~/js/donor.js"></script> } Layout: @await RenderSectionAsync("Scripts", required: false) Use: Per-page CSS/JS, extra headers.

Q4.18. Tag Helpers vs HTML Helpers?

Ans. Tag Helpers (modern): <form asp-action="Create"> <input asp-for="Name" /> </form> HTML Helpers (legacy): @Html.BeginForm("Create") @Html.TextBoxFor(m => m.Name) @Html.ValidationMessageFor(m => m.Name) Tag Helpers readable, HTML-jaisa syntax, designer-friendly. Recommended.

Q4.19. Filters kya hain?

Ans. Attributes jo action execute hone se pehle ya baad me kuch karte hain. Types: • Authorization - pehle chalte hain • Resource - cached responses • Action - before/after action • Exception - errors • Result - result ke before/after Examples: [Authorize], [AllowAnonymous], [ValidateAntiForgeryToken], [ResponseCache]. Global filters Program.cs me: options.Filters.Add(new AutoValidateAntiforgeryTokenAttribute()).

Q4.20. ViewBag vs ViewData vs TempData?

Ans. ViewBag: dynamic property, Controller → View same request. @ViewBag.Message ViewData: Dictionary<string, object>. @ViewData["Message"] TempData: Ek request se doosri tak (redirect ke baad). @TempData["Message"] Best practice: Strongly typed ViewModels use karo - type-safe, IntelliSense.

Q4.21. ViewModels kya hain?

Ans. Classes jo specifically ek view ke liye data structure define karti hain. Domain entities ko directly views me expose nahi karte. Jaise: public class AdminDashboardViewModel { public List<UserViewModel> Users { get; set; } public int TotalDonors { get; set; } public bool IsSuperAdmin { get; set; } } Fayde: Security (over-posting se bachav), flexibility, clear intent.

Q4.22. Areas kya hain?

Ans. MVC app ko sections me organize karte hain. Har area ke apne Controllers, Views, Models. Structure: Areas/Admin/Controllers/HomeController.cs Areas/Admin/Views/Home/Index.cshtml Route: [Area("Admin")] attribute, ya default route pattern me {area:exists}. Humare project me single area hai - abhi zaroorat nahi thi.

Q4.23. _ViewImports aur _ViewStart?

Ans. _ViewImports.cshtml: Common using statements, TagHelper imports. Har view me auto-apply. @using BloodConnect.Models @addTagHelper *, Microsoft.AspNetCore.Mvc.TagHelpers _ViewStart.cshtml: Default layout set karta hai. @{ Layout = "_Layout"; } Root folder me ek, subfolder me override kar sakte hain.

Q4.24. _Layout me RenderBody vs RenderSection?

Ans. @RenderBody(): Actual view ka content yahan render hota hai. Ek hi baar. @RenderSection("Scripts", required: false): Named section. Multiple ho sakte hain. required: false se optional. Layout: <body> @RenderBody() @await RenderSectionAsync("Scripts", false) </body>

Q4.25. ASP.NET Core MVC me DI kaise hoti hai?

Ans. Program.cs me services register: builder.Services.AddScoped<IDonorService, DonorService>(); builder.Services.AddSingleton<IMessageCache, MessageCache>(); builder.Services.AddDbContext<ApplicationDbContext>(...); Constructor injection: public class DonorController : Controller { private readonly ApplicationDbContext _context; private readonly IMessageCache _cache; public DonorController(ApplicationDbContext context, IMessageCache cache) { _context = context; _cache = cache; } }

Q4.26. Async controller actions kyun?

Ans. DB I/O slow hai. Async action se thread block nahi hoti, server zyada concurrent requests handle karta hai. [HttpGet] public async Task<IActionResult> Index() { var donors = await _context.Donors.ToListAsync(); return View(donors); } Best practice: Async all the way. Sync over async never.

SECTION 5

Entity Framework Core

ORM, DbContext, migrations, LINQ, concurrency, relationships

Q5.1. Entity Framework Core kya hai?

Ans. EF Core ek ORM hai - Object-Relational Mapper. C# objects aur database tables ke beech mapping karta hai. Features: • LINQ queries → SQL • Change tracking • Migrations • Relationship management • Concurrency handling • Multiple database providers Cross-platform, lightweight, high-performance.

Q5.2. ORM kya hai?

Ans. Object-Relational Mapping - technique jo object-oriented code ko relational database se connect karti hai. Mapping: Class → Table Object → Row Property → Column Navigation property → Foreign key relationship Fayde: No raw SQL, type-safe queries, portability, security (SQL injection protection).

Q5.3. Code First vs Database First vs Model First?

Ans. Code First: C# classes likho, EF migrations se database banao. Version control friendly, modern. Database First: Existing database se classes generate karo. Legacy systems ke liye. Model First: Visual designer se model banao, phir code + DB generate. Rarely used. Humara project Code First hai.

Q5.4. DbContext kya hai?

Ans. EF Core ka main class. Database session represent karta hai. public class ApplicationDbContext : IdentityDbContext<AppUser> { public DbSet<Donor> Donors { get; set; } public DbSet<BloodRequest> BloodRequests { get; set; } public DbSet<DonationOffer> DonationOffers { get; set; } public DbSet<ContactMessage> ContactMessages { get; set; } protected override void OnModelCreating(ModelBuilder mb) { ... } } Kaam: Change tracking, query execution, SaveChanges, transactions.

Q5.5. DbSet kya hai?

Ans. DbContext me ek property jo database table represent karta hai. _context.Donors → Donors table _context.Donors.Where(d => d.Age > 25) → LINQ query Operations: Add, Remove, Update, Find, Where, FirstOrDefaultAsync, ToListAsync. IQueryable<T> implement karta hai - deferred execution.

Q5.6. Migration kya hai?

Ans. Version control for database schema. Model change → migration → SQL script → database. dotnet ef migrations add InitialCreate dotnet ef database update Files generated: Migrations/20240101_InitialCreate.cs (Up/Down methods) Migrations/ApplicationDbContextModelSnapshot.cs Fayde: Team sync, rollback, production deployment.

Q5.7. Migration kaise rollback karte hain?

Ans. Last migration undo: dotnet ef database update PreviousMigrationName Specific migration remove: dotnet ef migrations remove All migrations ka SQL script: dotnet ef migrations script Production me: Idempotent script use karo (IF NOT EXISTS checks ke saath).

Q5.8. Primary Key convention?

Ans. By convention: • Property "Id" → PK • Property "{ClassName}Id" → PK • Property "{ClassName}ID" → PK Explicit: [Key] attribute Fluent API: entity.HasKey(e => e.Id) Composite key: entity.HasKey(e => new { e.RequestId, e.DonorId })

Q5.9. Foreign Key kaise define karte hain?

Ans. Convention se: public int DonorId { get; set; } public Donor Donor { get; set; } Explicit: [ForeignKey("DonorId")] public Donor Donor { get; set; } Fluent API: entity.HasOne(o => o.Donor) .WithMany(d => d.Offers) .HasForeignKey(o => o.DonorId) .OnDelete(DeleteBehavior.Cascade);

Q5.10. Relationships ke types?

Ans. One-to-One: User ↔ Donor profile One-to-Many: BloodRequest ↔ DonationOffers public ICollection<DonationOffer> Offers { get; set; } Many-to-Many: Book ↔ Author (junction table) EF Core 5+: Skip navigations se direct many-to-many.

Q5.11. Cascade delete kya hai?

Ans. Parent delete hone pe related children automatically delete. .onDelete(DeleteBehavior.Cascade) Options: • Cascade - children delete • Restrict - delete block • SetNull - FK null • NoAction - DB decide kare Hum offers ko request ke saath cascade delete karte hain - orphan records avoid karne ke liye.

Q5.12. LINQ kya hai EF Core me?

Ans. C# me query syntax. Objects, database, XML - sab pe same syntax. var donors = await _context.Donors .Where(d => d.BloodGroup == "O+" && d.IsAvailable) .OrderBy(d => d.Name) .Select(d => new DonorDto { Id = d.Id, Name = d.Name }) .Skip((page-1)*size).Take(size) .ToListAsync(); EF Core LINQ ko SQL me translate karta hai (expression tree parse karke).

Q5.13. AsNoTracking kab use karte hain?

Ans. Read-only queries me - jab result update nahi karna. Without tracking: EF change tracker me entity store nahi karta. Memory kam, faster. Hum: search queries me use karte hain. var donors = await _context.Donors.AsNoTracking() .Where(...).ToListAsync(); Update/edit forms me tracking chahiye - AsNoTracking nahi lagate.

Q5.14. Include() kya karta hai?

Ans. Related entities eagerly load karta hai - single query me. var request = await _context.BloodRequests .Include(r => r.Offers) .ThenInclude(o => o.Donor) .FirstOrDefaultAsync(r => r.Id == id); Without Include: Offers null rahega (lazy loading off hai to). Cartesian explosion se bachne ke liye AsSplitQuery() use kar sakte hain.

Q5.15. Lazy vs Eager vs Explicit loading?

Ans. Lazy: Property access pe auto-load. Alag query. N+1 problem. Eager: .Include() se pehle. Single query. Explicit: Manually .Load() jab zaroorat. _context.Entry(request).Collection(r => r.Offers).Load(); Best: Eager loading (predictable, efficient).

Q5.16. N+1 query problem kya hai?

Ans. Ek query se N records fetch, phir har record ka related data ke liye N queries. Jaise: var requests = await _context.BloodRequests.ToListAsync(); // 1 query foreach (var r in requests) { var offers = r.Offers; // N queries (lazy) } Solution: Include() se eager load, ya projection se specific data fetch.

Q5.17. Change Tracker kya karta hai?

Ans. DbContext me tracked entities ka state: • Added - SaveChanges pe INSERT • Modified - UPDATE • Deleted - DELETE • Unchanged - kuch nahi SaveChanges() pe EF Core in states ke hisaab se SQL generate karta hai. AsNoTracking() se is overhead se bach sakte hain read-only queries me.

Q5.18. SaveChanges vs SaveChangesAsync?

Ans. SaveChanges() sync - thread block. SaveChangesAsync() async - thread free. Web apps me async use karo: await _context.SaveChangesAsync(); Multiple changes ek transaction me commit hote hain - atomicity.

Q5.19. Transaction kaise use karte hain?

Ans. Single SaveChanges = implicit transaction. Multiple SaveChanges ek transaction me: using var tx = await _context.Database.BeginTransactionAsync(); try { await _context.SaveChangesAsync(); await _context.SaveChangesAsync(); await tx.CommitAsync(); } catch { await tx.RollbackAsync(); } Humara pledge accept - auto-reject baaki offers single SaveChanges me.

Q5.20. Concurrency token / RowVersion kya hai?

Ans. Optimistic concurrency ke liye. Har update pe automatically increment. public byte[] RowVersion { get; set; } // [Timestamp] attribute ya Fluent API Update attempt: UPDATE Donors SET Name=@p WHERE Id=@id AND RowVersion=@rv Agar RowVersion mismatch (koi aur update kar chuka): DbUpdateConcurrencyException. Hum: Donor entity pe RowVersion - concurrent edit protection.

Q5.21. Concurrency conflict handle kaise karte hain?

Ans. try { await _context.SaveChangesAsync(); } catch (DbUpdateConcurrencyException ex) { var entry = ex.Entries.Single(); var dbValues = await entry.GetDatabaseValuesAsync(); if (dbValues == null) { // Record deleted by someone return NotFound(); } // Option 1: Client wins (overwrite) // Option 2: DB wins (reload) // Option 3: Merge (user decides) } Hum: User ko friendly error + refresh suggestion.

Q5.22. EnableRetryOnFailure kya karta hai?

Ans. Transient errors pe automatically retry. Jaise network glitch, temporary DB unavailable. options.UseSqlServer(conn, sql => sql.EnableRetryOnFailure(maxRetryCount: 5, maxRetryDelay: TimeSpan.FromSeconds(10), errorNumbersToAdd: null)); SQL Azure, cloud environments me useful. Note: Idempotent operations me safe. Non-idempotent (insert) me duplicate risk.

Q5.23. Raw SQL queries kab likhte hain?

Ans. Jab LINQ complex ho ya performance critical ho. FromSqlRaw (read): var donors = _context.Donors .FromSqlRaw("SELECT * FROM Donors WHERE BloodGroup = {0}", bg) .ToListAsync(); ExecuteSqlRaw (write): await _context.Database.ExecuteSqlRawAsync("UPDATE Donors SET IsAvailable = 0 WHERE Age > 65"); Parameterized use karo - SQL injection se bachav.

Q5.24. Index kya hai aur kab lagate hain?

Ans. Data structure jo lookup fast karta hai. B-tree usually. Kab: • WHERE, JOIN, ORDER BY me frequently used columns • Foreign keys • Unique constraints Fluent API: entity.HasIndex(d => d.BloodGroup); entity.HasIndex(o => new { o.RequestId, o.DonorId }).IsUnique(); Cost: Insert/update slow, storage zyada.

Q5.25. Shadow properties kya hain?

Ans. Properties jo C# class me nahi hain, but database me hain. EF Core track karta hai. entity.Property<DateTime>("CreatedAt"); Use: Audit fields (CreatedBy, ModifiedAt), soft delete (IsDeleted), timestamp. Access: _context.Entry(donor).Property("CreatedAt").CurrentValue

Q5.26. Global query filters kya hain?

Ans. Har query pe automatically apply hone wale filters. entity.HasQueryFilter(d => !d.IsDeleted); Ab har _context.Donors query me soft-deleted records exclude. Use: Soft delete, multi-tenancy, IsActive flags.

Q5.27. EF Core me performance tips?

Ans. • AsNoTracking() read-only queries me • Include() se eager loading (N+1 se bachav) • Projection (Select) - sirf zaroori columns • Pagination - Skip/Take • Indexed columns pe filter • Async methods • Compiled queries (repeated queries) • Split queries for multiple Includes • Bulk operations (EFCore.BulkExtensions)

Q5.28. DbContext lifetime kya hona chahiye?

Ans. Scoped - ek HTTP request ke andar ek instance. builder.Services.AddDbContext<ApplicationDbContext>(opts => ...); Scoped default hai. Reason: • Change tracking per-request consistent • Connection pooling efficient • Thread-safe per request Transient: Multiple instances (rare). Singleton: Not recommended (thread safety, memory).

SECTION 6

Authentication & Authorization

Identity, cookies, claims, roles, CSRF, XSS, IDOR

Q6.1. Authentication vs Authorization?

Ans. Authentication: 'Aap kaun ho?' - Identity verify (login, password, MFA). Authorization: 'Aap kya kar sakte ho?' - Permissions check (role, policy). Order: Authentication pehle, phir authorization. Response codes: 401 Unauthorized (not authenticated), 403 Forbidden (authenticated but no permission).

Q6.2. ASP.NET Core Identity kya hai?

Ans. Built-in membership system - authentication + authorization. Features: • User registration, login, logout • Password hashing (PBKDF2) • Roles, claims, policies • Two-factor authentication • Account lockout • Email/phone confirmation • External logins (Google, Facebook) • Recovery codes Tables: AspNetUsers, AspNetRoles, AspNetUserRoles, AspNetUserClaims.

Q6.3. Identity me kaunsi tables banti hain?

Ans. • AspNetUsers - Users • AspNetRoles - Roles • AspNetUserRoles - Many-to-many user-role • AspNetUserClaims - User claims • AspNetRoleClaims - Role claims • AspNetUserLogins - External logins • AspNetUserTokens - 2FA tokens Humara AppUser : IdentityUser - custom fields add (IsBanned, CreatedAt).

Q6.4. Cookie authentication kaise kaam karta hai?

Ans. 1. Login POST - credentials verify. 2. Server encrypted cookie set karta hai (claims ke saath). 3. Cookie browser me store - name: .AspNetCore.Identity.Application. 4. Har request me browser cookie bhejta hai. 5. Authentication middleware cookie decrypt karta hai. 6. User.Identity populate hota hai. Cookie me: UserId, Roles, SecurityStamp, expiry.

Q6.5. Claims kya hote hain?

Ans. User ke baare me key-value pairs. Standard claims: NameIdentifier (UserId), Name, Email, Role. Example: var userId = User.FindFirstValue(ClaimTypes.NameIdentifier); var isAdmin = User.IsInRole("Admin"); Custom claims bhi ho sakte hain - jaise SubscriptionLevel, TenantId.

Q6.6. Roles kya hote hain?

Ans. Groups jisme users assign hote hain. Hum: SuperAdmin, Admin, User. Assign: await userManager.AddToRoleAsync(user, "Admin"); Check: [Authorize(Roles = "Admin,SuperAdmin")] In code: User.IsInRole("Admin")

Q6.7. Role vs Claim me difference?

Ans. Role: Predefined group. Boolean (member ya not). Simple authorization. Claim: Key-value pair. Flexible - koi bhi data (age, department, permission list). Policy-based: options.AddPolicy("MinimumAge18", p => p.RequireClaim("Age", ...)); Roles: Simple scenarios. Claims + policies: Complex rules.

Q6.8. Policy-based authorization?

Ans. Program.cs: builder.Services.AddAuthorization(options => { options.AddPolicy("CanDeleteDonor", p => p.RequireRole("Admin", "SuperAdmin")); options.AddPolicy("OwnerOnly", p => p.Requirements.Add(new OwnerRequirement())); }); Controller: [Authorize(Policy = "CanDeleteDonor")] Custom handler: public class OwnerHandler : AuthorizationHandler<OwnerRequirement> { ... }

Q6.9. [Authorize] attribute kaise kaam karta hai?

Ans. Filter hai. Action execute hone se pehle user check karta hai. [Authorize] - authenticated hona zaroori [Authorize(Roles = "Admin")] - specific role [Authorize(Policy = "CustomPolicy")] - policy [AllowAnonymous] - auth skip (jaise login page) Fail: 401 (not logged in) ya 403 (role nahi) - ya login page redirect.

Q6.10. Password hashing kaise hoti hai?

Ans. Plain text store NAHI. Hash + salt + iterations. ASP.NET Core Identity: PBKDF2 (Password-Based Key Derivation Function 2) • HMAC-SHA256 • 10,000+ iterations • 128-bit salt per password Verify: 1. Stored salt nikaalo 2. Same algorithm + iterations 3. Compare hash Database leak hone pe bhi password crack karna extremely slow (brute force impractical).

Q6.11. Lockout mechanism kya hai?

Ans. Brute-force attack se bachav. Kuch failed attempts ke baad account temporarily lock. Humara config: options.Lockout.MaxFailedAccessAttempts = 5; options.Lockout.DefaultLockoutTimeSpan = TimeSpan.FromMinutes(5); options.Lockout.AllowedForNewUsers = true; SignInManager.PasswordSignInAsync: result.IsLockedOut = true → lock Iske baad correct password bhi kaam nahi karega jab tak time khatam ho.

Q6.12. 2FA kya hai?

Ans. Two-Factor Authentication - do alag verification methods. Something you know: Password Something you have: Phone, hardware key Something you are: Biometric ASP.NET Core Identity 2FA support: • Authenticator app (TOTP) • Email verification • SMS • Recovery codes Humara project me 2FA nahi hai - roadmap me.

Q6.13. CSRF attack kya hai?

Ans. Cross-Site Request Forgery - attacker victim ke browser se unwanted action perform karwata hai. Scenario: 1. User logged in hai bank.com pe 2. Attacker ka site visit karta hai 3. Attacker ka page auto-submit karta hai form to bank.com/transfer 4. Browser automatically bank.com ka cookie bhejta hai 5. Server sochega legitimate request hai Defense: Anti-forgery token, SameSite cookies.

Q6.14. Anti-forgery token kaise kaam karta hai?

Ans. 1. Server unique token generate karta hai 2. Token form me hidden field me + cookie me 3. POST pe dono compare hote hain 4. Attacker cross-site se token + cookie dono match nahi kar sakta ASP.NET Core: Global filter: AutoValidateAntiforgeryTokenAttribute Form: <form asp-action> ya @Html.AntiForgeryToken() Hum globally apply kiya hai - har POST protected.

Q6.15. XSS kya hai?

Ans. Cross-Site Scripting - attacker malicious JavaScript inject karta hai page me. Types: • Stored - DB me save, phir render • Reflected - URL parameter se turant render • DOM-based - client-side JS Defense: • Output encoding (Razor default) • Input validation • Content Security Policy • HttpOnly cookies

Q6.16. SQL Injection kya hai?

Ans. Malicious SQL code input ke through inject. Vulnerable: "SELECT * FROM Users WHERE Name='" + input + "'" Input: ' OR '1'='1 Query: SELECT * FROM Users WHERE Name='' OR '1'='1' Defense: • Parameterized queries • EF Core (automatically parameterized) • Stored procedures • Input validation Hum EF Core use karte hain - safe.

Q6.17. IDOR kya hai?

Ans. Insecure Direct Object Reference - URL me ID change karke doosre user ka data access. Vulnerable: /Donor/Edit/5 → 6 karo → doosre ka data Defense: Server-side ownership check. Humara: var donor = await _context.Donors.FindAsync(id); if (donor.UserId != User.FindFirstValue(ClaimTypes.NameIdentifier)) return Forbid(); Edit me: Owner-only. Delete me: Owner ya Admin.

Q6.18. SameSite cookie kya hai?

Ans. Cookie cross-site requests me bheji jaaye ya nahi. Strict: Cross-site never Lax: Top-level GET navigation me bheje None: Always bheje (Secure required) Hum Lax use karte hain - CSRF protection + normal workflow (login redirect etc.).

Q6.19. HttpOnly cookie kya hai?

Ans. JavaScript se access nahi ho sakti. document.cookie me nahi dikhegi. Use: Session cookies ke liye - XSS se session hijack rokta hai. Humne true set kiya hai: options.Cookie.HttpOnly = true; Access: Sirf HTTP request ke through (server-side).

Q6.20. Secure cookie flag?

Ans. Cookie sirf HTTPS pe bheji jaaye. Production me true hona chahiye: options.Cookie.SecurePolicy = CookieSecurePolicy.Always; Development me SameAsRequest (HTTP localhost ke liye).

Q6.21. Cookie expiry kaise set karte hain?

Ans. options.ExpireTimeSpan = TimeSpan.FromDays(7); options.SlidingExpiration = true; SlidingExpiration = true: Active user ka cookie extend hota hai. 7 din me se half time baad next request pe refresh. Absolute expiration: SlidingExpiration = false - fixed time pe expire.

Q6.22. Cookie events kya hain?

Ans. Lifecycle hooks: options.Events.OnValidatePrincipal = async ctx => { // Har request pe var user = await userManager.GetUserAsync(ctx.Principal); if (user.IsBanned) { ctx.RejectPrincipal(); await ctx.HttpContext.SignOutAsync(); } }; OnSigningIn: Login pe OnSignedIn: Login done OnSigningOut: Logout pe OnRedirectToLogin: Unauth request pe

Q6.23. Banned user ko sign-out kaise karte hain?

Ans. OnValidatePrincipal event - har request pe check: options.Events.OnValidatePrincipal = async ctx => { var user = await userManager.GetUserAsync(ctx.Principal); if (user?.IsBanned == true) { ctx.RejectPrincipal(); await ctx.HttpContext.SignOutAsync(); return; } await SecurityStampValidator.ValidatePrincipalAsync(ctx); }; Existing session instant invalidate. Next request pe login page.

Q6.24. Identity me SignInManager kya karta hai?

Ans. Login/logout operations: var result = await _signInManager.PasswordSignInAsync(email, password, isPersistent, lockoutOnFailure: true); result.Succeeded result.IsLockedOut result.IsNotAllowed (email/phone not confirmed) result.RequiresTwoFactor await _signInManager.SignOutAsync(); await _signInManager.RefreshSignInAsync(user);

Q6.25. UserManager kya karta hai?

Ans. User CRUD operations: var user = await _userManager.FindByEmailAsync(email); var result = await _userManager.CreateAsync(user, password); await _userManager.AddToRoleAsync(user, "Admin"); await _userManager.RemoveFromRoleAsync(user, "Admin"); await _userManager.IsInRoleAsync(user, "Admin"); await _userManager.GetRolesAsync(user); await _userManager.SetLockoutEndDateAsync(user, date); await _userManager.UpdateAsync(user);

Q6.26. RoleManager kya karta hai?

Ans. Role CRUD: await _roleManager.CreateAsync(new IdentityRole("Admin")); await _roleManager.RoleExistsAsync("Admin"); await _roleManager.DeleteAsync(role); Role hierarchy enforce karne ke liye hum custom logic likhte hain - Identity me built-in nahi hai.

Q6.27. Security Stamp kya hai?

Ans. User record pe ek token. Password change, roles change, 2FA change pe update hota hai. Har request pe cookie me bhi stamp check hota hai. Mismatch pe user sign-out. Use: Agar admin user ka password reset kare, purana session invalid ho jaata hai.

Q6.28. 3-tier role hierarchy kaise banayi?

Ans. SuperAdmin > Admin > User. Rules: • SuperAdmin immutable (koi ban/demote/delete nahi) • Admin can ban User, but not Admin/SuperAdmin • Only SuperAdmin can promote User→Admin • Self-protection: Koi khud ko ban/delete nahi Implementation: AdminController me custom checks - har action me current user ka role + target user ka role verify.

Q6.29. SuperAdmin immutable kyun?

Ans. Design decision: • Root authority hai - agar ban ho gaya to system locked • Recovery impossible without manual DB intervention • Safety net - Admin malicious ho jaye to SuperAdmin override kar sakta hai Code: if (targetUserRoles.Contains("SuperAdmin") && !currentIsSuperAdmin) return Forbid(); Even SuperAdmin khud ko ban nahi kar sakta - self-protection.

SECTION 7

BloodConnect Project Overview

Entities, structure, seeder, controllers, tech stack

Q7.1. BloodConnect kya hai?

Ans. BloodConnect ek blood donation management platform hai jo donors aur patients ko real-time connect karta hai. Built on: ASP.NET Core 10 MVC + EF Core 10 + SQL Server + Identity. Core features: • Donor profiles with CRUD • Blood requests (guest-friendly) • Blood compatibility-aware search • Pledge-based donation workflow • Role-based admin panel • Cached unread message count

Q7.2. Kyun banaya yeh project?

Ans. Real-world problem: Blood donation me coordination nahi hoti. Emergency me donors dhoondhna mushkil. Compatible blood group ka concept log nahi samajhte. BloodConnect solve karta hai: • Centralized donor database • Automatic compatibility matching • Pledge-based trusted workflow • Role-based moderation Educational goal: Full-stack ASP.NET Core development demonstrate karna - MVC, EF Core, Identity, caching, REST API, role hierarchy.

Q7.3. Tech stack detail me?

Ans. Runtime: .NET 10 (net10.0) Web: ASP.NET Core 10 MVC ORM: EF Core 10 (Code First) DB: SQL Server Express Auth: ASP.NET Core Identity + Cookie Caching: IMemoryCache (60s TTL) UI: Bootstrap 5 + custom CSS Icons: Inline SVG (Lucide-style) Fonts: Inter (Google Fonts) Animations: CSS3 + Vanilla JS Audio: Web Audio API (procedural)

Q7.4. Project ka architecture?

Ans. Layered MVC architecture: Controllers/ - Request handle, model binding, view orchestration Models/ - Domain entities + ViewModels + BloodCompatibility helper Services/ - Cross-cutting (MessageCache) Views/ - Razor templates (presentation only) Migrations/ - EF Core DB migrations wwwroot/ - Static assets (css, js) Flow: Request → Routing → Controller → Model/DbContext → View → Response

Q7.5. Project ka folder structure?

Ans. BloodConnect/ ■■■ Controllers/ (7 files: Home, Donor, BloodRequest, DonorApi, Account, Admin, Dashboard) ■■■ Models/ (10 files: AppUser, Donor, BloodRequest, DonationOffer, ContactMessage, BloodCompatibility, DomainValues, ValidationAttributes, ApplicationDbContext, DbSeeder, ViewModels) ■■■ Services/ (MessageCache) ■■■ Migrations/ ■■■ Views/ ■ ■■■ Shared/ (_Layout, _LoginPartial, Error) ■ ■■■ Home/ (Index, About, Contact, FAQ, Privacy) ■ ■■■ Donor/ (Index, _Form, Create, Edit, Delete) ■ ■■■ BloodRequest/ (+ Offers) ■ ■■■ Account/ (Register, Login, AccessDenied) ■ ■■■ Admin/ (Index, Donors, Requests, UserList, Messages) ■ ■■■ Dashboard/ ■■■ wwwroot/css/ (site.css, animations.css) ■■■ wwwroot/js/ (site.js, animations.js) ■■■ appsettings.json ■■■ appsettings.Development.json ■■■ Program.cs

Q7.6. Domain entities kaunse hain?

Ans. • AppUser : IdentityUser - Extends IdentityUser, IsBanned field • Donor - Donor profile (Name, BloodGroup, Age 18-65, Location, Contact, UserId FK, RowVersion) • BloodRequest - Patient request (PatientName, BloodGroup, UrgencyLevel, Location, Contact, Status, CreatedAt) • DonationOffer - Pledge (RequestId, DonorId, Status Pledged/Accepted/Rejected/Cancelled/Completed) • ContactMessage - Contact form data (Name, Email, Subject, Message, IsRead)

Q7.7. AppUser kya hai?

Ans. public class AppUser : IdentityUser { public bool IsBanned { get; set; } public DateTime CreatedAt { get; set; } public Donor? Donor { get; set; } // Navigation } IdentityUser se inherit karke standard fields milte hain: Id, Email, PasswordHash, PhoneNumber, LockoutEnd, etc. Custom fields: • IsBanned - banned users sign-out • CreatedAt - audit • Donor - navigation to donor profile

Q7.8. Donor entity detail me?

Ans. public class Donor { public int Id { get; set; } public string Name { get; set; } public string BloodGroup { get; set; } public int Age { get; set; } public string Location { get; set; } public string ContactNumber { get; set; } public bool IsAvailable { get; set; } public string UserId { get; set; } // FK public AppUser User { get; set; } public byte[] RowVersion { get; set; } // Concurrency public DateTime CreatedAt { get; set; } } Validation: Age [Range(18,65)], BloodGroup regex, Contact regex.

Q7.9. BloodRequest entity detail me?

Ans. public class BloodRequest { public int Id { get; set; } public string PatientName { get; set; } public string BloodGroup { get; set; } public string UrgencyLevel { get; set; } // Critical/High/Medium/Low public string Location { get; set; } public string ContactNumber { get; set; } public string? Description { get; set; } public RequestStatus Status { get; set; } // Active/InProgress/Fulfilled/Cancelled public string? UserId { get; set; } // Optional (guest) public DateTime CreatedAt { get; set; } public ICollection<DonationOffer> Offers { get; set; } } Guest-friendly - UserId null ho sakta hai.

Q7.10. DonationOffer entity detail me?

Ans. public class DonationOffer { public int Id { get; set; } public int RequestId { get; set; } public BloodRequest Request { get; set; } public int DonorId { get; set; } public Donor Donor { get; set; } public OfferStatus Status { get; set; } // Pledged/Accepted/Rejected/Cancelled/Completed public DateTime CreatedAt { get; set; } public DateTime? AcceptedAt { get; set; } public DateTime? CompletedAt { get; set; } } Unique index: (RequestId, DonorId) - duplicate offers rokta hai.

Q7.11. ContactMessage entity?

Ans. public class ContactMessage { public int Id { get; set; } public string Name { get; set; } public string Email { get; set; } public string Subject { get; set; } public string Message { get; set; } public bool IsRead { get; set; } public DateTime CreatedAt { get; set; } } Admin panel me inbox - read/unread badge cached (60s TTL).

Q7.12. BloodCompatibility.cs kya hai?

Ans. Static helper class - medical compatibility table encode karta hai. public static class BloodCompatibility { public static IReadOnlyList<string> GetCompatibleDonorGroups(string recipientGroup) { return recipientGroup switch { "O-" => new[] { "O-" }, "O+" => new[] { "O-", "O+" }, "A-" => new[] { "O-", "A-" }, "A+" => new[] { "O-", "O+", "A-", "A+" }, "B-" => new[] { "O-", "B-" }, "B+" => new[] { "O-", "O+", "B-", "B+" }, "AB-" => new[] { "O-", "A-", "B-", "AB-" }, "AB+" => new[] { "O-", "O+", "A-", "A+", "B-", "B+", "AB-", "AB+" }, _ => Array.Empty<string>() }; } }

Q7.13. DomainValues.cs kya hai?

Ans. Constants class - magic strings avoid karne ke liye. public static class BloodGroups { public const string O_Neg = "O-"; public const string O_Pos = "O+"; public const string A_Neg = "A-"; // ... etc public static readonly string[] All = { ... }; } public static class Roles { public const string SuperAdmin = "SuperAdmin"; public const string Admin = "Admin"; public const string User = "User"; } Use: Type-safe, refactor-friendly, IntelliSense.

Q7.14. ValidationAttributes.cs me kya hai?

Ans. Custom validation attributes. MustBeTrueAttribute - checkbox consent ke liye: public class MustBeTrueAttribute : ValidationAttribute { public override bool IsValid(object value) { return value is bool b && b; } } Use: public class RegisterViewModel { [MustBeTrue(ErrorMessage = "You must accept terms")] public bool AcceptTerms { get; set; } }

Q7.15. ApplicationDbContext kaise define kiya?

Ans. public class ApplicationDbContext : IdentityDbContext<AppUser> { public DbSet<Donor> Donors { get; set; } public DbSet<BloodRequest> BloodRequests { get; set; } public DbSet<DonationOffer> DonationOffers { get; set; } public DbSet<ContactMessage> ContactMessages { get; set; } public ApplicationDbContext(DbContextOptions<ApplicationDbContext> opts) : base(opts) { } protected override void OnModelCreating(ModelBuilder mb) { base.OnModelCreating(mb); // Unique index mb.Entity<DonationOffer>() .HasIndex(o => new { o.RequestId, o.DonorId }) .IsUnique(); // FK indexes mb.Entity<DonationOffer>().HasIndex(o => o.RequestId); mb.Entity<Donor>().HasIndex(d => d.UserId); } }

Q7.16. DbSeeder kya karta hai?

Ans. Application start pe seed data insert. public static class DbSeeder { public static async Task SeedAsync(IServiceProvider sp) { var roleMgr = sp.GetRequiredService<RoleManager<IdentityRole>>(); var userMgr = sp.GetRequiredService<UserManager<AppUser>>(); // Roles foreach (var role in new[] { "SuperAdmin", "Admin", "User" }) if (!await roleMgr.RoleExistsAsync(role)) await roleMgr.CreateAsync(new IdentityRole(role)); // Admin user if (await userMgr.FindByEmailAsync("admin@bloodconnect.com") == null) { var admin = new AppUser { Email = "...", UserName = "..." }; await userMgr.CreateAsync(admin, "Admin@123"); await userMgr.AddToRoleAsync(admin, "SuperAdmin"); await userMgr.AddToRoleAsync(admin, "Admin"); } } } Idempotent - multiple restarts pe duplicate nahi.

Q7.17. Program.cs me kya configure hota hai?

Ans. 1. DbContext register + SQL Server + retry 2. Identity + cookie config 3. Authorization 4. MVC + global anti-forgery filter 5. IMemoryCache + MessageCache singleton 6. Exception handling + status code pages 7. Middleware pipeline (auth, routing, static files) 8. DbSeeder call Kisi bhi ASP.NET Core app ka entry point yahi hai - service registration + middleware pipeline.

Q7.18. Kis controller me kya hai?

Ans. • HomeController - Home, About, FAQ, Contact, Error, Privacy • AccountController - Register, Login, Logout, AccessDenied • DonorController - Index, Search, Create, Edit, Delete, Details • BloodRequestController - Index, Create, Details, Offer, Accept, Reject, Complete, Cancel • DonorApiController - Search (REST API) • AdminController - Index, Donors, Requests, UserList, Ban, Unban, Promote, Demote, Delete, Messages • DashboardController - Index (user dashboard)

Q7.19. Search feature kaise kaam karta hai?

Ans. 1. User blood group + location daalta hai. 2. DonorController.Search(bloodGroup, location, exactMatch, page) 3. BloodCompatibility.GetCompatibleDonorGroups(bloodGroup) se compatible list 4. _context.Donors.Where(d => compatibleGroups.Contains(d.BloodGroup)) 5. Location filter (case-insensitive substring) 6. OrderByDescending(exact match) - exact matches pehle 7. Pagination - Skip/Take 8. AsNoTracking - read-only performance 9. View me render ya JSON (API me)

Q7.20. Blood request kaise create hoti hai?

Ans. 1. /BloodRequest/Create - GET form 2. [AllowAnonymous] - guest bhi bhej sakta hai 3. Form: PatientName, BloodGroup, UrgencyLevel, Location, ContactNumber, Description 4. POST me ModelState check 5. Naya BloodRequest entity 6. Status = Active 7. UserId = current user (agar logged in) ya null 8. SaveChanges 9. Detail page pe redirect + toast notification

Q7.21. Pledge workflow kaise implement kiya?

Ans. 1. Donor views request, clicks 'I can donate' 2. POST /BloodRequest/Offer(requestId) 3. Checks: - Donor logged in - Donor ka profile exists - Donor != request owner - Blood group compatible (server-side) - Request not fulfilled 4. DonationOffer create - Status = Pledged 5. Unique index (RequestId, DonorId) - duplicate prevent 6. Toast + redirect

Q7.22. Offer accept kaise kaam karta hai?

Ans. POST /BloodRequest/Accept(offerId) 1. Offer fetch with Request + Donor (Include) 2. Authorization: current user == request.UserId? 3. Offer.Status = Accepted 4. AcceptedAt = now 5. Same request ke baaki offers: baaki.Status = Rejected 6. Request.Status = InProgress 7. SaveChanges (single transaction - atomic) Contact details dono ko reveal - view me visible.

Q7.23. Complete kaise hota hai?

Ans. POST /BloodRequest/Complete(offerId) 1. Offer fetch 2. Authorization: Current user == Request.UserId OR Current user == Offer.Donor.UserId 3. Offer.Status = Completed 4. CompletedAt = now 5. Request.Status = Fulfilled 6. SaveChanges Flexibility - owner ya accepted donor, dono mark kar sakte hain.

Q7.24. Auto-reject kaise hota hai?

Ans. Accept method me: var offer = await _context.DonationOffers .Include(o => o.Request) .FirstOrDefaultAsync(o => o.Id == offerId); offer.Status = OfferStatus.Accepted; offer.AcceptedAt = DateTime.UtcNow; offer.Request.Status = RequestStatus.InProgress; // Auto-reject baaki var others = await _context.DonationOffers .Where(o => o.RequestId == offer.RequestId && o.Id != offerId && o.Status == OfferStatus.Pledged) .ToListAsync(); foreach (var o in others) o.Status = OfferStatus.Rejected; await _context.SaveChangesAsync(); // Single transaction - all or nothing

Q7.25. Contact details kab reveal hote hain?

Ans. Sirf offer accepted hone ke baad. View me: @if (Model.Offer.Status == OfferStatus.Accepted || Model.Offer.Status == OfferStatus.Completed) { Owner: @Model.Request.ContactNumber Donor: @Model.Offer.Donor.ContactNumber } Search results me contact nahi - spam se bachav, privacy by design.

Q7.26. Admin panel me kya kya hai?

Ans. • Dashboard - stats (donors, requests, users, messages) • Donors - list, delete (edit nahi - owner-only) • Requests - list, delete • UserList - saare users + roles • Ban/Unban - role hierarchy ke saath • Promote/Demote - Admin/User roles • Delete user - SuperAdmin only for Admins • Messages - inbox, read/unread, delete Har action server-side authorized - role hierarchy enforced.

Q7.27. Ban system kaise kaam karta hai?

Ans. 1. Admin ban action trigger karta hai 2. Server checks: role hierarchy (Admin can't ban Admin/SuperAdmin) 3. Self-protection (can't ban self) 4. user.IsBanned = true 5. SaveChanges 6. Banned user: - Next request pe OnValidatePrincipal event - RejectPrincipal + SignOut - Login attempt blocked with suspension message 7. Unban reverses IsBanned = false

Q7.28. Search API ka response kya hai?

Ans. GET /api/donor/search?bloodGroup=A%2B&location=Gujarat&exactMatch=false&page=1&pageSize=20 { "searchMode": "Compatible", "compatibleGroups": ["O-", "O+", "A-", "A+"], "isExactMatch": false, "page": 1, "pageSize": 20, "totalCount": 14, "items": [{ "donorId": 3, "name": "Ravi Kumar", "bloodGroup": "O+", "location": "Gujarat", "age": 27, "isAvailable": true, "isExactMatch": false }] } Contact number intentionally omit.

Q7.29. MessageCache kya hai aur kyun?

Ans. Services/MessageCache.cs - wraps IMemoryCache. public class MessageCache { private const string Key = "UnreadMessages"; private readonly IMemoryCache _cache; public async Task<int> GetUnreadCountAsync(AppDbContext db) { if (_cache.TryGetValue(Key, out int count)) return count; count = await db.ContactMessages.CountAsync(m => !m.IsRead); _cache.Set(Key, count, TimeSpan.FromSeconds(60)); return count; } public void Invalidate() => _cache.Remove(Key); } Kyun: Admin layout me unread badge HAR page pe render. Bina cache har request pe DB query - unnecessary load. 60s TTL + write invalidation = balance.

Q7.30. Cache invalidation kahan hoti hai?

Ans. Teen jagah: 1. ContactMessage create (ContactController.Create pe save ke baad) 2. Mark as read (AdminController.MarkRead) 3. Delete message (AdminController.DeleteMessage) Sab me _messageCache.Invalidate() call. Iske bina: Admin ko 60s tak purani count dikhegi after new message. Non-admin users: Zero overhead - cache code path hi nahi chalta.

Q7.31. Search me exact matches pehle kaise aate hain?

Ans. var ordered = donors .OrderByDescending(d => d.BloodGroup == requestedGroup) .ThenBy(d => d.BloodGroup) .ThenBy(d => d.Name); OrderByDescending(bool): true (1) pehle, false (0) baad me. Result: Exact match group pehle, phir other compatible groups alphabetically, phir name se sort. Reason: Donor search me user ko exact match upar dikhana UX ke liye better.

Q7.32. RowVersion kaise configure kiya?

Ans. Donor entity me: public byte[] RowVersion { get; set; } Fluent API: mb.Entity<Donor>() .Property(d => d.RowVersion) .IsRowVersion(); SQL Server rowversion column ban jaata hai - auto-increment on update. Edit me: _context.Update(donor); // RowVersion original value se check try { await _context.SaveChangesAsync(); } catch (DbUpdateConcurrencyException) { // Conflict - user ko message }

Q7.33. Global anti-forgery kaise enable kiya?

Ans. Program.cs: builder.Services.AddControllersWithViews(options => { options.Filters.Add(new AutoValidateAntiforgeryTokenAttribute()); }); Ab har POST/PUT/DELETE automatically validate. GET safe hai (idempotent). Form me: <form asp-action="Create"> - Tag helper auto token inject Ya @Html.AntiForgeryToken() API me [IgnoreAntiforgeryToken] agar JWT use kar rahe ho.

Q7.34. Styled error pages kaise implement kiye?

Ans. Program.cs: if (!app.Environment.IsDevelopment()) app.UseExceptionHandler("/Home/Error"); app.UseStatusCodePagesWithReExecute("/Home/Error", "?code={0}"); HomeController.Error(int? code): ViewBag.Code = code ?? 500; return View(); Error.cshtml me branded design, code-specific message. Production me no stack traces - sirf friendly message.

Q7.35. Age validation 18-65 kyun?

Ans. Medical guidelines: • Minimum 18 - legal consent, safe blood donation, weight criteria usually met • Maximum 65 - older donors me health risks, BP/medication issues [Range(18, 65, ErrorMessage = "Age must be 18-65")] public int Age { get; set; } Client-side jQuery validation + server-side same rule. Direct API call pe bhi server-side reject.

Q7.36. Blood group validation?

Ans. [RegularExpression(@"^(A|B|AB|O)[+-]$", ErrorMessage = "Invalid blood group")] public string BloodGroup { get; set; } Valid: A+, A-, B+, B-, AB+, AB-, O+, O- DomainValues.BloodGroups.All me list bhi rakhi hai - dropdown ke liye. Server-side + client-side dono validate.

Q7.37. Guest request kaise implement ki?

Ans. BloodRequestController.Create me [AllowAnonymous]. Reason: Emergency me koi register karne ka time nahi. Guest bhi request bhej sake. Form fields: Patient name, contact, blood group, urgency, location, description. UserId = null for guest. Admin panel me guest requests bhi list hoti hain - deletion bhi possible. Note: Security - rate limiting roadmap me hai.

SECTION 8

Blood Compatibility Domain

Universal donor, receiver, RBC table, validation

Q8.1. Blood groups kitne hain?

Ans. 8 main groups: A+, A-, B+, B-, AB+, AB-, O+, O- ABO system (A, B, AB, O) + Rh factor (+, -). Aur rare: Bombay blood group (hh phenotype) - ABO system me O jaisa, but anti-H antibody. India me distribution (approx): O+ 35%, B+ 30%, A+ 22%, AB+ 7%, O- 2%, B- 1.5%, A- 1%, AB- 0.5%.

Q8.2. Universal donor kaun?

Ans. O- (O negative). Reason: RBC pe koi A, B, ya Rh antigen nahi. Recipient ke antibodies (anti-A, anti-B, anti-D) react nahi karenge. Isliye O- blood kisi bhi patient ko diya ja sakta hai. Emergency me unknown blood group ke patient ko O- diya jaata hai. Note: Yeh RBC compatibility hai. Plasma compatibility different hai - AB plasma universal donor hai plasma ke liye.

Q8.3. Universal receiver kaun?

Ans. AB+ (AB positive). Reason: Plasma me anti-A, anti-B, anti-D antibodies nahi hote. RBC pe A, B, aur D antigens already hote hain. Isliye AB+ kisi bhi blood group se receive kar sakta hai. Plasma ke hisaab se: AB universal plasma donor hai (AB plasma me A, B antibodies nahi - kisi bhi recipient ko safe).

Q8.4. RBC compatibility table?

Ans. Recipient → Can receive from: O- → O- O+ → O-, O+ A- → O-, A- A+ → O-, O+, A-, A+ B- → O-, B- B+ → O-, O+, B-, B+ AB- → O-, A-, B-, AB- AB+ → All 8 groups Rule: Recipient ke antibodies donor ke antigens se react nahi karne chahiye. O- recipient: anti-A, anti-B, anti-D hai → sirf O- safe. AB+ recipient: koi antibody nahi → sab safe.

Q8.5. Anti-A, anti-B antibodies kya hote hain?

Ans. IgM antibodies jo plasma me naturally hote hain. Blood group A → anti-B antibodies (B attack karenge) Blood group B → anti-A antibodies Blood group O → anti-A + anti-B Blood group AB → koi nahi Isliye: • A recipient ko B blood nahi (anti-B react) • B recipient ko A blood nahi (anti-A react) • O recipient ko sirf O blood (both antibodies) • AB recipient ko koi bhi blood (no antibodies)

Q8.6. Rh factor kya hai?

Ans. Rhesus factor - D antigen RBC pe. Rh+ → D antigen present Rh- → D antigen absent Rh- person me anti-D antibody naturally nahi hoti - but Rh+ blood exposure ke baad banti hai. Isliye: • Rh- ko Rh+ blood ek baar de sakte hain, but subsequent exposure me reaction • Rh- ko sirf Rh- blood dena chahiye (safety)

Q8.7. BloodCompatibility.cs kaise encode kiya?

Ans. Static lookup - switch expression: public static IReadOnlyList<string> GetCompatibleDonorGroups(string recipient) => recipient switch { "O-" => new[] { "O-" }, "O+" => new[] { "O-", "O+" }, "A-" => new[] { "O-", "A-" }, "A+" => new[] { "O-", "O+", "A-", "A+" }, "B-" => new[] { "O-", "B-" }, "B+" => new[] { "O-", "O+", "B-", "B+" }, "AB-" => new[] { "O-", "A-", "B-", "AB-" }, "AB+" => new[] { "O-", "O+", "A-", "A+", "B-", "B+", "AB-", "AB+" }, _ => Array.Empty<string>() }; Unit-testable, no side effects, pure function.

Q8.8. Search me compatibility kaise use hoti hai?

Ans. 1. User blood group daalta hai (jaise A+) 2. BloodCompatibility.GetCompatibleDonorGroups("A+") → ["O-", "O+", "A-", "A+"] 3. Query: var donors = await _context.Donors .AsNoTracking() .Where(d => compatibleGroups.Contains(d.BloodGroup)) .Where(d => d.IsAvailable) .ToListAsync(); 4. OrderByDescending(exact match) 5. Pagination 6. Return

Q8.9. Exact vs compatible match toggle?

Ans. Default: Compatible mode. ?exactMatch=true → sirf same group (A+ ke liye sirf A+ donors). UI toggle: Radio button ya switch. Server-side: if (exactMatch) groups = new[] { bloodGroup }; else groups = BloodCompatibility.GetCompatibleDonorGroups(bloodGroup); Response me isExactMatch flag bhi bhejte hain.

Q8.10. Server-side compatibility validation kyun?

Ans. Agar sirf UI pe validation ho, koi: • DevTools se • Direct API call se • Custom client bana ke incompatible offer bhej sakta hai. Hum BloodRequestController.Offer me check karte hain: var compatible = BloodCompatibility.GetCompatibleDonorGroups(request.BloodGroup); if (!compatible.Contains(donor.BloodGroup)) return BadRequest("Incompatible blood group"); Yeh defense in depth hai.

Q8.11. Compatibility medically kyun important?

Ans. Incompatible blood transfusion → hemolytic reaction: 1. Recipient ke antibodies donor RBC attack karte hain 2. RBC lysis (destruction) - hemoglobin release 3. Symptoms: Fever, chills, back pain, dark urine 4. Complications: Kidney failure (hemoglobinuria), DIC, shock 5. Kabhi kabhi fatal (5-10% mortality) Isliye critical hai - hospital me cross-matching hoti hai transfusion se pehle. BloodConnect me hum search aur offer validation me encode karte hain.

Q8.12. Viva me compatibility kaise explain karoge?

Ans. Simple answer: 'Sir, RBC compatibility ka rule hai. Recipient plasma me antibodies hote hain - A group me anti-B, B me anti-A, O me dono, AB me koi nahi. Donor RBC pe antigens hote hain - A, B, Rh. Agar donor antigen recipient antibody se match kare, reaction hoti hai. Isliye: • O- (no antigen) universal donor • AB+ (no antibody) universal receiver Hum isko BloodCompatibility.cs me static switch expression se encode karte hain. Search endpoint aur offer validation, dono me use karte hain. Server-side validate karte hain, sirf UI pe nahi.'

Q8.13. Kya hum plasma compatibility bhi handle karte hain?

Ans. Nahi - yeh scope ke bahar hai. BloodConnect RBC-based compatibility use karta hai (whole blood / RBC transfusion). Plasma compatibility different: • AB plasma universal donor (no A, B antibodies) • O plasma universal recipient Real hospital me component-wise (RBC, plasma, platelets) different rules. Educational project me simplified.

Q8.14. Kitne donors ek blood group ke ho sakte hain?

Ans. Koi limit nahi - 1 se infinite. Search me pagination hai: • Default pageSize = 20 • Max 100 per page • Response me totalCount Multiple pages - Skip((page-1)*size).Take(size) Performance: Indexed columns (BloodGroup, Location) pe filter fast.

Q8.15. Agar blood group empty ho ya invalid?

Ans. Two-layer defense: 1. Model validation: [RegularExpression(@"^(A|B|AB|O)[+-]$")] 2. BloodCompatibility helper: Default case → empty array → no results Search endpoint invalid group pe: {"totalCount": 0, "items": []} Ya 400 Bad Request (depending on config).

Q8.16. Location filter kaise kaam karta hai?

Ans. Case-insensitive substring match: if (!string.IsNullOrEmpty(location)) query = query.Where(d => d.Location.ToLower().Contains(location.ToLower())); Better (EF Core translates): query = query.Where(d => EF.Functions.Like(d.Location, $"%{location}%")); Response me matching donors - city, state, region sab kaam karega agar location string me included ho.

SECTION 9

Donation Workflow Deep Dive

Pledge state machine, auto-reject, contact reveal, edge cases

Q9.1. Pledge-based workflow kya hai?

Ans. Traditional: donor directly fulfill mark kare → fake ho sakta hai. Humara: pledge (intent) → owner accept → completion. Steps: 1. Donor 'I can donate' click - DonationOffer create (Status=Pledged) 2. Request owner offer accept kare - Status=Accepted, baaki auto-Reject 3. Contact details reveal 4. Donation actual ho 5. Owner ya donor complete mark - Status=Completed, Request.Status=Fulfilled Fayde: Audit trail, fake fulfillment rok, choice for owner.

Q9.2. State machine ke states?

Ans. DonationOffer states: • Pledged - initial • Accepted - owner accepted • Rejected - owner rejected OR auto-rejected • Cancelled - donor ne cancel (sirf Pledged se) • Completed - donation done (sirf Accepted se) Request states: • Active - open for offers • InProgress - ek offer accepted • Fulfilled - donation complete • Cancelled - owner cancel Transitions define hain - invalid transitions reject.

Q9.3. State transitions kaun kaun valid hain?

Ans. Pledged → Accepted (owner accepts) Pledged → Rejected (owner rejects) Pledged → Cancelled (donor cancels) Accepted → Completed (owner/donor marks) Accepted → Rejected (owner cancels after accept - edge case) Invalid: Completed → any (terminal) Rejected → any (terminal) Cancelled → any (terminal) Controller me state checks server-side - invalid pe 400.

Q9.4. Pledged se Accepted kab hota hai?

Ans. Jab request owner offer accept karta hai. Controller: [HttpPost] [Authorize] public async Task<IActionResult> Accept(int offerId) { var offer = await _context.DonationOffers .Include(o => o.Request) .FirstOrDefaultAsync(o => o.Id == offerId); if (offer == null) return NotFound(); if (offer.Request.UserId != CurrentUserId) return Forbid(); if (offer.Request.Status == Fulfilled) return BadRequest(); offer.Status = Accepted; offer.AcceptedAt = DateTime.UtcNow; offer.Request.Status = InProgress; // auto-reject others... await _context.SaveChangesAsync(); }

Q9.5. Accepted se Completed kab?

Ans. Owner ya accepted donor mark kar sakta hai. Controller: [HttpPost] public async Task<IActionResult> Complete(int offerId) { var offer = await _context.DonationOffers .Include(o => o.Request).ThenInclude(r => r.UserId) .Include(o => o.Donor) .FirstOrDefaultAsync(o => o.Id == offerId); var isOwner = offer.Request.UserId == CurrentUserId; var isDonor = offer.Donor.UserId == CurrentUserId; if (!isOwner && !isDonor) return Forbid(); if (offer.Status != Accepted) return BadRequest(); offer.Status = Completed; offer.CompletedAt = DateTime.UtcNow; offer.Request.Status = Fulfilled; await _context.SaveChangesAsync(); }

Q9.6. Rejected vs Cancelled?

Ans. Rejected: Owner ne reject kiya, ya auto-reject hua (doosra offer accept hua). Cancelled: Donor ne khud cancel kiya (sirf Pledged state se). Distinction: • Rejected - owner-initiated ya system • Cancelled - donor-initiated Audit trail me pata chalta hai kaun initiate kiya - analytics, dispute resolution me useful.

Q9.7. Duplicate offers kaise rokte hain?

Ans. Unique index (RequestId, DonorId): mb.Entity<DonationOffer>() .HasIndex(o => new { o.RequestId, o.DonorId }) .IsUnique(); Insert attempt duplicate pe: DbUpdateException. Controller me pre-check bhi: var existing = await _context.DonationOffers .AnyAsync(o => o.RequestId == requestId && o.DonorId == donorId); if (existing) return BadRequest("Already offered"); Two layers - UX friendly + DB integrity.

Q9.8. Self-offer kaise rokte hain?

Ans. Server-side check: var request = await _context.BloodRequests.FindAsync(requestId); if (request.UserId == CurrentUserId) { return BadRequest("You cannot offer on your own request"); } UI pe bhi button hide karte hain: @if (Model.Request.UserId != User.FindFirstValue(ClaimTypes.NameIdentifier)) { <button>I can donate</button> } Server-side primary defense hai - bypass nahi ho sakta.

Q9.9. Auto-reject kaise kaam karta hai?

Ans. Jab ek offer accept hota hai: var others = await _context.DonationOffers .Where(o => o.RequestId == offer.RequestId && o.Id != offer.Id && o.Status == OfferStatus.Pledged) .ToListAsync(); foreach (var o in others) o.Status = OfferStatus.Rejected; await _context.SaveChangesAsync(); Sab ek hi transaction me - atomicity. Agar beech me error aaye, koi change apply nahi hota.

Q9.10. Contact details kab reveal hote hain?

Ans. Sirf Accepted ya Completed status pe. View me: @if (Model.Offer.Status == OfferStatus.Accepted || Model.Offer.Status == OfferStatus.Completed) { Request owner: @Model.Request.ContactNumber Donor: @Model.Offer.Donor.ContactNumber } Search results aur Pledged state me contact nahi - privacy by design.

Q9.11. Fulfilled request pe naya offer?

Ans. Controller check: if (request.Status == RequestStatus.Fulfilled) return BadRequest("Request already fulfilled"); if (request.Status == RequestStatus.Cancelled) return BadRequest("Request cancelled"); UI me button bhi hide: @if (Model.Request.Status == RequestStatus.Active) { <button>I can donate</button> } Yeh data integrity + UX dono ke liye.

Q9.12. Kitne offers ek request pe ho sakte hain?

Ans. Unlimited - jitne compatible donors pledge karte hain. But practical: usually 3-10. Owner ek ko accept karta hai, baaki auto-reject. Multiple offers ka fayda: Owner ko choice - location, contact, response time dekh kar choose kar sakta hai. Unique constraint: ek donor ek request pe ek hi offer - (RequestId, DonorId) unique.

Q9.13. Request cancel kaise hoti hai?

Ans. Owner cancel kar sakta hai (Active state me): if (request.UserId != CurrentUserId) return Forbid(); if (request.Status != Active) return BadRequest(); request.Status = Cancelled; Saare pending offers bhi cancel: var offers = await _context.DonationOffers .Where(o => o.RequestId == request.Id && o.Status == Pledged) .ToListAsync(); foreach (var o in offers) o.Status = Cancelled; await _context.SaveChangesAsync();

Q9.14. Offer cancel karne ke rules?

Ans. Donor apna offer cancel kar sakta hai sirf Pledged state me: if (offer.Donor.UserId != CurrentUserId) return Forbid(); if (offer.Status != Pledged) return BadRequest("Cannot cancel after acceptance"); offer.Status = Cancelled; await _context.SaveChangesAsync(); Accepted hone ke baad - owner se coordinate karna padega (medical commitment serious hai).

Q9.15. Owner kitne offers accept kar sakta hai?

Ans. Sirf ek - per request. Reason: • Ek donor ka blood sufficient hota hai (usually 350-450ml) • Multiple acceptance confusing hoga • Tracking simple Auto-reject baaki offers ko Rejected set karta hai jaise hi ek accept hota hai.

Q9.16. Workflow me audit trail kaise maintain hota hai?

Ans. Har state change pe timestamp: • CreatedAt - offer pledge time • AcceptedAt - accept time • CompletedAt - complete time Status field bhi persistent - pata chalta hai final state kya tha. Request pe bhi: • CreatedAt • Status (Active → InProgress → Fulfilled) Analytics: Kitne pledges average, kitne accept hote, kitne complete hote - sab track ho sakta hai.

Q9.17. Agar donor delete ho jaaye after accept?

Ans. Currently cascade delete nahi hai Donor pe. Scenario: 1. Offer Accepted 2. Donor record delete 3. Offer reference dangling Prevention: Delete se pehle check - active offers hain? To warn. Roadmap: Soft delete (IsDeleted flag) - data preserve, queries filter. Abhi: Delete karne pe offer bhi delete (cascade) - but yeh ideal nahi, audit trail lose.

Q9.18. Workflow me concurrency issue kaise handle?

Ans. Do owners same offer accept karein (impossible - ek owner hota hai). Real concurrency: 1. Two requests same offer accept - rare (single owner). 2. Owner accept + donor cancel simultaneously. Solution: • RowVersion on DonationOffer (roadmap) • OR pessimistic check before update Abhi: Fast operations + unique constraints. Race conditions theoretically possible but rare in practice.

SECTION 10

REST API Deep Dive

Endpoints, pagination, error handling, versioning, Postman

Q10.1. REST API kya hai?

Ans. REST (Representational State Transfer) - architectural style for APIs. Principles: • Client-server separation • Stateless • Cacheable • Uniform interface • Layered • Resource-oriented HTTP methods = operations, URLs = resources, JSON/XML = representation. Humara: GET /api/donor/search - REST endpoint.

Q10.2. API me yeh endpoint kya karta hai?

Ans. GET /api/donor/search Purpose: Compatibility-aware donor search. Query params: • bloodGroup (string) - e.g. A+ • location (string) - e.g. Gujarat • exactMatch (bool) - true/false • page (int, default 1) • pageSize (int, default 20, max 100) Response: JSON with pagination + compatible groups. Contact numbers omitted - privacy.

Q10.3. Search API ka response shape?

Ans. { "searchMode": "Compatible", "compatibleGroups": ["O-", "O+", "A-", "A+"], "isExactMatch": false, "page": 1, "pageSize": 20, "totalCount": 14, "items": [{ "donorId": 3, "name": "Ravi Kumar", "bloodGroup": "O+", "location": "Gujarat", "age": 27, "isAvailable": true, "isExactMatch": false }] } Structured, paginated, metadata included.

Q10.4. Pagination kaise implement ki?

Ans. Query params: • page (1-based, default 1) • pageSize (default 20, max 100) EF Core: var total = await query.CountAsync(); var items = await query .Skip((page - 1) * pageSize) .Take(pageSize) .ToListAsync(); Response me: { "page": 1, "pageSize": 20, "totalCount": 14, "items": [...] } Client total pages nikaal sakta hai: ceil(totalCount / pageSize).

Q10.5. AsNoTracking kyun use kiya search me?

Ans. Read-only query. Results DTO me map karte hain - entity update nahi karni. AsNoTracking() se: • Change tracker me entity store nahi • Memory kam • Faster (~30-50% improvement) Donors search me: var donors = await _context.Donors .AsNoTracking() .Where(...) .ToListAsync(); Edit form me AsNoTracking nahi - tracking chahiye update ke liye.

Q10.6. Ordering kaise karte hain?

Ans. var ordered = donors .OrderByDescending(d => d.BloodGroup == requestedGroup) .ThenBy(d => d.BloodGroup) .ThenBy(d => d.Name); OrderByDescending(bool): • true (exact match) = 1 → pehle • false = 0 → baad me Reason: User ko exact match upar dikhana UX-friendly. SQL me: ORDER BY CASE WHEN BloodGroup = @g THEN 1 ELSE 0 END DESC, BloodGroup, Name

Q10.7. API vs MVC controller?

Ans. MVC Controller: HTML views return karta hai, Razor render. public class DonorController : Controller API Controller: JSON/data return karta hai. public class DonorApiController : ControllerBase Difference: • Controller - View(), RedirectToAction() • ControllerBase - Ok(), BadRequest(), NotFound() Hum DonorApiController use karte hain API ke liye - lighter, no view overhead.

Q10.8. IActionResult vs ActionResult<T>?

Ans. IActionResult: Status code return (200, 404, 400). No strong typing. public IActionResult Get() { return Ok(data); } ActionResult<T>: Strong typing + status codes. public ActionResult<DonorDto> Get() { if (notFound) return NotFound(); return data; // implicit Ok } Fayde: OpenAPI/Swagger me better documentation, type safety.

Q10.9. API authentication kaise karte hain?

Ans. Currently: Anonymous search (public data). Roadmap: JWT Bearer tokens. [Authorize] [HttpGet("profile")] public IActionResult Profile() { ... } Client login → token → Authorization: Bearer {token} Other options: • API Keys (simple, less secure) • OAuth 2.0 (third-party access) • Cookie auth (same-origin SPA)

Q10.10. API versioning?

Ans. Methods: 1. URL: /api/v1/donor/search 2. Query: /api/donor/search?api-version=1.0 3. Header: X-API-Version: 1 4. Media type: Accept: application/vnd.bloodconnect.v1+json Currently version 1 (implicit). Future me URL-based add karenge jab breaking changes aayein. Package: Asp.Versioning.Mvc

Q10.11. Postman me API test kaise kare?

Ans. 1. New Request → GET 2. URL: http://localhost:5006/api/donor/search?bloodGroup=A%2B&location=Gujarat&page=1&pageSize=20 3. Send 4. Response JSON me check: - status 200 - items array - compatibleGroups - totalCount Collection banao - sare endpoints save. Environment variables - {{baseUrl}}, {{token}}.

Q10.12. REST vs SOAP?

Ans. REST: • JSON/XML, lightweight • HTTP methods • Flexible URLs • Stateless • Modern SOAP: • XML only • WSDL contract • WS-Security • Strict protocol • Legacy enterprise BloodConnect REST use karta hai - simple, modern, mobile-friendly.

Q10.13. REST vs GraphQL?

Ans. REST: Multiple endpoints, over/under-fetching possible. GraphQL: Single endpoint, client specifies exact fields. No over-fetching. Example GraphQL query: { donor(id: 3) { name bloodGroup } } Nuksan: Complex caching, N+1 risk, learning curve. Hum REST use karte hain - simpler, standard, adequate.

Q10.14. Rate limiting kyun zaroori?

Ans. Abuse prevention: • Brute-force attacks • DDoS • Scraping • Cost control (cloud) Implement: builder.Services.AddRateLimiter(o => o .AddFixedWindowLimiter("api", opt => { opt.Window = TimeSpan.FromSeconds(10); opt.PermitLimit = 10; })); app.UseRateLimiter(); Response: 429 Too Many Requests + Retry-After header. Roadmap me hai.

Q10.15. API error response standard?

Ans. Consistent structure: { "error": { "code": "INVALID_BLOOD_GROUP", "message": "Blood group 'X+' is not valid.", "details": [{ "field": "bloodGroup", "issue": "Invalid format" }] } } HTTP status codes: • 400 - validation error • 401 - unauthenticated • 403 - unauthorized • 404 - not found • 429 - rate limited • 500 - server error ProblemDetails (RFC 7807) .NET me built-in.

Q10.16. Swagger/OpenAPI kya hai?

Ans. API documentation auto-generate. builder.Services.AddSwaggerGen(); app.UseSwagger(); app.UseSwaggerUI(); /swagger pe interactive docs. Fayde: • Client SDK generate • Testing UI • Contract-first development • Team collaboration Humare project me roadmap me.

Q10.17. Content negotiation kya hai?

Ans. Client Accept header ke hisaab se response format. Accept: application/json → JSON Accept: application/xml → XML Accept: */* → default ASP.NET Core me AddControllers(options => options.RespectBrowserAcceptHeader = true). Hum JSON default use karte hain.

Q10.18. HATEOAS kya hai?

Ans. Hypermedia As The Engine Of Application State - REST ke principles me se. Response me links hote hain next actions ke: { "donorId": 3, "name": "Ravi", "_links": { "self": "/api/donor/3", "offers": "/api/donor/3/offers", "edit": "/api/donor/3/edit" } } Client hard-code URLs nahi karta - discover karta hai. Practical me rare - most APIs simpler.

SECTION 11

Caching Deep Dive

IMemoryCache, TTL, invalidation, distributed, MessageCache

Q11.1. Caching kya hai?

Ans. Frequently-used data ko faster storage (memory) me rakhna, taki next access fast ho. Levels: • Client (browser cache) • CDN • Reverse proxy (Nginx) • Application (IMemoryCache) • Distributed (Redis) • Database (query cache) Fayde: Speed, load reduce, cost save. Tradeoff: Stale data risk.

Q11.2. IMemoryCache kya hai?

Ans. ASP.NET Core in-process cache. Key-value store in-memory. Registration: builder.Services.AddMemoryCache(); Use: public class MyService { private readonly IMemoryCache _cache; public MyService(IMemoryCache cache) { _cache = cache; } public async Task<int> GetCount() { if (_cache.TryGetValue("key", out int val)) return val; val = await FetchFromDb(); _cache.Set("key", val, TimeSpan.FromSeconds(60)); return val; } } Single-server me fast. Distributed me shared nahi.

Q11.3. Cache TTL kya hai?

Ans. Time To Live - entry kitni der valid. Absolute expiration: _cache.Set(key, val, TimeSpan.FromMinutes(5)); Sliding expiration: _cache.Set(key, val, new MemoryCacheEntryOptions { SlidingExpiration = TimeSpan.FromMinutes(5) }); Hum unread count: 60 seconds absolute. Reason: 60s me stale data acceptable. Write pe invalidate - accuracy maintain.

Q11.4. Cache invalidation kab karte hain?

Ans. Write operations pe: 1. ContactMessage.Create - new message 2. MarkRead - unread count kam 3. DeleteMessage - unread count kam _messageCache.Invalidate(); Cache.Remove(Key) internally. Next GetUnreadCountAsync fresh value fetch karke cache karega. 'There are only two hard things in computer science: cache invalidation and naming things.'

Q11.5. Write-through vs write-behind caching?

Ans. Write-through: Write pe cache bhi update. Consistent but slower writes. Write-behind: Write pe sirf DB, cache async update. Fast but stale window. Cache-aside (humar■): Read pe check, miss pe DB fetch + cache. Write pe invalidate. Humara pattern: Cache-aside + invalidation on write. Simple, predictable, correct.

Q11.6. MessageCache class detail me?

Ans. public class MessageCache { private const string CacheKey = "UnreadMessages"; private readonly IMemoryCache _cache; public MessageCache(IMemoryCache cache) { _cache = cache; } public async Task<int> GetUnreadCountAsync(ApplicationDbContext db) { if (_cache.TryGetValue(CacheKey, out int count)) return count; count = await db.ContactMessages .CountAsync(m => !m.IsRead); _cache.Set(CacheKey, count, TimeSpan.FromSeconds(60)); return count; } public void Invalidate() { _cache.Remove(CacheKey); } } Registration: Singleton (shared across requests).

Q11.7. Non-admin pe cache overhead kyun nahi?

Ans. MessageCache.GetUnreadCountAsync sirf admin layout me call hota hai. Non-admin users: • Login karte hain • Layout render hota hai (without unread badge) • Cache service call nahi hoti Zero overhead - na cache lookup, na DB query. Yeh conditional access design hai.

Q11.8. Distributed caching kab use karte hain?

Ans. Multiple servers load-balanced ho to. Problem: Server A ka IMemoryCache, Server B ka alag. User Server A pe hit → cache warm. Next request Server B pe → cache miss → DB query. Inconsistent. Solution: IDistributedCache + Redis/Memcached/SQL Server. Hum single server me - IMemoryCache kaafi hai. Scale pe Redis pe shift.

Q11.9. Redis kya hai?

Ans. In-memory key-value store. Multiple data structures (strings, hashes, lists, sets). Use: Cache, session store, message broker, rate limiting, leaderboards. .NET: StackExchange.Redis ya Microsoft.Extensions.Caching.StackExchangeRedis. Setup: builder.Services.AddStackExchangeRedisCache(o => o.Configuration = "localhost:6379"); Fast, persistent (optional), cluster-capable.

Q11.10. Cache stampede kya hai?

Ans. Popular cache entry expire → concurrent requests ek saath DB pe → DB overload. Solutions: • Lock: Ek request fetch kare, baaki wait • Stale-while-revalidate: Purani value serve, background refresh • Background refresh: Scheduled update • Jittered TTL: Alag expiry times Hum unread count me 60s TTL - load kam hai. Stampede issue nahi.

Q11.11. Cache key design best practice?

Ans. • Specific, unique • Namespace prefix: "unread:messages:user:123" • Versioned: "unread:v2:user:123" • Short but descriptive Avoid: Generic keys (collision), user input directly (injection). Humara: "UnreadMessages" (single value - count, no user-specific).

Q11.12. Memory cache eviction policies?

Ans. Memory pressure aane pe cache entries evict karta hai: • Absolute expiration - fixed time • Sliding expiration - inactivity based • Size limit - SetSizeLimit • Priority - CacheItemPriority (Low, Normal, High, NeverRemove) _config: new MemoryCacheEntryOptions { Priority = CacheItemPriority.High, AbsoluteExpirationRelativeToNow = TimeSpan.FromMinutes(5), Size = 1 }

Q11.13. Application me caching kahan kahan ho sakti hai?

Ans. Currently: Unread message count. Roadmap ideas: • Blood group compatibility lookup (static - hot path) • Popular search results • User profile data • Statistics (total donors, requests) • FAQ content Kahan nahi karni: • Real-time data (donation status) • Personal user data (security) • Frequently changing data

Q11.14. Cache vs Session vs Application state?

Ans. Cache: Read-heavy, can regenerate, may expire. Session: Per-user state, server-side, expires with session. Application state: Global to app, singleton, rarely changes (config). Hum cache use karte hain (unread count). Session avoid karte hain - stateless design. App state me config.

Q11.15. Cache hit ratio kya hai?

Ans. (cache hits / total requests) * 100 80%+ = good, 50% = average, <30% = caching ineffective. Improve: • Longer TTL (staleness tradeoff) • Better cache keys • More data cached • Warm cache on start Hum unread count: Admin views ke har request pe serve - high hit ratio.

SECTION 12

Security Deep Dive

OWASP, SQL injection, XSS, CSRF, HTTPS, headers, secrets

Q12.1. Security ke layers project me?

Ans. 1. Network: HTTPS, HSTS 2. Transport: TLS 3. Authentication: Cookie + Identity 4. Authorization: Roles + policies 5. CSRF: Anti-forgery token (global) 6. XSS: Output encoding (Razor) 7. SQL Injection: Parameterized (EF Core) 8. IDOR: Ownership checks 9. Session: HttpOnly, SameSite cookies 10. Password: PBKDF2 hashing 11. Rate limiting: Roadmap 12. Error handling: No stack traces (prod)

Q12.2. OWASP Top 10 kya hai?

Ans. Top 10 web app security risks: 1. Broken Access Control (IDOR, missing auth) 2. Cryptographic Failures (weak crypto, no HTTPS) 3. Injection (SQL, XSS, Command) 4. Insecure Design (weak architecture) 5. Security Misconfiguration (default passwords, verbose errors) 6. Vulnerable Components (outdated libraries) 7. Auth & Session Failures (weak passwords, session fixation) 8. Data Integrity Failures (unsigned updates) 9. Logging Failures (no audit trail) 10. SSRF (server-side request forgery) Hum multiple defenses implement kiye.

Q12.3. SQL injection kaise rokte hain?

Ans. Parameterized queries + EF Core. EF Core LINQ → SQL: var donors = await _context.Donors .Where(d => d.BloodGroup == bg) .ToListAsync(); → SELECT * FROM Donors WHERE BloodGroup = @p0 @p0 parameterized - user input SQL code nahi ban sakta. Avoid: string concatenation in FromSqlRaw. Agar zaroori ho to parameters use karo.

Q12.4. XSS kaise rokte hain?

Ans. Razor automatic HTML encoding: @Model.Name // User input <script> → <script> - executes nahi Agar raw HTML chahiye (trusted): @Html.Raw(Model.HtmlContent) // Careful - sirf trusted content pe Layers: • Output encoding (default) • Input validation • HttpOnly cookies (session protection) • Content Security Policy (roadmap)

Q12.5. CSRF kaise rokte hain?

Ans. 1. Anti-forgery token globally: options.Filters.Add(new AutoValidateAntiforgeryTokenAttribute()); 2. Form me token: <form asp-action="Create"> → auto token 3. SameSite cookies: options.Cookie.SameSite = SameSiteMode.Lax; 4. Custom header for AJAX (roadmap) CSRF attack: Attacker site victim ka form auto-submit karti hai - token match nahi hoga, reject.

Q12.6. IDOR kaise rokte hain?

Ans. Ownership checks server-side - har sensitive action pe. Edit Donor: var donor = await _context.Donors.FindAsync(id); if (donor.UserId != CurrentUserId) return Forbid(); Delete Donor: var isOwner = donor.UserId == CurrentUserId; var isAdmin = User.IsInRole("Admin") || User.IsInRole("SuperAdmin"); if (!isOwner && !isAdmin) return Forbid(); Admin cannot edit - sirf delete. Owner-only edit.

Q12.7. Password hashing detail?

Ans. Algorithm: PBKDF2 (Password-Based Key Derivation Function 2) • HMAC-SHA256 • 10,000+ iterations (default in ASP.NET Core Identity) • 128-bit random salt per password Verification: 1. Salt stored along with hash 2. Same algorithm + iterations applied 3. Compare hashes (constant time) DB leak pe bhi brute force impractical - 10,000 iterations per guess.

Q12.8. Timing attack kya hai?

Ans. Attacker time difference se information nikalta hai. Example: Password compare: if (input == stored) // byte-by-byte compare → time varies Defense: Constant-time comparison (subtle: CryptographicOperations.FixedTimeEquals). ASP.NET Core Identity internally use karta hai. Hamari custom code me sensitive compares me careful.

Q12.9. HTTPS kyun zaroori?

Ans. HTTP plain text - man-in-the-middle data dekhe. HTTPS = HTTP + TLS: • Encryption (confidentiality) • Integrity (tamper detection) • Authentication (server identity) Production me: • Valid certificate • HSTS header • Secure cookies (Secure flag) • Redirect HTTP → HTTPS app.UseHttpsRedirection(); app.UseHsts();

Q12.10. Content Security Policy?

Ans. HTTP header - browser ko batata hai kaunse resources load karo. Example: Content-Security-Policy: default-src 'self'; script-src 'self' https://cdn.example.com; style-src 'self' 'unsafe-inline'; img-src 'self' data:; Fayde: Inline scripts block (XSS mitigation), only trusted CDNs. Roadmap me hai - abhi nahi.

Q12.11. Security misconfiguration examples?

Ans. • Default admin password in production • Verbose error pages with stack traces • Directory listing enabled • Debug mode on in production • Unused services running • Missing security headers Defense: • Change defaults (humara seed credentials - change on first login) • Styled error pages • ASPNETCORE_ENVIRONMENT=Production • Remove unused middleware

Q12.12. Vulnerable components?

Ans. Outdated NuGet packages me known CVEs. Tools: • dotnet list package --vulnerable • GitHub Dependabot • Snyk • OWASP Dependency-Check Best practice: • Regular updates • Monitor advisories • Lock versions in production

Q12.13. Logging aur monitoring kyun zaroori?

Ans. Security incidents detect + respond. Log karo: • Login attempts (success + failure) • Role changes (audit) • Ban/unban events • Delete operations • Suspicious activity (rate limiting triggers) Kya log NAHI karo: • Passwords • Session tokens • Credit card numbers • PII (unless required) ASP.NET Core ILogger + Serilog.

Q12.14. Data protection API kya hai?

Ans. ASP.NET Core ka built-in encryption for: • Cookie authentication tokens • Anti-forgery tokens • Session state • TempData Uses machine key (auto-generated). Production: Persist keys (Azure Key Vault, file share) taki multiple servers/restarts me consistent rahe. builder.Services.AddDataProtection().PersistKeysToFileSystem(...);

Q12.15. Secrets management best practice?

Ans. NEVER commit secrets: • Connection strings • API keys • SMTP passwords • JWT signing keys Development: dotnet user-secrets init dotnet user-secrets set "Key" "Value" Production: • Environment variables (basic) • Azure Key Vault • AWS Secrets Manager • HashiCorp Vault Hum: Production ke liye ConnectionStrings__DefaultConnection env var recommend karte hain.

Q12.16. Security headers kya add karo?

Ans. • Strict-Transport-Security (HSTS) - HTTPS force • X-Content-Type-Options: nosniff - MIME sniffing block • X-Frame-Options: DENY - clickjacking • Referrer-Policy: no-referrer - privacy • Content-Security-Policy - XSS mitigation • Permissions-Policy - feature restrictions app.Use(async (ctx, next) => { ctx.Response.Headers.Add("X-Content-Type-Options", "nosniff"); await next(); });

Q12.17. Penetration testing kya hai?

Ans. Authorized simulation of attacks - vulnerabilities dhoondhna. Types: • Black box (no info) • White box (full info) • Gray box (partial) Tools: • OWASP ZAP • Burp Suite • Nmap • SQLMap Humare project me manual testing + future automated tools.

SECTION 13

Medium Level Questions

DI lifetimes, async/await, filters, options pattern, logging

Q13.1. Dependency Injection kya hai?

Ans. Design pattern - class apni dependencies khud create nahi karti, bahar se milti hain. Types: • Constructor injection (recommended) • Property injection • Method injection public class DonorController : Controller { private readonly ApplicationDbContext _context; private readonly IMessageCache _cache; public DonorController(ApplicationDbContext context, IMessageCache cache) { _context = context; _cache = cache; } } Fayde: Testability, loose coupling, lifecycle management.

Q13.2. Service lifetimes kya hain?

Ans. • Transient - Har request pe naya instance. Lightweight stateless services. • Scoped - Ek HTTP request ke andar same instance. DbContext, per-request state. • Singleton - Poori app me ek instance. Cache, config. builder.Services.AddTransient<IMyService, MyService>(); builder.Services.AddScoped<IMyService, MyService>(); builder.Services.AddSingleton<IMyService, MyService>(); Humara: DbContext=Scoped, MessageCache=Singleton, Controllers=Transient.

Q13.3. Lifetime mismatch kya hai?

Ans. Singleton Scoped pe depend nahi kar sakta: // BAD services.AddSingleton<MySingleton>(); services.AddScoped<MyScoped>(); // MySingleton constructor MyScoped maangta hai → error Error: 'Cannot consume scoped service from singleton' Solutions: 1. IServiceScopeFactory inject karo 2. Design change - Singleton ko Scoped dependency na chahiye 3. Scoped service ko Singleton banao (careful - thread safety)

Q13.4. async/await kya karta hai?

Ans. Async - method asynchronous banata hai, Task return. await - Task complete hone tak wait, thread block nahi. public async Task<IActionResult> Index() { var donors = await _context.Donors.ToListAsync(); return View(donors); } Behind the scenes: State machine generated. I/O pe thread pool thread free. Rule: Async all the way. Sync over async never (.Result, .Wait() avoid).

Q13.5. Task vs Thread vs ValueTask?

Ans. Thread - OS-level execution unit. Heavy, limited (~1000 practical). Task - Higher-level abstraction over thread pool. Lightweight. Async operations. ValueTask - Struct version of Task. For very high-performance scenarios - reduces allocations. Web apps: Task (default). ValueTask (hot paths, allocations critical).

Q13.6. Task vs Task<T>?

Ans. Task - Asynchronous operation, no return value. Task<T> - Returns value of type T. public async Task DoSomething() { ... } public async Task<int> GetCount() { return 42; } await DoSomething(); int x = await GetCount(); Task.CompletedTask, Task.FromResult(value) - cached/synchronous completions.

Q13.7. ConfigureAwait(false) kab use karo?

Ans. Library code me - SynchronizationContext capture avoid. var data = await httpClient.GetStringAsync(url) .ConfigureAwait(false); Application code (controllers) me zaroori nahi - ASP.NET Core me no SyncContext by default. Fayde: Performance, deadlock prevention (older frameworks me).

Q13.8. Filters kya hain MVC me?

Ans. Attributes jo request processing me inject hote hain. Types: • Authorization - [Authorize] • Resource - caching, short-circuit • Action - before/after action method • Exception - global error handling • Result - result execution pe Global: options.Filters.Add(new MyFilter()); Hum: AutoValidateAntiforgeryToken global filter.

Q13.9. Action filter ka example?

Ans. public class LogActionFilter : IActionFilter { private readonly ILogger _logger; public LogActionFilter(ILogger<LogActionFilter> log) { _logger = log; } public void OnActionExecuting(ActionExecutingContext ctx) { _logger.LogInformation("Starting {Action}", ctx.ActionDescriptor.DisplayName); } public void OnActionExecuted(ActionExecutedContext ctx) { _logger.LogInformation("Finished {Action}", ctx.ActionDescriptor.DisplayName); } } // Register: options.Filters.Add<LogActionFilter>();

Q13.10. Options pattern kya hai?

Ans. Strongly-typed config access. public class EmailSettings { public string SmtpHost { get; set; } public int Port { get; set; } } // Program.cs builder.Services.Configure<EmailSettings>(builder.Configuration.GetSection("Email")); // Usage public class EmailService { private readonly EmailSettings _settings; public EmailService(IOptions<EmailSettings> opts) { _settings = opts.Value; } }

Q13.11. Logging levels kya hain?

Ans. • Trace - Most detailed (dev only) • Debug - Debugging info • Information - Normal flow • Warning - Unexpected but OK • Error - Failure, recoverable • Critical - Severe, app-level _logger.LogInformation("User {UserId} searched {Group}", userId, group); Structured logging - parameters separate, searchable. Config: appsettings.json me level set.

Q13.12. Global exception handling?

Ans. Program.cs: if (!app.Environment.IsDevelopment()) { app.UseExceptionHandler("/Home/Error"); app.UseHsts(); } app.UseStatusCodePagesWithReExecute("/Home/Error", "?code={0}"); HomeController.Error(int? code) - branded page. Development: Developer Exception Page (detailed). Production: User-friendly, no stack trace.

Q13.13. Razor view me C# kitna?

Ans. Minimum - presentation only. OK: @foreach, @if, @Model.Property Not OK: • DB queries • Business logic • Complex computations Exception: _LoginPartial me IMessageCache service inject - unread badge ke liye. Service call acceptable, direct DbContext nahi. Better: ViewComponent (C# class + view) for complex embedded logic.

Q13.14. Areas kab use karte hain?

Ans. Large apps me sections organize: Areas/Admin/ Controllers/ Views/ Models/ Areas/Customer/ ... Route: [Area("Admin")] attribute. Humara simple - single area concept. Bade project me helpful.

Q13.15. Partial vs ViewComponent?

Ans. Partial View: Reusable Razor chunk, model pass. Simple. <partial name="_DonorCard" model="donor" /> ViewComponent: C# class + view, apna logic (DB, computation). public class UnreadBadgeViewComponent : ViewComponent { public async Task<IViewComponentResult> InvokeAsync() { var count = await _cache.GetUnreadCountAsync(...); return View(count); } } Use: @await Component.InvokeAsync("UnreadBadge") Complex embedded UI ke liye ViewComponent better.

Q13.16. Bootstrap kyun use kiya?

Ans. Rapid responsive development: • Grid system (12 columns) • Pre-built components (buttons, forms, modals, alerts) • Utility classes (m-, p-, d-) • Mobile-first Custom CSS se override - humara warm palette (cream + burgundy + gold), Inter font, animations. Combination: Bootstrap base + custom design system.

Q13.17. Responsive design kaise kiya?

Ans. Mobile-first approach: Bootstrap grid: <!-- Full width mobile, half tablet, third desktop --> Media queries in custom CSS: @media (max-width: 768px) { .hero-title { font-size: 24px; } } Test: Chrome DevTools device emulation.

Q13.18. Sound effects kaise implement kiye?

Ans. Web Audio API - procedural tones (no files). const ctx = new AudioContext(); function playTone(freq, duration, type='sine') { const osc = ctx.createOscillator(); const gain = ctx.createGain(); osc.frequency.value = freq; osc.type = type; osc.connect(gain); gain.connect(ctx.destination); gain.gain.exponentialRampToValueAtTime(0.001, ctx.currentTime + duration); osc.start(); osc.stop(ctx.currentTime + duration); } Success: high chime (800Hz) Error: low buzz (200Hz sawtooth) Click: short tick (1200Hz) Notify: ping (1000Hz) Toggle: localStorage - user preference.

Q13.19. Custom confirmation modal?

Ans. Auto-intercept onsubmit="return confirm()": // animations.js const originalConfirm = window.confirm; // Attach global listener document.addEventListener('submit', async (e) => { const form = e.target; const confirmMsg = form.getAttribute('data-confirm'); if (confirmMsg && !form.dataset.confirmed) { e.preventDefault(); const ok = await showModal(confirmMsg); if (ok) { form.dataset.confirmed = 'true'; form.submit(); } } }); No per-view changes - existing forms work as-is.

Q13.20. Toast notifications kaise?

Ans. Server se TempData me message: TempData["Toast"] = "Donor created!"; TempData["ToastType"] = "success"; Layout me render: @if (TempData["Toast"] != null) { <div class="toast-container" data-toast="@TempData["Toast"]" data-type="@TempData["ToastType"]"> } JS: showToast(message, type); // Slide-in, auto-dismiss after 3s // Type-aware colors: success (green), error (red), warning (orange), info (blue)

Q13.21. Global exception pages kaise test kare?

Ans. Development: • /Home/Error pe redirect nahi hota • Developer Exception Page full stack trace Production test: ASPNETCORE_ENVIRONMENT=Production dotnet run Ya controller me manually: return StatusCode(500); return NotFound(); Styled page verify karo - 404, 500, 403 sab.

Q13.22. Razor view me null se bachne ke tips?

Ans. • Null checks: @if (Model.Donor != null) { ... } • Null coalescing: @(Model.Name ?? "Unknown") • Null-conditional: @Model?.Donor?.Name • Default values in ViewModel • #nullable enable project-level • Required attribute on properties Best: ViewModel me defaults set karo - view me assume safe.

Q13.23. Kya aapne automated tests likhe?

Ans. Nahi - currently manual testing. Roadmap: xUnit/NUnit + Moq: [Fact] public void BloodCompatibility_OnNegative_OnlyOnNegative() { var result = BloodCompatibility .GetCompatibleDonorGroups("O-"); Assert.Single(result); Assert.Contains("O-", result); } Integration tests: WebApplicationFactory + TestContainers. Yeh improve karne layak area hai.

Q13.24. Controller thin kaise rakha?

Ans. Pattern: 1. Receive request 2. Model binding 3. Service/DB call 4. Return result Business logic OUT: • BloodCompatibility - separate static class • MessageCache - separate service • Validation - attributes Max ~50-80 lines per action. Complex actions me private helpers.

Q13.25. Views me DB calls kyun nahi?

Ans. MVC separation principle: • Views = presentation • Controller = orchestration • Model/Service = data + logic Exceptions rare - humara _LoginPartial me IMessageCache service inject (unread count). Yeh caching service hai, DB direct nahi. Agar DB query chahiye - controller me fetch karo, ViewModel me pass karo.

Q13.26. Concurrency ke types?

Ans. Optimistic: Assumption - conflicts rare. RowVersion check. Pessimistic: Lock during transaction. Slow, deadlock risk. Hum: Optimistic (RowVersion on Donor). Reason: Web apps me concurrent edits rare. Users edit unke own records. Locking overhead unnecessary. Conflict handling: DbUpdateConcurrencyException catch.

Q13.27. Rate limiting kaise implement kar sakte hain?

Ans. .NET 7+ built-in: builder.Services.AddRateLimiter(options => { options.AddFixedWindowLimiter("fixed", opt => { opt.Window = TimeSpan.FromSeconds(10); opt.PermitLimit = 20; opt.QueueLimit = 5; }); }); app.UseRateLimiter(); Policy on action: [EnableRateLimiting("fixed")] Response: 429 Too Many Requests. Roadmap me hai.

Q13.28. Health checks kya hain?

Ans. Endpoints jo app health report karte hain. builder.Services.AddHealthChecks() .AddDbContextCheck<ApplicationDbContext>() .AddCheck("custom", () => HealthCheckResult.Healthy()); app.MapHealthChecks("/health"); Response: Healthy / Degraded / Unhealthy. Use: Kubernetes readiness/liveness, monitoring, load balancer. Roadmap me.

Q13.29. API versioning kab zaroori?

Ans. Jab breaking changes karni ho: • Field remove • Response structure change • Behavior change Strategy: 1. Add version 2. Old version deprecated mark 3. Support window (6-12 months) 4. Remove old Hum abhi v1 pe - koi breaking changes nahi.

Q13.30. Structured logging kya hai?

Ans. Logs me parameters separate - searchable, parse-able. _logger.LogInformation("User {UserId} searched {BloodGroup} in {Location}", userId, bloodGroup, location); Benefits: • Query: "Find logs where UserId=5" • No string concatenation cost • Structured in ELK/Splunk/App Insights Unstructured (avoid): _logger.LogInformation($"User {userId} searched...");

SECTION 14

Advanced / Tough Questions

SOLID, CQRS, Docker, Kubernetes, Redis, Saga, event sourcing

Q14.1. SOLID principles?

Ans. S - Single Responsibility: Ek class, ek kaam. Hum: Controllers thin, BloodCompatibility separate, MessageCache separate. O - Open/Closed: Extension open, modification closed. Hum: New blood group add karne pe - DomainValues me add, BloodCompatibility table update. L - Liskov Substitution: Derived base ki jagah use ho. Hum: AppUser : IdentityUser - sab jagah IdentityUser ki tarah use. I - Interface Segregation: Chhoti specific interfaces. Hum: IMessageCache chhoti interface - sirf 2 methods. D - Dependency Inversion: Abstractions pe depend. Hum: Controllers IMessageCache pe depend, MessageCache class pe nahi.

Q14.2. Repository pattern kya hai?

Ans. Data access ko abstract karta hai: public interface IDonorRepository { Task<Donor> GetByIdAsync(int id); Task<List<Donor>> GetCompatibleAsync(string bloodGroup); Task AddAsync(Donor donor); } Fayde: Testability (mock repo), swappable DB. Nuksan: EF Core already Unit of Work + Repository. Extra layer = leaky abstraction for complex queries. Hum: EF Core directly use karte hain - simpler, modern.

Q14.3. Unit of Work pattern?

Ans. Multiple operations ek transaction me. public interface IUnitOfWork { IDonorRepository Donors { get; } IRequestRepository Requests { get; } Task<int> SaveChangesAsync(); } Hum: DbContext khud Unit of Work hai. Auto-reject - multiple updates single SaveChangesAsync me atomic.

Q14.4. CQRS kya hai?

Ans. Command Query Responsibility Segregation: • Commands - Write operations • Queries - Read operations Alag models, alag databases (optional - read replica). Fayde: Scale reads/writes independently, optimized models. Nuksan: Complexity, eventual consistency. Humara: Simple MVC - CQRS overkill educational project me.

Q14.5. Event Sourcing?

Ans. State ke bajaye events store karo: Instead of: donor.Status = Accepted Store: { event: 'OfferAccepted', offerId: 5, time: ... } Current state = events replay. Fayde: Full audit trail, time travel, event-driven. Nuksan: Complex queries, storage growth. Humara: Traditional state - simple, adequate.

Q14.6. Saga Pattern?

Ans. Distributed transactions - local transactions chain. Agar step fail: 1. Compensating transaction for previous steps 2. Choreography (events) ya Orchestration (central) Humara: Monolith me pledge workflow single transaction me - saga ki zaroorat nahi.

Q14.7. Microservices vs Monolith?

Ans. Monolith: Ek deployable unit, sab modules. Microservices: Alag services, independent deploy. Monolith fayde: • Simple deploy • Easy debug • No network latency • Transactions easy Microservices fayde: • Independent scale • Tech diversity • Team autonomy Humara: Monolith - student project ke liye correct.

Q14.8. Docker kya hai?

Ans. Containerization - app + dependencies bundle. Dockerfile: FROM mcr.microsoft.com/dotnet/sdk:10 AS build WORKDIR /src COPY . . RUN dotnet publish -c Release -o /app FROM mcr.microsoft.com/dotnet/aspnet:10 WORKDIR /app COPY --from=build /app . ENTRYPOINT ["dotnet", "BloodConnect.dll"] Build: docker build -t bloodconnect . Run: docker run -p 5006:80 bloodconnect Fayde: "Works on my machine" solve, consistent environments.

Q14.9. Kubernetes kya hai?

Ans. Container orchestration: • Deployment • Scaling (auto, HPA) • Load balancing • Self-healing (restart failed pods) • Rolling updates • Service discovery YAML manifests: kind: Deployment spec: replicas: 3 template: spec: containers: - name: bloodconnect image: bloodconnect:latest Complex - chhote projects me overkill.

Q14.10. Load balancing algorithms?

Ans. • Round Robin - Turn-by-turn • Least Connections - Jo free hai • IP Hash - Same IP → Same server (sticky) • Weighted - Server capacity ke hisaab se • Random Tools: Nginx, HAProxy, Azure Load Balancer, AWS ELB. Humara: Cookie-based auth stateless - koi bhi algorithm fine.

Q14.11. Horizontal vs Vertical scaling?

Ans. Vertical (Up): Bigger machine. Simple, limited. Horizontal (Out): More machines. Unlimited, complex. ASP.NET Core: Horizontal scale ke liye stateless hona chahiye. Humara: Cookie auth stateless. Session server-side me nahi - distributed cache pe shift kar sakte hain.

Q14.12. Redis kya hai?

Ans. In-memory data store: • Key-value • Data structures (list, set, hash, sorted set) • Pub/sub • Persistence options • Cluster Use cases: • Distributed cache • Session store • Rate limiting • Leaderboards • Queue Hum: Single server IMemoryCache. Scale pe Redis - IDistributedCache interface swap.

Q14.13. Message queue kab?

Ans. Async processing: • Producer → Queue → Consumer Use: • Email sending (pledge accepted) • Image processing • Report generation • Third-party API calls Tools: RabbitMQ, Kafka, Azure Service Bus. Fayde: Decoupling, retry, scale independent.

Q14.14. SignalR kya hai?

Ans. Real-time communication: • WebSockets (primary) • Server-Sent Events • Long polling (fallback) Use: • Live notifications • Chat • Real-time dashboards • Collaborative apps BloodConnect me: • Pledge accept → real-time notify • Live donor count • Chat between owner and donor Roadmap me hai.

Q14.15. gRPC kya hai?

Ans. Google RPC framework: • HTTP/2 • Protocol Buffers (binary) • Strongly typed (.proto files) • Streaming support Fayde: Fast, efficient, polyglot. Use: Microservices internal communication. REST: External APIs, browser. gRPC: Internal, high-performance.

Q14.16. N+1 problem solve karna?

Ans. Problem: var requests = await _context.BloodRequests.ToListAsync(); // 1 query foreach (var r in requests) { var count = r.Offers.Count; // N queries } Solution: Include() var requests = await _context.BloodRequests .Include(r => r.Offers) .ToListAsync(); // 1 query Or projection: .Select(r => new { r.Id, OfferCount = r.Offers.Count })

Q14.17. Connection pooling?

Ans. DB connections reuse - naya connection kholna slow. ADO.NET default: 100 connections per connection string. Config: "Server=...;Max Pool Size=200;Min Pool Size=5" Best practice: • DbContext short-lived (Scoped) • Dispose properly (DI handles) • Long-running operations avoid

Q14.18. Deadlock kaise prevent karte hain?

Ans. DB deadlock: Do transactions ek doosre ka lock wait. Prevention: • Consistent lock order • Short transactions • Proper indexes (less locking) • READ COMMITTED SNAPSHOT (SQL Server) • Retry logic (transient) Async code me deadlock: • .Result / .Wait() avoid • ConfigureAwait(false) library me

Q14.19. Distributed transaction kaise handle?

Ans. Multiple DBs ke across - challenging. Patterns: • Two-Phase Commit (2PC) - slow, blocking • Saga - compensations • Eventual consistency - accept temporary inconsistency • Outbox pattern - events reliable Humara: Single DB - no need.

Q14.20. Blue-Green deployment?

Ans. Do identical environments: • Blue = live • Green = staging Deploy: 1. New version → Green 2. Test 3. Traffic switch (Blue → Green) 4. Rollback instant (switch back) Zero downtime, safe.

Q14.21. Canary deployment?

Ans. Gradual rollout: 1. 5% users → new version 2. Monitor metrics, errors 3. If stable → 10% → 25% → 50% → 100% 4. If issues → rollback Fayde: Small blast radius, real-world testing.

Q14.22. CI/CD kya hai?

Ans. CI (Continuous Integration): • Code push → auto build + test • Tools: GitHub Actions, Azure DevOps, Jenkins CD (Continuous Deployment): • Passing build → auto deploy • Staging → Production Pipeline stages: 1. Build 2. Unit tests 3. Integration tests 4. Security scan 5. Deploy staging 6. Smoke tests 7. Deploy production

Q14.23. Monitoring aur observability?

Ans. Monitoring: Known metrics track Observability: Unknown issues investigate 3 pillars: • Metrics - Numeric (CPU, requests/sec, error rate) • Logs - Events (structured) • Traces - Request flow across services Tools: • Prometheus + Grafana • ELK Stack (Elasticsearch, Logstash, Kibana) • Application Insights • Datadog ASP.NET Core: ILogger + OpenTelemetry

SECTION 15

Project-Specific Deep Dive

Har controller, view, feature ka complete flow

Q15.1. DonorController.Index ka poora flow?

Ans. 1. [Authorize] - logged in check 2. DbContext query: var donors = await _context.Donors .AsNoTracking() .OrderByDescending(d => d.CreatedAt) .Take(50) .ToListAsync(); 3. ViewModel map (agar zaroori) 4. return View(donors); 5. Razor renders list with cards 6. Each card: Name, BloodGroup, Location, actions (if owner)

Q15.2. DonorController.Search ka flow?

Ans. 1. [HttpGet] [AllowAnonymous] - public search 2. Parameters: bloodGroup, location, exactMatch, page 3. BloodCompatibility.GetCompatibleDonorGroups(bloodGroup) 4. Query: var q = _context.Donors.AsNoTracking() .Where(d => groups.Contains(d.BloodGroup)); if (!empty location) q = q.Where(like); 5. OrderByDescending(exact), ThenBy(group), ThenBy(name) 6. Skip/Take for pagination 7. ViewBag for pagination metadata 8. return View(donors)

Q15.3. DonorController.Create ka flow?

Ans. GET: • Check karo user ka already donor profile exists? • Agar haan - redirect Edit pe • Nahi - blank form POST: 1. ModelState.IsValid check 2. Additional validation: - Age 18-65 (attribute already) - BloodGroup valid (attribute) 3. Duplicate check (UserId already donor?) 4. Create Donor entity: donor.UserId = CurrentUserId; donor.CreatedAt = DateTime.UtcNow; 5. SaveChangesAsync 6. TempData toast "Donor created" 7. RedirectToAction("Index")

Q15.4. DonorController.Edit flow?

Ans. GET: 1. Donor fetch by id 2. Ownership check: donor.UserId == CurrentUserId? Nahi to 404 ya 403 3. View with model POST: 1. ModelState check 2. Ownership re-check (server-side - trust no one) 3. RowVersion present? 4. Update fields 5. _context.Update(donor) 6. try { await SaveChangesAsync(); } catch (DbUpdateConcurrencyException) { TempData error "Record updated by someone else" return View(donor) } 7. RedirectToAction Index

Q15.5. DonorController.Delete flow?

Ans. GET: 1. Fetch donor 2. Authorization: - Owner: can delete own - Admin/SuperAdmin: can delete anyone 3. Show confirmation page POST: 1. Re-fetch donor 2. Same authorization check 3. Check active offers? Warning 4. Cascade delete (offers) 5. SaveChangesAsync 6. Toast + redirect Admin Edit nahi kar sakta - sirf delete

Q15.6. BloodRequestController.Create flow?

Ans. GET: [AllowAnonymous] • Form for guest + user POST: 1. ModelState check 2. New BloodRequest: request.Status = Active; request.CreatedAt = UtcNow; request.UserId = User.Identity.IsAuthenticated ? CurrentUserId : null; 3. SaveChangesAsync 4. TempData toast 5. Redirect to Details

Q15.7. BloodRequestController.Offer flow?

Ans. POST /Offer(requestId): 1. [Authorize] - logged in 2. Donor profile exists? var donor = await _context.Donors.FirstOrDefaultAsync(d => d.UserId == CurrentUserId); if (donor == null) return BadRequest("Create profile first"); 3. Request fetch 4. Checks: • Request not owner's own • Request status == Active • Donor blood group compatible 5. Duplicate check (unique index) 6. Create DonationOffer: Status = Pledged, CreatedAt = UtcNow 7. SaveChangesAsync 8. Redirect to request details

Q15.8. BloodRequestController.Accept flow?

Ans. POST /Accept(offerId): 1. [Authorize] 2. Offer fetch with Include(Request) 3. Ownership: offer.Request.UserId == CurrentUserId? 4. State check: offer.Status == Pledged? 5. offer.Status = Accepted 6. offer.AcceptedAt = UtcNow 7. offer.Request.Status = InProgress 8. Auto-reject: var others = await _context.DonationOffers .Where(o => o.RequestId == offer.RequestId && o.Id != offerId && o.Status == Pledged) .ToListAsync(); foreach (var o in others) o.Status = Rejected; 9. SaveChangesAsync (single transaction) 10. Toast + redirect

Q15.9. BloodRequestController.Complete flow?

Ans. POST /Complete(offerId): 1. [Authorize] 2. Offer fetch with Request + Donor 3. Authorization: isOwner = request.UserId == CurrentUserId isDonor = offer.Donor.UserId == CurrentUserId if (!isOwner && !isDonor) return Forbid 4. State: offer.Status == Accepted? 5. offer.Status = Completed 6. offer.CompletedAt = UtcNow 7. offer.Request.Status = Fulfilled 8. SaveChangesAsync 9. Toast "Donation completed!" + redirect

Q15.10. AdminController.Index flow?

Ans. 1. [Authorize(Roles = "Admin,SuperAdmin")] 2. Stats fetch: var stats = new AdminDashboardViewModel { TotalDonors = await _context.Donors.CountAsync(), TotalRequests = await _context.BloodRequests.CountAsync(), ActiveRequests = await _context.BloodRequests .CountAsync(r => r.Status == Active), TotalUsers = await _userManager.Users.CountAsync(), UnreadMessages = await _messageCache.GetUnreadCountAsync(_context), PendingOffers = await _context.DonationOffers .CountAsync(o => o.Status == Pledged) }; 3. return View(stats);

Q15.11. AdminController.Ban flow?

Ans. POST /Ban(userId): 1. [Authorize(Roles = "Admin,SuperAdmin")] 2. Current user roles fetch 3. Target user fetch + roles 4. Rules: • Self-protection: userId != CurrentUserId • SuperAdmin immutable - anyone can't ban • Admin can't ban Admin/SuperAdmin 5. targetUser.IsBanned = true 6. Update + SaveChanges 7. Toast + redirect Defense: Role hierarchy server-side enforced.

Q15.12. AdminController.Promote flow?

Ans. POST /Promote(userId): 1. [Authorize(Roles = "Admin,SuperAdmin")] 2. Target user check - role User hona chahiye 3. Role hierarchy: • Admin can promote User → Admin • SuperAdmin can promote to any 4. await _userManager.AddToRoleAsync(user, "Admin") 5. SaveChanges 6. Toast Demote: Admin → User (sirf SuperAdmin)

Q15.13. MessageCache ka step-by-step?

Ans. 1. Admin layout render (har page pe) 2. _LoginPartial me inject: IMessageCache 3. badge call: @await _cache.GetUnreadCountAsync(_context) 4. Method: a. TryGetValue(CacheKey) → hit? return b. Miss: COUNT(*) WHERE IsRead = 0 c. Set cache 60s TTL d. Return 5. Badge render: @if (count > 0) { @count } Non-admin: Badge HTML render nahi hota - zero cache lookup.

Q15.14. Cache invalidation kab trigger hoti hai?

Ans. 1. ContactController.Create: await _context.SaveChangesAsync(); _messageCache.Invalidate(); 2. AdminController.MarkRead(id): msg.IsRead = true; await _context.SaveChangesAsync(); _messageCache.Invalidate(); 3. AdminController.DeleteMessage(id): _context.ContactMessages.Remove(msg); await _context.SaveChangesAsync(); _messageCache.Invalidate(); Next badge call: cache miss → fresh count → cache with 60s TTL.

Q15.15. Program.cs ka detailed flow?

Ans. var builder = WebApplication.CreateBuilder(args); // 1. Services builder.Services.AddDbContext<ApplicationDbContext>(opts => opts.UseSqlServer(conn, sql => sql.EnableRetryOnFailure(5, TimeSpan.FromSeconds(10), null))); builder.Services.AddIdentity<AppUser, IdentityRole>(opts => { opts.Password.RequiredLength = 6; opts.Lockout.MaxFailedAccessAttempts = 5; opts.Lockout.DefaultLockoutTimeSpan = TimeSpan.FromMinutes(5); }) .AddEntityFrameworkStores<ApplicationDbContext>() .AddDefaultTokenProviders(); builder.Services.ConfigureApplicationCookie(opts => { opts.Cookie.HttpOnly = true; opts.Cookie.SameSite = SameSiteMode.Lax; opts.ExpireTimeSpan = TimeSpan.FromDays(7); opts.SlidingExpiration = true; opts.Events.OnValidatePrincipal = async ctx => { var user = await ctx.HttpContext.RequestServices .GetRequiredService<UserManager<AppUser>>() .GetUserAsync(ctx.Principal); if (user?.IsBanned == true) ctx.RejectPrincipal(); }; }); builder.Services.AddMemoryCache(); builder.Services.AddSingleton<MessageCache>(); builder.Services.AddControllersWithViews(opts => { opts.Filters.Add(new AutoValidateAntiforgeryTokenAttribute()); }); var app = builder.Build(); // 2. Pipeline if (app.Environment.IsDevelopment()) app.UseDeveloperExceptionPage(); else { app.UseExceptionHandler("/Home/Error"); app.UseHsts(); } app.UseStatusCodePagesWithReExecute("/Home/Error", "?code={0}"); app.UseHttpsRedirection(); app.UseStaticFiles(); app.UseRouting(); app.UseAuthentication(); app.UseAuthorization(); app.MapControllerRoute("default", "{controller=Home}/{action=Index}/{id?}"); // 3. Seed using (var scope = app.Services.CreateScope()) await DbSeeder.SeedAsync(scope.ServiceProvider); app.Run();

Q15.16. Authentication flow step-by-step?

Ans. REGISTER: 1. GET /Account/Register - form 2. POST: ModelState check 3. new AppUser { Email, UserName } 4. await _userManager.CreateAsync(user, password) 5. await _userManager.AddToRoleAsync(user, "User") 6. await _signInManager.SignInAsync(user, false) 7. RedirectToAction Home LOGIN: 1. GET /Account/Login 2. POST: SignInManager.PasswordSignInAsync 3. Result checks: Succeeded, IsLockedOut, Banned 4. On success: cookie set automatically 5. Redirect return URL or Home LOGOUT: 1. POST /Account/Logout 2. SignOutAsync() - cookie clear 3. Redirect Home

Q15.17. Dashboard ka flow?

Ans. 1. [Authorize] - logged in 2. Current user fetch 3. Donor profile check 4. Fetch user-specific data: • Own donor profile • Own blood requests • Offers made (as donor) • Donations fulfilled 5. ViewModel: public class DashboardViewModel { public Donor? Donor { get; set; } public List<BloodRequest> MyRequests { get; set; } public List<DonationOffer> MyOffers { get; set; } public List<DonationOffer> CompletedDonations { get; set; } public bool HasDonorProfile { get; set; } } 6. View renders cards

Q15.18. Saare controllers list + purpose?

Ans. • HomeController - Static pages (Home, About, FAQ, Contact, Privacy, Error) • AccountController - Register, Login, Logout, AccessDenied • DonorController - CRUD + Search • BloodRequestController - CRUD + Offer, Accept, Reject, Complete, Cancel • DonorApiController - REST API search • AdminController - Role mgmt, Ban, Promote, Messages • DashboardController - Per-user overview Total: 7 controllers, ~40+ action methods.

Q15.19. Saare models list?

Ans. Domain entities: • AppUser : IdentityUser • Donor • BloodRequest • DonationOffer • ContactMessage Helpers: • BloodCompatibility - static lookup • DomainValues - constants • ValidationAttributes - MustBeTrue • ApplicationDbContext - EF Core • DbSeeder - seed data ViewModels: • RegisterViewModel, LoginViewModel, etc. • AdminDashboardViewModel, ContactViewModel • DonorSearchViewModel, DashboardViewModel

Q15.20. Saare views ka purpose?

Ans. Shared/ • _Layout - master template • _LoginPartial - auth badge • _ValidationScriptsPartial • Error.cshtml Home/: Index, About, Contact, FAQ, Privacy, Error Donor/: Index, _Form (partial), Create, Edit, Delete, Details, Search BloodRequest/: Index, Create, Details, MyRequests, Offers Account/: Register, Login, AccessDenied Admin/: Index, Donors, Requests, UserList, Messages Dashboard/: Index

Q15.21. Layout me kya kya include hai?

Ans. • HTML doctype, meta tags (viewport, charset) • Google Fonts - Inter • Bootstrap 5 CSS • Custom CSS (site.css, animations.css) • Header nav (logo, links, login partial) • @RenderBody() - actual content • Footer • Toast container • Confirm modal container • Bootstrap JS + custom JS (site.js, animations.js) • @RenderSection("Scripts")

Q15.22. Custom CSS me kya kya hai?

Ans. site.css: • CSS variables (--burgundy, --gold, --cream) • Typography (Inter font) • Buttons (primary, outline, ghost) • Cards (premium shadow, hover lift) • Forms (custom inputs) • Navbar (glass effect, backdrop blur) • Responsive tweaks animations.css: • Toast slide-in • Modal fade + scale • Loader spin • Card tilt (3D) • Cursor glow • Button hover • Section fade-in

Q15.23. JavaScript me kya kya hai?

Ans. site.js: • Nav toggle (mobile) • Form validation helpers • Scroll to top button animations.js: • showToast(message, type) • Custom confirm modal • Sound effects (Web Audio API) • Card 3D tilt (mousemove) • Magnetic buttons • Cursor glow trail • Scroll animations (IntersectionObserver) All vanilla JS - no jQuery dependency.

Q15.24. Toast types + colors?

Ans. Success - Green (#1B6B3A) - "Donor created!" Error - Red (#C41E3A) - "Invalid input" Warning - Orange (#E67E22) - "Record updated by someone else" Info - Blue/Teal (#0A7E8C) - "Please check email" Server-side set: TempData["Toast"] = "..."; TempData["ToastType"] = "success"; Layout renders: @if (TempData["Toast"] != null) { <div data-toast="@TempData["Toast"]" data-type="@TempData["ToastType"]"> } JS reads and shows toast with slide-in animation.

Q15.25. Custom confirm modal kaise replace karta hai browser confirm?

Ans. Browser confirm() ko override karte hain: // animations.js window.__originalConfirm = window.confirm; window.confirm = async function(message) { return await showModal(message); }; Ya better - form interception: document.addEventListener('submit', async (e) => { const form = e.target; if (form.dataset.confirm && !form.dataset.confirmed) { e.preventDefault(); const ok = await showModal(form.dataset.confirm); if (ok) { form.dataset.confirmed = 'true'; form.submit(); } } }, true); // capture phase Existing forms - no per-view changes.

Q15.26. Error pages ka design?

Ans. Error.cshtml: • Blood drop icon • Large status code (404, 500, etc.) • Friendly message per code • "Back to Home" button • Search bar (optional) • Emergency contact (blood-related) HomeController: public IActionResult Error(int? code) { ViewBag.Code = code ?? 500; ViewBag.Message = code switch { 404 => "Page not found", 403 => "Access denied", 500 => "Something went wrong", _ => "An error occurred" }; return View(); }

Q15.27. Search results me card kaise dikhate hain?

Ans. Each donor card: • Blood group badge (large, colored) • Name (bold) • Location with pin icon • Age • Availability status (Available/Unavailable) • "View Details" button If exact match: • Gold border • "Exact Match" badge Contact number: HIDDEN (unless via workflow). Layout: Bootstrap grid (col-12 col-md-6 col-lg-4).

Q15.28. Workflow actions ka UI?

Ans. Request detail page: • Owner view: - "View Offers" button (with count badge) - Own request - can't offer on own - Cancel button (if Active) • Donor view (logged in, profile exists): - "I can donate" button (if Active + compatible) - "Already offered" message if duplicate • Offers section (owner only): - Each offer card: Donor info, Status badge - Actions: Accept (if Pledged) - After accept: Contact revealed - "Mark Complete" button • Accepted donor view: - Owner's contact revealed - "Mark Complete" button

Q15.29. Emergency cases handle kaise karte hain?

Ans. Current features: • Urgency levels: Critical/High/Medium/Low • Filters - urgency by sort • Guest posting - no registration Roadmap: • SMS notifications for Critical • Push notifications • Priority in search • Emergency contact directory • 24/7 helpline link Educational scope me limited.

Q15.30. Agar database corrupt ho jaaye?

Ans. Prevention: • SQL Server backups (daily) • Point-in-time recovery • Regular integrity checks • Migration history table Recovery: • Full backup restore • Transaction log restore (PITR) • Migration rollback Production me automated backups critical. Educational project - local DB, easy re-setup.

SECTION 16

Comparison Questions

X vs Y - 30+ comparisons - clear differences

Q16.1. ViewData vs ViewBag vs TempData?

Ans. ViewBag - dynamic property: @ViewBag.Message = "Hello"; @ViewBag.Message // in view ViewData - Dictionary: ViewData["Message"] = "Hello"; @ViewData["Message"] TempData - Persists across redirect: TempData["Toast"] = "Saved!"; // survives one redirect Best: Strongly-typed ViewModels. Type safety, IntelliSense.

Q16.2. Controller vs ControllerBase?

Ans. Controller - MVC controller base: • View(), PartialView() • RedirectToAction() • JSON helper methods ControllerBase - API controller base: • No view methods • Ok(), BadRequest(), NotFound() • Lighter Humara: DonorApiController : ControllerBase, others : Controller.

Q16.3. MVC vs Web API?

Ans. MVC: Views render, HTML response, users ke liye. Web API: JSON/XML data, mobile/SPA/3rd party ke liye. Same framework me: • Controller - MVC • ControllerBase - API • [ApiController] attribute - API conventions Humara: MVC app + ek REST endpoint (DonorApiController).

Q16.4. IActionResult vs ActionResult<T>?

Ans. IActionResult - Non-generic, multiple return types: public IActionResult Get(int id) { if (id <= 0) return BadRequest(); var donor = GetDonor(id); if (donor == null) return NotFound(); return Ok(donor); } ActionResult<T> - Generic, type-safe: public ActionResult<DonorDto> Get(int id) { if (id <= 0) return BadRequest(); var donor = GetDonor(id); if (donor == null) return NotFound(); return donor; // implicit Ok } Generic better for API - OpenAPI docs clear.

Q16.5. GET vs POST vs PUT vs PATCH vs DELETE?

Ans. GET - Read resource. Safe, idempotent. POST - Create new. Not safe, not idempotent. PUT - Full replace. Idempotent. PATCH - Partial update. Usually idempotent. DELETE - Remove. Idempotent. Humara: • GET /Donor/Search • POST /Donor/Create • POST /Donor/Edit/5 (form) • POST /Donor/Delete/5 (form) Note: MVC forms me POST for all state changes - HTML limitation.

Q16.6. 200 vs 201 vs 204?

Ans. 200 OK - Success, response body present. 201 Created - Naya resource bana, Location header me URL. 204 No Content - Success, koi body nahi (usually DELETE). Hum: MVC me 200 (HTML pages), API me 200 mostly. 201 API me create pe (roadmap).

Q16.7. 400 vs 401 vs 403?

Ans. 400 Bad Request - Client ki galti (validation fail). 401 Unauthorized - Not authenticated (login karo). 403 Forbidden - Authenticated but not authorized (permission nahi). Misnomer: 401 'Unauthorized' actually 'Unauthenticated' hai. Hum: Validation 400, logged out 401, IDOR 403.

Q16.8. ViewBag vs ViewModel?

Ans. ViewBag - dynamic, weak-typed, IntelliSense nahi. @ViewBag.Message ViewModel - strongly typed class: public class DonorListViewModel { public List<Donor> Donors { get; set; } public int TotalPages { get; set; } public string SearchTerm { get; set; } } @Model.Donors, @Model.TotalPages Hum: ViewModels use karte hain - better.

Q16.9. Interface vs Abstract Class?

Ans. Interface - contract only, multiple implement kar sakte, no state. Abstract Class - partial implementation, single inheritance, state allowed. Rule: 'Is-a' relationship → abstract. 'Can-do' capability → interface. Hum: IMessageCache interface (contract). AppUser abstract nahi - direct class.

Q16.10. Struct vs Class?

Ans. Struct - Value type (stack), copy by value, small data. No inheritance. Class - Reference type (heap), copy by reference, larger data. Inheritance. Use struct: Small immutable data (Point, Color, Money). Use class: Everything else. C# me default class.

Q16.11. List<T> vs IEnumerable<T> vs IQueryable<T>?

Ans. IEnumerable - In-memory iteration, LINQ-to-Objects. Deferred. IQueryable - Translated to SQL, remote execution. Deferred. List - Materialized, in-memory collection. var q1 = _context.Donors.Where(...); // IQueryable - no SQL yet var q2 = q1.AsEnumerable(); // IEnumerable - client eval var list = q1.ToList(); // List - SQL executed Best: IQueryable for DB, IEnumerable for iteration, List for materialization.

Q16.12. First vs FirstOrDefault vs Single vs SingleOrDefault?

Ans. First - First element, error if none. FirstOrDefault - First element or null. Single - Exactly one element, error if 0 or >1. SingleOrDefault - Exactly one or null, error if >1. Best practice: FirstOrDefaultAsync mostly. Single* when uniqueness expected. Hum: FindAsync, FirstOrDefaultAsync use karte hain.

Q16.13. Any vs Count vs Exists?

Ans. Any() - Boolean, koi bhi item hai? Early exit. Count() - Total number. Full iteration. Faster: Any() > Count() > 0 (short-circuits). Hum: Duplicate check me AnyAsync: var exists = await _context.DonationOffers .AnyAsync(o => o.RequestId == r && o.DonorId == d);

Q16.14. Select vs SelectMany?

Ans. Select - Transform each element 1:1. donors.Select(d => d.Name) // List<string> SelectMany - Flatten nested collections. requests.SelectMany(r => r.Offers) // All offers from all requests Equivalent to SQL: JOIN for flattening.

Q16.15. Where vs Take vs Skip?

Ans. Where - Filter by condition. Take(n) - First n elements. Skip(n) - Skip first n. Pagination: query.Skip((page-1)*size).Take(size) SQL: WHERE ... OFFSET n ROWS FETCH NEXT m ROWS ONLY.

Q16.16. Include vs ThenInclude?

Ans. Include - Direct navigation load: .Include(r => r.Offers) ThenInclude - Nested navigation: .Include(r => r.Offers).ThenInclude(o => o.Donor) Multiple Includes: .Include(r => r.Offers).Include(r => r.Owner) Without Include: Navigation properties null (lazy off).

Q16.17. Sync vs Async methods?

Ans. Sync - Blocks thread. Simple but wasteful. var data = context.Donors.ToList(); Async - Non-blocking. Better for I/O. var data = await context.Donors.ToListAsync(); Web apps: Always async. Server zyada requests handle kar sakta hai. Exception: CPU-bound work async me thread pool pe chalti hai - but web me usually I/O bound.

Q16.18. Task.Run vs async Task?

Ans. Task.Run - Offloads work to thread pool. CPU-bound ke liye. public Task<int> ComputeAsync() { return Task.Run(() => HeavyComputation()); } async Task - True async I/O: public async Task<int> FetchAsync() { return await _httpClient.GetFromJsonAsync<int>(url); } Web apps me: I/O - async Task. CPU-heavy - avoid (block server threads).

Q16.19. String vs StringBuilder?

Ans. String - Immutable. Concatenation creates new object. StringBuilder - Mutable. Efficient for multiple appends. var sb = new StringBuilder(); for (int i = 0; i < 1000; i++) sb.Append(i); Use: StringBuilder in loops. String for few concatenations. C# me compiler optimizes: $"a{b}c" is efficient.

Q16.20. Struct vs Record?

Ans. Struct - Value type, mutable typically. Record - Reference type, immutable by default, value equality, with-expressions. public record DonorDto(int Id, string Name); var d2 = d1 with { Name = "Updated" }; Record best for DTOs, immutable data. C# 9+.

Q16.21. var vs explicit type?

Ans. var - Compiler infers type. Compile-time static, runtime same. var name = "Ravi"; // string Explicit - Clear type: string name = "Ravi"; Use var: Type obvious from RHS. Use explicit: Clarity matters. Compilation identical - compiler doesn't care.

Q16.22. const vs readonly vs static readonly?

Ans. const - Compile-time constant. Literal only. Inlined at usage. public const string BloodGroup = "A+"; readonly - Runtime constant. Constructor me set. Per-instance. public readonly string Id; static readonly - Runtime constant, per-class. public static readonly string[] Groups = { ... }; Hum: DomainValues me const for strings, static readonly for arrays.

Q16.23. ref vs out vs in?

Ans. ref - Pass by reference (read+write). Initialized before call. out - Pass by reference (write only). Must assign inside. in - Read-only reference. Large structs ke liye. void Update(ref int x) { x = 10; } void Get(out int x) { x = 10; } void Read(in Point p) { /* read only */ }

Q16.24. Stack vs Queue vs LinkedList?

Ans. Stack - LIFO. Push/Pop. Undo, recursion. Queue - FIFO. Enqueue/Dequeue. Processing order. LinkedList - Doubly linked. Fast insert/delete middle. Slow index access. Hum: ICollection in navigation properties (EF Core handles).

Q16.25. Dictionary vs HashSet vs List?

Ans. Dictionary<K,V> - Key-value pairs. Fast lookup by key. var cache = new Dictionary<int, Donor>(); HashSet<T> - Unique items. Fast Contains. var groups = new HashSet<string>(); List<T> - Ordered collection. Index access. Hum: Dictionary me cache, HashSet me unique groups (search compatible), List me ordered results.

Q16.26. try-catch vs global exception handler?

Ans. try-catch - Specific errors handle karo where you can recover. try { await _context.SaveChangesAsync(); } catch (DbUpdateConcurrencyException) { // Handle specifically } Global handler - Catch-all for unexpected: app.UseExceptionHandler("/Home/Error"); Both - Depth defense. Specific first, generic fallback.

Q16.27. IEnumerable vs ICollection vs IList?

Ans. IEnumerable<T> - Iteration only. Base interface. ICollection<T> - IEnumerable + Count, Add, Remove. IList<T> - ICollection + Index access (this[int]). Best practice: • Parameter: IEnumerable (most generic) • Return: ICollection ya IList (specific) • Property: ICollection for navigation Hum: ICollection<DonationOffer> for Request.Offers navigation.

Q16.28. File upload: IFormFile?

Ans. public IActionResult Upload(IFormFile file) { if (file == null || file.Length == 0) return BadRequest(); var path = Path.Combine("uploads", file.FileName); using var stream = System.IO.File.Create(path); await file.CopyToAsync(stream); } Form: <form enctype="multipart/form-data"> <input type="file" name="file" /> </form> Security: Validate size, type, sanitize filename. Humara: No file uploads (not needed).

Q16.29. Cookie vs JWT vs Session?

Ans. Cookie - Server-side session, opaque token. Auto-bhejta hai. HTTPS+HttpOnly secure. JWT - Stateless, self-contained (claims signed). Client sends Authorization header. Mobile/API friendly. Session - Server storage, ID in cookie. Memory heavy at scale. Hum: Cookie (Identity default) - same-origin MVC ke liye best.

Q16.30. OAuth vs OpenID Connect?

Ans. OAuth 2.0 - Authorization framework. 'Access to resource'. Use: 'Allow app X to access my Google Drive'. OpenID Connect - Authentication layer on OAuth. 'Who is user'. Use: 'Login with Google' - identity token. Hum: Local login (Identity). External login roadmap.

Q16.31. Authentication vs Authorization?

Ans. Authentication - WHO you are. • Login, password, 2FA • Returns identity • Cookie/token issue Authorization - WHAT you can do. • Role check, policy • [Authorize(Roles="Admin")] • Access grant/deny Sequence: Auth first, then authz.

SECTION 17

Scenario-Based Questions

Real situations - kya hoga, kaise handle karo

Q17.1. Agar database temporarily down ho jaaye?

Ans. EF Core EnableRetryOnFailure automatically retry (5 times, 10s delay). If still fails: • Exception thrown • Global handler → /Home/Error • User sees styled 500 page (no stack trace) • Server logs full details Next successful request - everything back to normal. For critical operations: Circuit breaker pattern (Polly) roadmap me.

Q17.2. Agar user 5 baar galat password daale?

Ans. Lockout trigger: • After 5 failures, account lock 5 minutes • Correct password bhi reject • Message: 'Account locked' Counter: Reset on successful login ya lockout expiry. Admin manually unlock: _userManager.SetLockoutEndDateAsync(user, null) Defense: Brute-force impractical (5-min gaps).

Q17.3. Agar banned user cookies ke saath request bheje?

Ans. OnValidatePrincipal event - every request: 1. Cookie decrypt 2. User fetch from DB 3. if (user.IsBanned) → RejectPrincipal() 4. SignOutAsync() Effect: • Cookie invalidated • User signed out • Next request → login page Even existing session cannot continue.

Q17.4. Agar Admin khud ko ban karne ki koshish kare?

Ans. Self-protection rule: if (targetUserId == currentUserId) return BadRequest("Cannot ban yourself"); Same for: • Delete self • Demote self Reason: Prevent accidental lockout. SuperAdmin bhi self-protect. UI: Button disabled for self.

Q17.5. Agar Admin doosre Admin ko ban karne ki koshish kare?

Ans. Role hierarchy check: if (currentUserIsAdmin && !currentUserIsSuperAdmin) { if (targetRoles.Contains("Admin") || targetRoles.Contains("SuperAdmin")) return Forbid(); } Result: 403 error toast 'Insufficient permissions'. Admin can only ban regular Users. SuperAdmin can ban Admins but not other SuperAdmins (immutable).

Q17.6. Agar SuperAdmin ko koi ban karne ki koshish kare?

Ans. Immutable rule: if (targetRoles.Contains("SuperAdmin")) return Forbid("SuperAdmin is immutable"); Even SuperAdmin can't ban another SuperAdmin. Reason: System lockout prevention. Root authority must always exist. Design decision - fail-safe.

Q17.7. Agar do donors same request pe offer karein?

Ans. Both succeed: • Unique index (RequestId, DonorId) - different DonorIds • Both offers Pledged status Owner dekhta hai 2 offers. On accept: • Accepted offer → Accepted • Other → Auto-Rejected Owner choice: Location, contact, response time consider karke accept.

Q17.8. Agar same donor same request pe dobara offer kare?

Ans. Server check (pre): var exists = await _context.DonationOffers .AnyAsync(o => o.RequestId == r && o.DonorId == d); if (exists) return BadRequest("Already offered"); DB constraint (post): Unique index violation → DbUpdateException UI: Button hide/disabled for duplicate. User message: 'You already offered on this request.'

Q17.9. Agar donor apna offer cancel kare after accept?

Ans. State check: if (offer.Status != OfferStatus.Pledged) return BadRequest("Cannot cancel after acceptance"); Reason: Accepted = commitment. Cancel = medical workflow break. UI: Cancel button only for Pledged status. Edge case: Emergency - owner se coordinate karke unaccept. Roadmap: Owner-initiated removal.

Q17.10. Agar request already fulfilled hai aur naya offer aaye?

Ans. Server check: if (request.Status == RequestStatus.Fulfilled) return BadRequest("Request already fulfilled"); UI: Button hidden - only Active requests me offer visible. Race condition: Check pe Active, save pe Fulfilled? Unique index + transaction prevent.

Q17.11. Agar search me invalid blood group daale?

Ans. Two layers: 1. Client-side: [RegularExpression] attribute - form validation 2. Server-side: BloodCompatibility.GetCompatibleDonorGroups("XYZ") → empty array → 0 results Response: Empty list, isExactMatch: false. Better: 400 Bad Request with error message.

Q17.12. Agar concurrency issue ho - do users same donor edit karein?

Ans. RowVersion check: 1. User1 loads (RowVersion=1) 2. User2 loads (RowVersion=1) 3. User1 saves (RowVersion→2) 4. User2 saves attempt: UPDATE ... WHERE RowVersion=1 0 rows affected → DbUpdateConcurrencyException 5. Catch block: TempData error: 'Record updated by someone else, please refresh' Silent overwrite prevented.

Q17.13. Agar user 2 tabs me login kare?

Ans. Cookie browser-wide: • Same session both tabs • Logout in one → logout in other (next request) • Role change → both tabs see new role Standard behavior. No issue. Agar different users chahiye: Different browsers ya Incognito.

Q17.14. Agar cookie expire ho mid-session?

Ans. SlidingExpiration = true: • Active users ka cookie refresh hota hai • 7-day inactivity pe expire After expiry: • Next request pe authentication fails • Redirect to /Account/Login?ReturnUrl=... • Login → same page pe wapas User friendly.

Q17.15. Agar SQL injection attempt search box me?

Ans. EF Core parameterized query: User input: ' OR '1'='1 Generated SQL: SELECT ... WHERE BloodGroup = @p0 @p0 = '" OR '1'='1" Literal string - SQL code nahi. Result: 0 results ya error. Injection fails. Even safer: [RegularExpression] validates blood group format.

Q17.16. Agar XSS attempt contact form me?

Ans. Razor output encoding: User input: <script>alert('XSS')</script> Rendered: <script>alert('XSS')</script> Browser me: <script>alert('XSS')</script> as text - execute nahi. Defense: Automatic + server-side validation. Agar Html.Raw use kiya - careful, only trusted content.

Q17.17. Agar admin panel me 100 messages aayein?

Ans. Pagination (roadmap - abhi all): Messages.Skip((page-1)*20).Take(20) Unread count cached: • 60s TTL • Not DB queried per page load Bulk operations: • Mark all read - batch update • Delete all - batch delete Memory: Manageable with pagination.

Q17.18. Agar user apna password bhool jaaye?

Ans. Current: No self-service password reset. Workaround: • Admin manually reset • New password set in DB • User informed via phone/email Roadmap: Email-based reset • Forgot Password link • Token generation • Email with reset link • Token validation + expiry Educational scope me limited.

Q17.19. Agar DB migration fail ho jaaye production me?

Ans. Prevention: • Test migration on staging first • Backup before migrate • Migration script review Recovery: 1. Revert migration: dotnet ef database update PreviousMigration 2. Or rollback from backup 3. Fix migration code 4. Re-apply Hum: Local DB, easy re-setup.

Q17.20. Agar application crash ho jaaye?

Ans. Layers: 1. ExceptionHandler middleware: • Development: detailed error • Production: styled 500 page 2. Logs me full details (Serilog/ILogger) 3. Kestrel restart (systemd/IIS) 4. Health check endpoint (roadmap) Graceful degradation - critical paths up, others down.

Q17.21. Agar user bulk data import kare?

Ans. Currently: No bulk import. If added: • CSV upload form • Parse + validate • Batch insert (EFCore.BulkExtensions) • Progress bar • Error report (which rows failed) Performance: 10K+ rows bulk extension zaroori. Regular loop slow.

Q17.22. Agar user ko real-time notification chahiye?

Ans. Current: Page reload needed. Solution: SignalR // Server await _hubContext.Clients.User(ownerId) .SendAsync("OfferReceived", offerId); // Client connection.on("OfferReceived", (id) => { showToast(`New offer #${id}`); updateBadge(); }); Use: Instant pledge notification, live dashboard. Roadmap me hai.

Q17.23. Agar user 10 MB profile picture upload kare?

Ans. Currently: No file uploads. If added: 1. Size check: if (file.Length > 2 * 1024 * 1024) → reject 2. Type check: JPEG/PNG only 3. Store: ~/wwwroot/uploads/{guid}.jpg 4. Resize: ImageSharp 5. DB: Store path/URL, not blob Security: Sanitize filename, scan for malware (production), CDN serve.

Q17.24. Agar admin bot se spam messages aayein?

Ans. Mitigation: 1. Rate limiting (roadmap): [EnableRateLimiting("contact-form")] 2. CAPTCHA (roadmap): reCAPTCHA v3 invisible 3. IP-based blocking 4. Content filtering 5. Manual deletion in admin Currently: Manual only. Simple approach for educational project.

Q17.25. Agar server pe 10,000 concurrent users ho?

Ans. Scale strategy: 1. Load balancer (Nginx/ALB): Multiple app servers 2. Stateless app: Cookie-based auth already stateless 3. Distributed cache (Redis): IMemoryCache → IDistributedCache 4. DB scale: Read replica, connection pooling, indexes 5. CDN: Static files (CSS/JS/images) 6. Auto-scaling: Kubernetes HPA / Azure autoscale Single server: ~500-1000 RPS handle kar sakta hai.

Q17.26. Agar bugs production me mile?

Ans. Response process: 1. Incident log + severity 2. Reproduce in staging 3. Fix + unit test 4. Code review 5. Deploy to staging 6. Verify 7. Deploy to production (canary if possible) 8. Monitor 9. Post-mortem Process: Detect → Respond → Recover → Learn. Blame-free culture - focus on systems, not people.

SECTION 18

Viva Preparation Tips

Strategies, must-prepare, mock viva, first 5 minutes

Q18.1. Viva me pehla impression kaise banao?

Ans. 1. Confident greeting - 'Good morning sir/ma'am' 2. Project ka 30-second pitch: 'BloodConnect is a blood donation platform built on ASP.NET Core MVC, with medical compatibility logic, pledge-based workflow, and role-based admin.' 3. Smile, eye contact, steady voice 4. Notebook ready - whiteboard opportunity ke liye 5. Clear bolo - tak tak mat bolo

Q18.2. Agar koi question nahi pata to kya karo?

Ans. NEVER say just 'Pata nahi'. Say: 'Sir, main is topic pe deeply nahi socha. Mera best understanding yeh hai ki... [related concept explain karo]. Main isko aur padhunga.' Yeh dikhata hai: • Honesty • Thought process • Learning attitude • Related knowledge Examiner ko yeh pasand aata hai vs fake blabber.

Q18.3. Diagram kab banana chahiye?

Ans. Agar examiner permission de ya space available ho: Best diagrams: • MVC request flow • Donation workflow state machine • Authentication flow (login → cookie → request) • Architecture (Controllers/Models/Views) • ER diagram (entities + relationships) Diagrams = 2x clarity. Plus confidence boost. Sochne ka time bhi milta hai.

Q18.4. Viva me kitna deep jaana chahiye?

Ans. Rule: Level 1 pe ruk■, Level 2 pe hint do, Level 3 pe offer karo. Example - 'MVC kya hai?': • L1: Model-View-Controller pattern • L2: Separation of concerns • L3: Request flow: Routing → Controller → Model → View Agar examiner aur poochta hai to L3+ pe jao. Bahut deep jaake confuse mat karo.

Q18.5. Kaunse questions prepare karne MUST hain?

Ans. Top 20 must-prepare: 1. Project kya hai + kyun 2. MVC pattern + request flow 3. .NET vs .NET Framework 4. Authentication vs Authorization 5. EF Core ORM kya hai 6. DbContext + migrations 7. SQL injection + XSS + CSRF 8. IDOR + ownership checks 9. Role hierarchy in project 10. Blood compatibility rules 11. Pledge workflow state machine 12. Caching (MessageCache) + invalidation 13. Async/await 14. DI + lifetimes 15. REST API principles 16. GET vs POST 17. Cookie authentication 18. RowVersion concurrency 19. Seeder + Admin default 20. Testing approach

Q18.6. Agar examiner counter-question kare?

Ans. Confidence maintain karo. Ye achha sign hai - wo engage ho rahe hain. Steps: 1. Counter-question dhyaan se suno 2. Repeat in your words (clarify): 'Sir, aap yeh pooch rahe hain ki...?' 3. Structured answer do 4. Reference project example 5. Agar galti hui thi - acknowledge, correct karo Defensive mat bano - engage karo.

Q18.7. Practical demo kaise kare?

Ans. Agar laptop allowed hai: 1. App start (dotnet run) 2. Login as admin 3. Show 2-3 critical flows: • Register → donor profile • Search → compatibility check • Pledge → accept → complete • Cache invalidation (open 2 tabs) 4. Show code snippet - key files Demo = strong impression. But verify karo sab kaam kar raha hai beforehand.

Q18.8. Kya examiner se pooch sakte hain?

Ans. Haan, but judiciously: Achhe questions: • 'Sir, kya aap chahenge main XYZ feature kaise implement karta?' • 'Sir, is approach me koi alternate better hai kya?' • 'Sir, production ke liye kya improvements chahiye?' Avoid: • 'Sir, yeh kaam kaise hota hai?' (basic) • 'Sir, iska syntax kya hai?' (Google-able) Dikhao ki tum sochte ho.

Q18.9. Agar answer me galti ho jaaye?

Ans. Acknowledge quickly, correct karo: 'Sir, sorry, main galat bol gaya. Sahi yeh hai ki...' Better than: • Continue with error • Defend wrong answer • Blame others Examiner dekhta hai: • Honesty • Self-correction ability • Composure Yeh professional trait hai.

Q18.10. Agar bahut questions na aayein?

Ans. Achha sign hai - confident answers diye. But be proactive: 'Sir, agar aap chahein to main project ke ek interesting feature pe detail me bata sakta hoon?' Or: 'Sir, ek challenge tha jo humne encounter kiya - [concurrency/auto-reject]. Bataun?' Lead the conversation - dikhata hai confidence + prepared.

Q18.11. Presentation ke liye tips?

Ans. Agar presentation required: Slides (10-12): 1. Title + team 2. Problem statement 3. Solution overview 4. Tech stack 5. Architecture diagram 6. Key features (2-3 slides) 7. Blood compatibility logic 8. Pledge workflow state machine 9. Security measures 10. Caching strategy 11. Challenges + learnings 12. Roadmap + thanks Time: 8-10 min max. Rehearse karo. Stand tall, speak slow, hands visible.

Q18.12. Group viva me kya strategy?

Ans. Agar team me ho: 1. Roles divide karo beforehand: • Frontend - Views, UI, JS • Backend - Controllers, EF, Domain • Security - Auth, roles • DB - Schema, migrations 2. Ek lead kare intro, others supplementary 3. Specific questions apne expert ko do (eye contact ya subtle nod) 4. Overlap avoid karo - ek banda answer de, dusra add kare agar zaroori 5. Agar team member stuck ho - politely interject: 'Sir, main isme add karna chahunga...'

Q18.13. Agar examiner frustrated lag rahe hain?

Ans. Calm raho. Reasons: • Time pressure • Complex answer chahiye • Same question ke variants Response: 1. Bolo 'Sir, main understand karta hoon' 2. Structured, concise answer 3. Poocho: 'Sir, kya main specific direction me jaun?' 4. Time efficient bano Never argue, never defensive. Adjust tone - match theirs.

Q18.14. Pehle 5 minutes critical kyun?

Ans. Examiner first impression se question decide karta hai. First 5 minutes me: 1. Confident greeting 2. Clear project pitch (30 sec) 3. Slide/demo quick 4. Top 2-3 questions anchor Agar ye strong → examiner impressed → friendly questions. Agar weak → skeptical → tough questions. So: first 5 minutes = 50% of viva weight.

Q18.15. Mock viva kaise kare?

Ans. Dost ke saath: 1. Actual viva jaisa set up (table, chair) 2. Random questions from this guide 3. Timer 15 min 4. Recording - baad me dekho 5. Feedback: Speed, confidence, clarity At least 2-3 mock sessions. Bolke practice - likhne se alag hai.

Q18.16. Ek ghante me kya revise karo viva se pehle?

Ans. 1. Project pitch (5 min rehearse) 2. Section 1: Project Overview (10 min) 3. Section 7-8: Blood + Workflow (10 min) 4. Section 6: Auth (5 min) 5. Section 14: Project-specific (15 min) 6. Section 18: These tips (5 min) Skip: • Super detailed deep dives • New topics Focus: Refresh known, not learn new. Deep breath. Hydrate. Smile.

Q18.17. Agar lagta hai bilkul hi nahi aata?

Ans. Sabko lagta hai. Universal feeling. But: 1. Tumne project banaya hai - code likha hai. Ye tumhare fav■ur me hai. 2. 90% questions basic + project-specific hote hain 3. Yeh guide padha hai - 450+ Q&A cover 4. Tum sirf bolke practice chahiye Repeat after me: 'I built this. I know this. I've got this.' Confidence = 50% of viva. Baaki 50% hai answers. Tumhare paas dono hain.

SECTION 19

MonsterASP.NET & Deployment

Hosting, FTP, SSL, migrations, common errors

Q19.1. MonsterASP.NET kya hai?

Ans. MonsterASP.NET ek Windows-based shared hosting provider hai jo ASP.NET Core, .NET Framework, PHP, aur SQL Server support karta hai. Yeh free aur paid plans deta hai - students aur small projects ke liye best hai. Isme aap apni .NET app upload karke live kar sakte hain - bina apna server manage kiye. Features: • Windows Server + IIS • SQL Server databases (free plan me bhi) • FTP/WebDeploy se upload • Custom domain support • Free SSL certificate • One-click .NET version selection

Q19.2. MonsterASP kyun choose kiya deployment ke liye?

Ans. Reasons: • Free tier available (students ke liye best) • Windows + IIS - ASP.NET Core native support • SQL Server included (humar■ database chahiye) • FTP/WebDeploy se easy upload • Free SSL certificate • Custom domain connect kar sakte hain • Management panel simple hai • 24/7 support (paid plans) Alternative: • Somee.com - free ASP.NET + SQL Server • Azure App Service - paid, scalable • AWS Elastic Beanstalk - complex MonsterASP best for educational projects.

Q19.3. MonsterASP pe deploy karne ka step-by-step process?

Ans. 1. Account banao - monsterasp.net pe register 2. Free plan select karo (ya paid) 3. Website create karo - dashboard se 4. .NET version select karo (8, 9, ya 10) 5. SQL Server database create karo 6. Connection string copy karo 7. Local pe app publish karo: dotnet publish -c Release -o published 8. FTP credentials lo 9. FileZilla ya WinSCP se upload karo published/ folder ka content 10. Web.config me sahi .NET version set 11. Environment variables set karo (connection string) 12. Migrations apply karo (ya SQL script run karo) 13. Domain configure karo 14. SSL enable karo 15. Test karo - https://yourdomain.com

Q19.4. MonsterASP pe connection string kaise set karte hain?

Ans. 1. Dashboard → Databases → SQL Server 2. Connection string copy karo - usually yeh format: Server=xxx.monsterasp.net;Database=xxx_db;User Id=xxx;Password=xxx; 3. Dashboard → Website → Environment Variables (ya web.config) 4. Set karo: ConnectionStrings__DefaultConnection = "your-connection-string" Note: Double underscore (__) nested config ke liye hai. 5. Restart app (auto ya manual). 6. Test - app start hone pe migration check karega. Best practice: Password environment variable me, appsettings.json me placeholder rakho.

Q19.5. MonsterASP pe database migration kaise karte hain?

Ans. Do tareeke: 1. SQL Script (recommended for shared hosting): Local pe script generate karo: dotnet ef migrations script -o migrate.sql Phir MonsterASP SQL panel me execute karo. 2. Direct from local: dotnet ef database update --connection "<monsterasp-conn-string>" Local machine se MonsterASP DB pe connect karo. Shared hosting me direct EF command nahi chalti - isliye script better. Auto-migration on startup: app.Services.GetRequiredService<ApplicationDbContext>().Database.Migrate(); Note: Production me careful - backup pehle lo.

Q19.6. Free plan me kya limitations hain MonsterASP pe?

Ans. • Limited disk space (usually 1GB) • Limited bandwidth (monthly cap) • SQL Server database size limit (usually 100-500MB) • App pool recycle frequent (idle pe) • Custom domain limited • No dedicated resources • Email sending limited • SSL - free tier me basic Educational project ke liye: kaafi hai. Production ke liye: paid plan ya Azure/VPS.

Q19.7. MonsterASP pe deployment ke liye publish kaise karte hain?

Ans. 1. Local pe: dotnet publish -c Release -o published 2. Folder structure: published/ ■■■ BloodConnect.dll ■■■ BloodConnect.exe ■■■ appsettings.json ■■■ web.config ■■■ wwwroot/ ■■■ ... 3. Web.config MonsterASP ke liye: <?xml version="1.0" encoding="utf-8"?> <configuration> <location path="." inheritInChildApplications="false"> <system.webServer> <handlers> <add name="aspNetCore" path="*" verb="*" modules="AspNetCoreModuleV2" resourceType="Unspecified" /> </handlers> <aspNetCore processPath="dotnet" arguments=".\BloodConnect.dll" stdoutLogEnabled="false" stdoutLogFile=".\logs\stdout" hostingModel="inprocess" /> </system.webServer> </location> </configuration> 4. FTP se upload karo.

Q19.8. Hosting model kya hai - inprocess vs outofprocess?

Ans. InProcess: • App IIS ke andar chalta hai • Faster (no extra hop) • IIS ke settings follow karta hai • Recommended for Windows/IIS OutOfProcess: • IIS ek reverse proxy banta hai • App alag Kestrel process me • Cross-platform deployments me common web.config: hostingModel="inprocess" // ya "outofprocess" MonsterASP me: inprocess recommended (default).

Q19.9. MonsterASP pe SSL/HTTPS enable kaise karte hain?

Ans. 1. Dashboard → SSL Certificates 2. Free SSL (Let's Encrypt) select karo 3. Domain verify karo (usually auto) 4. Certificate issue hone me 5-10 min 5. Auto-renew enable karo App me: • UseHttpsRedirection() middleware • Cookies: SecurePolicy = Always (production) • HSTS enable karo Test: https://yourdomain.com - green lock dikhna chahiye.

Q19.10. FTP vs WebDeploy - kaunsa better hai?

Ans. FTP (FileZilla, WinSCP): • Manual, file-by-file • Slow for many files • Simple setup • Free tools WebDeploy: • Automated, Visual Studio se • Incremental (only changed files) • Fast for multiple deploys • Requires WebDeploy package on host MonsterASP: FTP commonly used. Paid plans me WebDeploy. Recommendation: FTP for first deploy, WebDeploy for iterations.

Q19.11. MonsterASP pe app start nahi ho raha - kya check karo?

Ans. 1. .NET version match karo - local vs hosting 2. web.config check - dotnet path sahi? 3. Connection string verify karo - valid hai? 4. Logs check karo - /logs folder ya dashboard 5. File permissions - wwwroot read/write? 6. Port conflicts - IIS settings 7. Environment variables - sahi set hain? 8. App pool recycle karo (dashboard se) 9. Database accessible hai? - firewall check 10. Custom error mode off - detailed error dekho MonsterASP panel me error log viewer hota hai.

Q19.12. MonsterASP pe custom domain kaise add karte hain?

Ans. 1. Dashboard → Domains → Add Domain 2. Domain enter karo (jaise bloodconnect.com) 3. DNS records update karo: • A record → MonsterASP IP • CNAME www → your-monsterasp-subdomain 4. Wait for DNS propagation (5 min - 48 hours) 5. SSL enable karo 6. Test: http://yourdomain.com MonsterASP subdomain bhi milta hai free: yourapp.monsterasp.net Custom domain paid plans me.

Q19.13. Production me logging kaise karte hain MonsterASP pe?

Ans. 1. Dashboard → Logs → Error logs 2. Stdout logs web.config se: stdoutLogEnabled="true" stdoutLogFile=".\logs\stdout" 3. Serilog file logging: Log.Logger = new LoggerConfiguration() .WriteTo.File("logs/app-.log", rollingInterval: RollingInterval.Day) .CreateLogger(); 4. Logs folder FTP se access karo. Note: Shared hosting me log rotation important - disk fill na ho.

Q19.14. MonsterASP pe environment variables kaise use karte hain?

Ans. 1. Dashboard → Website → Environment Variables 2. Add karo: Key: ASPNETCORE_ENVIRONMENT Value: Production Key: ConnectionStrings__DefaultConnection Value: Server=... 3. Save + restart. Alternative: web.config me: <aspNetCore ...> <environmentVariables> <environmentVariable name="ASPNETCORE_ENVIRONMENT" value="Production" /> <environmentVariable name="ConnectionStrings__DefaultConnection" value="..." /> </environmentVariables> </aspNetCore> Best: Panel me set karo - web.config me nahi (git me commit nahi hoga).

Q19.15. Deployment ke baad kya test karo?

Ans. 1. Home page load - / 2. Register/Login - authentication 3. Donor CRUD - create, edit, delete 4. Search - compatibility filter 5. Blood request - post as guest 6. Offer workflow - pledge, accept, complete 7. Admin panel - role check 8. Messages - cache invalidation 9. 404 page - styled error 10. HTTPS redirect - auto 11. Mobile responsive - Chrome DevTools 12. Performance - Lighthouse score Agar sab pass → deployment successful.

Q19.16. Common deployment errors aur solutions?

Ans. • 500 Internal Server Error: → Check logs, connection string, .NET version • HTTP Error 500.30 - ANCM In-Process Start Failure: → App startup me crash - logs dekho • HTTP Error 500.31 - Failed to load ASP.NET Core runtime: → .NET version mismatch - hosting pe correct version install • HTTP Error 502.5 - Process Failure: → App start nahi hui - check dotnet path • Directory listing denied: → Default document add karo (index.html / app) • Database connection fail: → Firewall, credentials, IP whitelisting

Q19.17. Kya hum Azure pe deploy kar sakte the?

Ans. Haan, but: • Cost - paid only (free tier limited) • Setup - complex vs shared hosting • Learning curve - Azure portal, ARM templates Azure advantages: • Auto-scaling • Managed services (DB, cache, CDN) • DevOps integration • Global regions For educational project: MonsterASP/Somee simpler. For enterprise/production: Azure better. Roadmap me Azure deployment bhi hai.

Q19.18. MonsterASP free tier vs paid tier?

Ans. Free: • 1 website • 1GB disk • SQL Server 100MB • Limited bandwidth • Shared IP • Free SSL • No SLA Paid: • Multiple websites • 5-50GB disk • Larger DB (500MB - 5GB) • More bandwidth • Dedicated IP (higher plans) • Priority support • SLA uptime • WebDeploy, scheduled tasks Students: free sufficient. Business: paid worth it.

Q19.19. Deployment ke baad performance kaise improve karo?

Ans. 1. Enable compression - Gzip/Brotli 2. Static files caching - Cache-Control headers 3. Output caching (roadmap) 4. Distributed cache (Redis) - scale pe 5. Database indexing - FK columns, search fields 6. AsNoTracking - read-only queries 7. Image optimization - WebP, lazy loading 8. CDN - Cloudflare free tier 9. Minify CSS/JS - bundling 10. Precompiled views - Razor compilation

Q19.20. Kya hum CI/CD setup kar sakte hain MonsterASP ke saath?

Ans. Haan, but limited: GitHub Actions example: - FTP upload action - WebDeploy action name: Deploy on: push: branches: [main] jobs: deploy: runs-on: ubuntu-latest steps: - uses: actions/checkout@v3 - uses: actions/setup-dotnet@v3 with: dotnet-version: '10.x' - run: dotnet publish -c Release -o ./publish - uses: SamKirkland/FTP-Deploy-Action@4.3.0 with: server: ftp.monsterasp.net username: ${{ secrets.FTP_USER }} password: ${{ secrets.FTP_PASS }} local-dir: ./publish/ Benefits: Auto-deploy on push, no manual FTP.

Q19.21. MonsterASP me SQL Server kaise manage karte hain?

Ans. 1. Dashboard → SQL Server → Management 2. Tools available: • Web-based query editor • Backup/Restore • User management • Import/Export (CSV, SQL) • Scheduled backups (paid) 3. External access: • Enable remote connections • Add your IP to whitelist • Connect via SSMS/SQL Server Management Studio • Azure Data Studio (cross-platform) 4. Connection string - dashboard se copy.

Q19.22. Deployment checklist - production ke liye?

Ans. Pre-deploy: ■ ASPNETCORE_ENVIRONMENT=Production ■ Connection string env var me ■ Seed admin password change ■ appsettings.Production.json review ■ HTTPS only ■ Debug mode off ■ Detailed errors off ■ Migration script ready ■ Backup current DB ■ Static files cache-busting enabled Post-deploy: ■ All pages load ■ Login works ■ Register works ■ Donor CRUD works ■ Search returns results ■ Pledge workflow works ■ Admin panel accessible ■ 404/500 pages styled ■ SSL valid ■ Mobile responsive ■ Logs accessible

Q19.23. Kya MonsterASP pe background jobs chala sakte hain?

Ans. Free tier: Generally no. Paid tier: Scheduled tasks available. Alternatives: 1. Windows Service - but shared hosting me nahi 2. Hangfire - in-app scheduler • Uses DB for job storage • Dashboard included • Works on shared hosting 3. Quartz.NET - in-app scheduler Use cases: • Daily email digest • Cleanup old requests • Auto-cancel stale pledges Humare project me: koi background job nahi (roadmap).

Q19.24. MonsterASP vs VPS me difference?

Ans. MonsterASP (Shared): • Multiple apps same server • Managed by provider • Cheap ($0-10/month) • Limited control • Perfect for students VPS (Virtual Private Server): • Dedicated resources • Full root access • More expensive ($10-50/month) • You manage everything (security, updates) • Better for production scale Humare educational project: MonsterASP perfect. Growing startup: VPS ya Azure.

SECTION 20

Bilkul Basic Questions

Computer, RAM, OS, variables, loops - beginner level

Q20.1. Computer program kya hota hai?

Ans. Computer program instructions ka set hai jo computer ko batata hai kya karna hai. Yeh ek specific language me likha jaata hai (C#, Python). Steps: 1. Instructions likho (source code) 2. Compile/interpret karo 3. Computer execute kare 4. Output mile Real life example: Recipe book ki tarah - instructions follow karo, dish banti hai.

Q20.2. Software aur Hardware me difference?

Ans. Hardware: Physical parts - monitor, keyboard, CPU, RAM, hard disk. Chhu sakte ho. Software: Programs aur instructions - OS, apps, games. Chhu nahi sakte, dekh nahi sakte. Analogy: Hardware = body, Software = brain + thoughts. Donon ek doosre ke bina kaam nahi karte.

Q20.3. Operating System kya hai?

Ans. OS ek software hai jo hardware aur user ke beech mediator ka kaam karta hai. Programs ko run karne deta hai, resources manage karta hai. Examples: • Windows - most common • Linux - servers, Android • macOS - Apple • iOS, Android - mobile Kaam: • Process management • Memory management • File system • User interface • Security

Q20.4. RAM aur ROM me difference?

Ans. RAM (Random Access Memory): • Volatile - power off pe data gone • Temporary storage • Fast access • Running programs + data ROM (Read Only Memory): • Non-volatile - data persists • Permanent storage • Slower • Firmware, BIOS stored Analogy: RAM = study table (aaj ka kaam), ROM = bookshelf (permanent).

Q20.5. Compiler kya karta hai?

Ans. Compiler ek program hai jo human-readable source code ko machine code me convert karta hai. C# example: Program.cs → csc compiler → BloodConnect.dll (IL) Phir JIT runtime → machine code Kaam: 1. Syntax check 2. Type check 3. Optimization 4. Code generation Compile-time errors: Tab pakde jaate hain. Runtime errors: Execute karte waqt.

Q20.6. Bug aur Debugging kya hai?

Ans. Bug: Program me error ya defect. 'Bug' naam 1940s se aaya - actual moth jo computer me phasi thi. Types: • Syntax errors - galat code structure • Logic errors - sahi likha but galat kaam • Runtime errors - execution pe crash Debugging: Bugs dhoondhna aur fix karna. Tools: • Visual Studio debugger • Breakpoints • Watch variables • Console logs • Print statements (basic)

Q20.7. Algorithm kya hai?

Ans. Algorithm ek step-by-step procedure hai problem solve karne ka. Properties: • Finite steps • Clear instructions • Input → Output • Unambiguous Example - Linear search: 1. Start from first element 2. Check if matches target 3. If yes, return index 4. If no, move to next 5. Repeat till end 6. If not found, return -1 Efficiency matters - different algorithms different speed.

Q20.8. Flowchart kya hai?

Ans. Diagram jo algorithm ya process ko visual form me dikhata hai. Symbols: • Oval - Start/End • Rectangle - Process/Step • Diamond - Decision • Parallelogram - Input/Output • Arrow - Flow direction Example - Login flow: [Start] → [Enter email/password] → <Valid?> ↓ Yes → [Home page] ↓ No → [Error message] → [Retry]

Q20.9. Frontend vs Backend?

Ans. Frontend: User jo dekhta hai aur interact karta hai. • HTML, CSS, JavaScript • Bootstrap, React, Angular • Browser me run hota hai Backend: Server-side logic + database. • C#, Python, Node.js • ASP.NET Core, Django • Server pe run hota hai Analogy: Restaurant - Frontend = menu + waiter, Backend = kitchen + chef. Humara BloodConnect: Dono - Razor views (frontend) + Controllers (backend).

Q20.10. Client aur Server me difference?

Ans. Client: Request bhejne wala. Aapka browser, mobile app. Server: Request receive karne wala. Web server + application. Flow: Client → Request (HTTP) → Server Server → Response (HTML/JSON) → Client Types of servers: • Web server (IIS, Nginx) • Application server (.NET app) • Database server (SQL Server) • Mail server

Q20.11. Database kya hai?

Ans. Data ka organized collection jahan information store hoti hai. Types: • Relational (SQL) - tables, rows, columns. MySQL, SQL Server, PostgreSQL. • NoSQL - documents, key-value. MongoDB, Redis. Relational me: • Table = Entity (Donors) • Row = Record (ek donor) • Column = Field (Name) • Primary key = unique ID • Foreign key = relationship Hum: SQL Server use karte hain.

Q20.12. DBMS kya hai?

Ans. Database Management System - software jo databases manage karta hai. Kaam: • Data store karna • Query karna (SELECT, INSERT) • Security • Backup/recovery • Concurrency control • Integrity Examples: SQL Server, MySQL, Oracle, PostgreSQL. SQL Server Express free version use kar rahe hain.

Q20.13. SQL kya hai?

Ans. Structured Query Language - database se baat karne ki language. Categories: • DDL - Data Definition (CREATE, ALTER, DROP) • DML - Data Manipulation (SELECT, INSERT, UPDATE, DELETE) • DCL - Data Control (GRANT, REVOKE) • TCL - Transaction Control (COMMIT, ROLLBACK) Example: SELECT Name, BloodGroup FROM Donors WHERE Age > 25; Hum mostly EF Core LINQ use karte hain - internally SQL generate hoti hai.

Q20.14. Primary key kya hai?

Ans. Column ya columns jo table me har row ko uniquely identify karta hai. Rules: • Unique hoga - duplicate nahi • NULL nahi ho sakta • Ek table me ek • Usually auto-increment Humara: Donor table me Id (Primary Key) BloodRequest me Id DonationOffer me Id Without PK: Confusion, duplicate records, no relationships.

Q20.15. Foreign key kya hai?

Ans. Column jo doosri table ke primary key ko reference karta hai - relationship banata hai. Humara: DonationOffer.RequestId → BloodRequest.Id DonationOffer.DonorId → Donor.Id Fayde: • Referential integrity • Cascade delete • Join queries • Data consistency Without FK: Orphan records, broken relationships.

Q20.16. Table kya hai database me?

Ans. Rows aur columns ka collection - spreadsheet jaisa. Example - Donors table: | Id | Name | BloodGroup | Age | |----|-------|-----------|-----| | 1 | Ravi | O+ | 27 | | 2 | Priya | A+ | 24 | Columns = Fields (Id, Name, BloodGroup, Age) Rows = Records (ek-ek donor) Humare tables: Donors, BloodRequests, DonationOffers, ContactMessages, AspNetUsers, AspNetRoles.

Q20.17. Normalization kya hai?

Ans. Database design technique - redundancy kam karna, data integrity improve karna. Normal forms: • 1NF - Atomic values, no repeating groups • 2NF - 1NF + no partial dependency • 3NF - 2NF + no transitive dependency Humara: • Separate tables for Donor, BloodRequest, DonationOffer • No duplicate data • Foreign keys se relationships Denormalization: Sometimes performance ke liye - but not humara case.

Q20.18. Variable kya hai?

Ans. Named storage location memory me jahan value store hoti hai. Declaration: int age = 27; string name = "Ravi"; bool isAvailable = true; Rules: • Must have type (C# me) • Meaningful name • Case-sensitive (age != Age) Types: • Local - method ke andar • Field - class ke andar • Property - field wrapper • Static - class-level

Q20.19. Data types kya hain?

Ans. Types of data jo variable hold kar sakta hai: Value types: • int - 4 bytes, -2B to 2B • long - 8 bytes, bigger • double - decimal • bool - true/false • char - single character • DateTime - date+time Reference types: • string - text • object - any • class - custom • array C# strongly typed - type mismatch compile error.

Q20.20. Operator kya hai?

Ans. Symbol jo operation perform karta hai. Types: • Arithmetic: + - * / % • Comparison: == != < > <= >= • Logical: && || ! • Assignment: = += -= *= • Increment: ++ -- • Ternary: ?: • Null-coalescing: ?? Example: int x = 5 + 3; // 8 bool ok = (x > 5) && (x < 10); // true string name = input ?? "Unknown";

Q20.21. Condition kya hoti hai?

Ans. Logical test jo true ya false return karta hai - program ka decision point. Types: • if-else • switch • Ternary (?:) Example: if (age >= 18) { Console.WriteLine("Eligible"); } else { Console.WriteLine("Not eligible"); } Humare: Donor age check (18-65), blood group compatible check.

Q20.22. Loop kya hai?

Ans. Repeatedly instructions execute karne ka mechanism. Types: • for - known count • while - condition based • do-while - at least once • foreach - collection iter Example: for (int i = 0; i < 10; i++) { Console.WriteLine(i); } foreach (var donor in donors) { Console.WriteLine(donor.Name); } Warning: Infinite loops - condition false na ho.

Q20.23. Function kya hai?

Ans. Reusable block of code jo specific task karta hai. Structure: returnType Name(parameters) { // body return value; } Example: public int Add(int a, int b) { return a + b; } Types: • Static method • Instance method • Extension method • Async method Benefits: Reusability, testability, readability.

Q20.24. Parameter aur Argument me difference?

Ans. Parameter: Function definition me placeholder naam. public void Greet(string name) { } // 'name' parameter Argument: Actual value jo pass karte hain. Greet("Ravi"); // "Ravi" argument Related terms: • Default parameter - void Greet(string name = "World") • Named argument - Greet(name: "Ravi") • Out/Ref - by reference • Params - variable arguments

Q20.25. Array kya hai?

Ans. Same type ke multiple values ek variable me. Declaration: int[] numbers = { 1, 2, 3, 4, 5 }; string[] names = new string[3]; Access: int first = numbers[0]; // 1 (zero-indexed) Properties: • Length - total items • Zero-indexed (0 to n-1) • Fixed size Hum: List<T> zyada use karte hain - dynamic.

Q20.26. List kya hai?

Ans. Dynamic collection - items add/remove kar sakte hain. Declaration: List<Donor> donors = new List<Donor>(); donors.Add(new Donor()); donors.RemoveAt(0); Methods: • Add, AddRange • Remove, RemoveAt, Clear • Contains, Find, FindAll • Count, Sort, Reverse List<T> - hum bahut use karte hain search results, donor lists.

Q20.27. Class aur Object ka real-life example?

Ans. Class = Blueprint Object = Actual house bana us blueprint se Example - Car: Class Car { string Brand; string Color; void Drive() { } } Object: var car1 = new Car { Brand = "Honda", Color = "Red" }; var car2 = new Car { Brand = "Toyota", Color = "Blue" }; car1.Drive(); // car1 ka drive car2.Drive(); // car2 ka drive Humara: Donor class, phir har registered donor ek object.

Q20.28. Encapsulation kya hai?

Ans. Data aur methods ko ek class me bundle karna, internal details hide karna. Example: public class Donor { private string _password; // hidden public string Password { get => _password; set => _password = Hash(value); // controlled } } Fayde: • Data protection • Controlled access • Implementation hide • Easier maintenance Humara: Donor class me RowVersion private - sirf EF Core access kare.

Q20.29. Inheritance kya hai?

Ans. Ek class doosri class ki properties aur methods inherit karti hai. Syntax: class Child : Parent { } Humara: public class AppUser : IdentityUser { // IdentityUser ke saare fields + methods inherit public bool IsBanned { get; set; } // extra } Types: • Single - one parent • Multi-level - chain • Hierarchical - one parent, many children C# me multiple class inheritance nahi - interfaces se.

Q20.30. Polymorphism kya hai?

Ans. Ek naam, different behaviors. Types: 1. Compile-time (Overloading): public void Print(int x) { } public void Print(string s) { } 2. Runtime (Overriding): class A { public virtual void Show() { } } class B : A { public override void Show() { } } Real example: A.ToString() - sab classes apna version dete hain. Humara: ToString() override karte hain models me (agar chahiye).

Q20.31. Abstraction kya hai?

Ans. Complex details hide karke sirf essential expose karna. Real example: Car drive karte ho - engine, fuel injection, transmission ka internal kaam nahi jaante. Sirf steering, pedals. Programming: public interface IMessageCache { Task<int> GetUnreadCountAsync(); void Invalidate(); } User ko sirf yeh 2 methods dikhte hain. Internal IMemoryCache details hide. Abstract class ya interface se achieve hoti hai.

Q20.32. Object kya hota hai?

Ans. Class ka instance - memory me actual thing. Example: Donor donor = new Donor(); // 'donor' is an object donor.Name = "Ravi"; donor.BloodGroup = "O+"; Object ke components: • State - properties (Name, BloodGroup) • Behavior - methods (Save, Update) • Identity - unique reference Memory me: Heap pe store, garbage collector manage karta hai.

Q20.33. Constructor kya karta hai?

Ans. Object create hone pe automatically call hone wala method - initialization ke liye. Example: public class Donor { public string Name { get; set; } public Donor() { // default Name = "Unknown"; } public Donor(string name) { // parameterized Name = name; } } Usage: var d1 = new Donor(); // "Unknown" var d2 = new Donor("Ravi"); // "Ravi" Humara: Controllers me constructor DI ke liye.

Q20.34. Method overloading kya hai?

Ans. Same method name, different parameters. Example: public class Calculator { public int Add(int a, int b) { return a + b; } public double Add(double a, double b) { return a + b; } public int Add(int a, int b, int c) { return a + b + c; } } Rules: • Same name, different signature • Compile-time resolution • Return type se overload nahi hota Fayde: Clean API, readability.

Q20.35. Exception kya hai?

Ans. Runtime error jo normal flow interrupt karta hai. Common: • NullReferenceException - null pe access • ArgumentException - invalid argument • DbUpdateException - DB constraint fail • DivideByZeroException • FileNotFoundException Handling: try { // risky code } catch (SpecificException ex) { // handle } finally { // cleanup } Humara: DbUpdateConcurrencyException catch karte hain.

Q20.36. try-catch kyun zaroori hai?

Ans. Unexpected errors se app crash rokta hai - graceful handling ke liye. Bina try-catch: App crash → user ko ugly error → bad experience With try-catch: try { var result = await _context.SaveChangesAsync(); } catch (DbUpdateConcurrencyException) { TempData["Error"] = "Record updated by someone else"; return View(donor); } User ko friendly message. Logs me details.

Q20.37. Console application aur Web application me difference?

Ans. Console App: • Terminal me chalti hai • Text output • No UI • Simple • dotnet run se chalti hai Web App: • Browser me chalti hai • HTML UI • Server + client • Complex • IIS/Kestrel pe host hoti hai • URLs se access Humara BloodConnect: Web app (ASP.NET Core MVC).

Q20.38. API kya hai?

Ans. Application Programming Interface - do software ek doosre se baat karte hain. Analogy: Restaurant menu - aap dish order karte ho (API call), kitchen banake deta hai (response). Aapko kitchen ke andar ka pata nahi. Types: • REST API - JSON/HTTP (modern) • SOAP - XML, strict • GraphQL - query language • gRPC - binary, high-performance Humara: /api/donor/search ek REST endpoint.

Q20.39. JSON kya hai?

Ans. JavaScript Object Notation - data format. Structure: { "name": "Ravi", "age": 27, "bloodGroup": "O+", "available": true, "groups": ["O+", "O-"] } Types: • Object { } • Array [ ] • String " " • Number 123 • Boolean true/false • null Human-readable, language-independent. Modern APIs ka standard.

Q20.40. HTML kya hai?

Ans. HyperText Markup Language - web page ka structure. Example: <!DOCTYPE html> <html> <head><title>BloodConnect</title></head> <body> <h1>Welcome</h1> Donate blood, save lives. Search Donors </body> </html> Elements: • Tags <tag> • Attributes <a href="..."> • Nesting Hum: Razor views HTML generate karte hain.

Q20.41. CSS kya hai?

Ans. Cascading Style Sheets - HTML ko style karta hai. Example: body { font-family: 'Inter', sans-serif; background: #FFF8F0; } .btn-primary { background: #8B1E3F; color: white; padding: 12px 24px; border-radius: 8px; } Selectors: • Element - p • Class - .btn • ID - #header Humara: site.css custom design system - warm palette.

Q20.42. JavaScript kya hai?

Ans. Client-side scripting language - web pages ko interactive banata hai. Example: document.querySelector('.btn').addEventListener('click', () => { alert('Clicked!'); }); Use: • Form validation • DOM manipulation • AJAX calls • Animations • Local storage Humara: Toast, modal, sound effects, animations - sab vanilla JS se.

Q20.43. Server kahan rehta hai?

Ans. Server typically datacenter me hota hai - physical ya cloud. Types: • Physical server - company ka data center • Cloud (Azure, AWS, GCP) - virtual machines • Shared hosting (MonsterASP) - multiple apps same server Aap ghar pe bhi bana sakte ho: • Localhost - development • Public IP + port forward - personal hosting Hum: Development local pe, production MonsterASP pe.

Q20.44. Port kya hai?

Ans. Number jo network service identify karta hai - apartment number jaisa. Common ports: • 80 - HTTP • 443 - HTTPS • 3306 - MySQL • 1433 - SQL Server • 22 - SSH • 21 - FTP • 5006 - Kestrel dev (humar■) URL me: http://localhost:5006/Donor/Search ↑ port number

Q20.45. Localhost kya hai?

Ans. Aapki own machine ko refer karta hai - 127.0.0.1. Jab dev server chalate ho: http://localhost:5006 - app local pe Fayde: • Development testing • No internet needed • Fast • Safe (external access nahi) Production me: domain name (bloodconnect.com).

Q20.46. Domain name kya hai?

Ans. Human-readable website address - IP address ka alias. Example: bloodconnect.com → 192.168.1.100 (example IP) Components: • TLD - .com, .org, .in • Domain - bloodconnect • Subdomain - www.bloodconnect.com DNS resolve karta hai - domain → IP. Registration: Namecheap, GoDaddy, BigRock. Cost: ~■800-1500/year.

Q20.47. URL kya hota hai?

Ans. Uniform Resource Locator - web resource ka address. Parts: https://bloodconnect.com:443/Donor/Search?bloodGroup=A%2B#results ■ ■ ■ ■ ■ ■ ■ ■ ■ ■ ■ ■■ Fragment ■ ■ ■ ■ ■■■■■■■■■■■■■■ Query string ■ ■ ■ ■■■■■■■■■■■■■■■■■■■■■■■■■ Path ■ ■ ■■■■■■■■■■■■■■■■■■■■■■■■■■■■ Port ■ ■■ Host ■■■■■■■■■■■■■■■■■■■

Q20.51. Cloud computing kya hai?

Ans. Internet pe computing resources on-demand - servers, storage, databases. Providers: • Azure (Microsoft) • AWS (Amazon) • GCP (Google) Types: • IaaS - VMs (Virtual Machines) • PaaS - App Service, Elastic Beanstalk • SaaS - Office 365, Gmail Fayde: • Pay-per-use • Scale on-demand • No hardware maintenance • Global reach Hum: Cloud services ka limited use (MonsterASP shared hosting).

Q20.52. Backup kya hota hai?

Ans. Data ki copy safe jagah pe - disaster recovery ke liye. Types: • Full - complete copy • Incremental - changes since last backup • Differential - changes since last full Strategy: 3-2-1 rule • 3 copies • 2 different media • 1 offsite Humara: SQL Server automatic backups MonsterASP pe. Local dev me script se.

Q20.53. Security kya hai?

Ans. Protection from unauthorized access, threats, attacks. Layers: 1. Network - firewall, HTTPS 2. Application - auth, validation, encryption 3. Data - encryption at rest, backups 4. User - strong passwords, 2FA 5. Monitoring - logs, alerts Humare project me: • HTTPS (production) • Identity authentication • Role-based authorization • CSRF protection • XSS prevention • SQL injection prevention • Password hashing

Q20.54. Encryption kya hai?

Ans. Data ko unreadable banane ka process - secret key ke bina. Types: • Symmetric - same key encrypt/decrypt (AES) • Asymmetric - public/private key (RSA) Uses: • HTTPS - TLS encryption • Password hashing - one-way (PBKDF2) • Data at rest - file encryption Humara: • Cookies encrypted (Data Protection API) • Passwords hashed (Identity) • HTTPS (TLS) production me

Q20.55. Cache kya hai?

Ans. Frequently-used data ki temporary fast copy. Analogy: Study table pe current book rakhi ho, bar bar almari se nahi nikalni padti. Levels: • Browser cache - CSS, images • CDN cache - global • App cache - memory • DB cache - query results Humara: IMemoryCache - unread message count. Tradeoff: Speed vs staleness.

SECTION 21

IIS, Windows Hosting & Practical

IIS, app pool, web.config, backup, health checks

Q21.1. IIS kya hai?

Ans. Internet Information Services - Microsoft ka web server Windows ke liye. Features: • HTTP/HTTPS hosting • Multiple sites/domains • Application pools (isolation) • Authentication • URL rewriting • SSL certificates MonsterASP pe: IIS + ASP.NET Core ke saath. Aapka app IIS ke andar ya Kestrel + IIS proxy me chalti hai. vs Kestrel: Kestrel cross-platform, IIS Windows-only. Production me usually Kestrel + IIS/Nginx.

Q21.2. Application Pool kya hai?

Ans. IIS me isolation boundary - har app pool alag w3wp.exe process me chalti hai. Fayde: • Ek site crash → doosri safe • Different .NET versions • Separate permissions • Recycle independently MonsterASP: Aapki app ke liye alag app pool allocate hoti hai (paid plans me). Recycle reasons: • Idle timeout (default 20 min) • Memory limit • Config change • Manual restart

Q21.3. web.config kya hai?

Ans. IIS ke liye config file - app settings, handlers, modules. Example: <?xml version="1.0" encoding="utf-8"?> <configuration> <system.webServer> <handlers> <add name="aspNetCore" path="*" verb="*" modules="AspNetCoreModuleV2" resourceType="Unspecified" /> </handlers> <aspNetCore processPath="dotnet" arguments=".\BloodConnect.dll" hostingModel="inprocess" /> </system.webServer> <appSettings> <add key="ASPNETCORE_ENVIRONMENT" value="Production" /> </appSettings> </configuration> Note: .NET Core me appsettings.json primary hai - web.config sirf IIS config.

Q21.4. Windows Server kya hai?

Ans. Microsoft ka server OS - enterprise applications ke liye. Features: • Active Directory • IIS • Hyper-V (virtualization) • Storage management • Cluster services • Group Policy MonsterASP: Windows Server running IIS + SQL Server. Aapke liye transparent - sirf app upload karo, baaki manage provider karta hai.

Q21.5. Kestrel vs IIS?

Ans. Kestrel: • Cross-platform web server • .NET Core default • Fast, lightweight • Console app jaisa • No Windows dependency IIS: • Windows only • Feature-rich • Windows auth • URL rewrite module • Complex config Production patterns: 1. Kestrel only (Linux/cloud) 2. IIS as reverse proxy → Kestrel (Windows recommended) 3. IIS + ASP.NET Core module → in-process MonsterASP: IIS + Kestrel (in-process hosting).

Q21.6. In-process vs Out-of-process hosting?

Ans. In-process: • App IIS worker process (w3wp.exe) me chalti hai • Faster (no localhost network hop) • IIS features direct • Default in .NET Core 3+ Out-of-process: • IIS forward karta hai Kestrel ko • App alag process me • Slower (extra hop) • Cross-platform testing easier web.config: hostingModel="inprocess" // recommended for IIS hostingModel="outofprocess" // fallback MonsterASP: inprocess.

Q21.7. Web garden kya hai?

Ans. Ek application pool me multiple worker processes. Config: • Maximum Worker Processes > 1 (usually 1) Fayde: • Better concurrency • One crash → others fine Nuksan: • In-memory cache per process - inconsistency • Session state issues • More memory Humara: IMemoryCache - web garden me issue hoga. Isliye single process recommended. Distributed cache (Redis) agar scale karna ho.

Q21.8. Session affinity (sticky sessions) kya hai?

Ans. Load balancer me same client ko same server pe bhejna. Problem: In-memory cache, session state - per server. Solution: 1. Sticky sessions - client same server pe 2. Distributed cache - Redis, sab servers share karein 3. Stateless - har request independent (best) Humara: Cookie auth stateless. IMemoryCache single server me. Scale pe Redis + no sticky.

Q21.9. URL rewrite kya hai?

Ans. URL ko change karke redirect ya rewrite karna. Examples: • HTTP → HTTPS • www → non-www (ya vice versa) • Old URL → new URL (SEO) • Pretty URLs IIS: <rewrite> <rules> <rule name="Redirect to HTTPS" stopProcessing="true"> <match url="(.*)" /> <action type="Redirect" url="https://{HTTP_HOST}/{R:1}" /> </rule> </rules> </rewrite> Better: Code me UseHttpsRedirection() middleware.

Q21.10. Static files kaise serve hote hain?

Ans. ASP.NET Core me: app.UseStaticFiles(); Kya serve hota hai: • wwwroot/css/*.css • wwwroot/js/*.js • wwwroot/images/* Cache-busting: <link rel="stylesheet" href="~/css/site.css" asp-append-version="true" /> Yeh URL me hash add karta hai - file change pe naya URL - browser cache invalidate. IIS pe static file handlers optimize hote hain.

Q21.11. Production me App Pool recycle kaise handle karte hain?

Ans. Recycle = process restart. In-memory cache, sessions reset. Impact: • First request slow (JIT warm-up) • IMemoryCache empty • Any in-memory state loss Prevention: • Keep-alive pings (external cron) • Application Initialization module (IIS) • Warm-up script • Persistent cache (Redis) MonsterASP free: frequent recycle ho sakta hai. Paid: better control.

Q21.12. Warm-up kya hai?

Ans. App start pe JIT compilation, cache warm, connections prepare. First request slow - "cold start". Warm-up approaches: • IIS Application Initialization • Health check endpoint ping • Self-ping loop • ReadyToRun (pre-JIT) Humara: 5006 pe local pe. MonsterASP: keep-alive pings.

Q21.13. Log files kahan jaati hain production me?

Ans. ASP.NET Core logging: • Console - IIS me capture nahi hoti (unless configured) • File - custom path • EventLog - Windows • Application Insights - cloud IIS logs: %SystemDrive%\inetpub\logs\LogFiles\ MonsterASP: • Dashboard → Logs • WebDeploy folder me stdout logs web.config: stdoutLogEnabled="true" stdoutLogFile=".\logs\stdout" Note: Production me structured logging (Serilog) better.

Q21.14. Deployment slots kya hain?

Ans. Azure App Service feature: • Staging slot - test • Production slot - live • Swap - instant switch • Zero downtime MonsterASP: no slots (basic hosting). Alternative: • Manual swap - DNS change • Subdomain staging (staging.bloodconnect.com) • Separate paid hosting

Q21.15. Gzip compression kaise enable karte hain?

Ans. IIS: <httpCompression> <dynamicTypes> <add mimeType="application/json" enabled="true" /> <add mimeType="text/html" enabled="true" /> </dynamicTypes> </httpCompression> Code: builder.Services.AddResponseCompression(opts => { opts.EnableForHttps = true; opts.MimeTypes = ResponseCompressionDefaults.MimeTypes.Concat(new[] { "application/json" }); }); app.UseResponseCompression(); Fayde: 70-90% size reduction text content ka. Faster load.

Q21.16. CDN kya hai?

Ans. Content Delivery Network - static content global servers se serve. Example: Cloudflare free tier Fayde: • Faster (user ke paas wala server) • DDoS protection • Reduced server load • Caching Setup: 1. Cloudflare account 2. Nameservers change 3. DNS records 4. Caching rules Humara: Optional. Educational project me skip. Production me recommended.

Q21.17. FTP credentials safe kaise rakhte hain?

Ans. NEVER: • Code me hardcode • Git me commit • Public repos me share BETTER: • Password manager (Bitwarden, 1Password) • Environment variables • CI/CD secrets (GitHub Actions Secrets) • SSH keys (FTP-SSH) GitHub Actions me: ${{ secrets.FTP_USER }} ${{ secrets.FTP_PASS }} Rotation: Regular change karo, especially if team member leaves.

Q21.18. Backup strategy production me?

Ans. 3-2-1 rule: • 3 copies • 2 different media • 1 offsite SQL Server backup: BACKUP DATABASE BloodConnectDb TO DISK = 'C:\Backups\bloodconnect.bak' MonsterASP: • Dashboard → Backup • Scheduled (paid) • Manual download Frequency: • Daily - typical • Weekly - full, daily - incremental • Before major deploy - mandatory

Q21.19. Health check endpoint kya karta hai?

Ans. Endpoint jo app health report karta hai - monitoring ke liye. Setup: builder.Services.AddHealthChecks() .AddDbContextCheck<ApplicationDbContext>(); app.MapHealthChecks("/health"); Response: Healthy / Degraded / Unhealthy Uses: • Load balancer - route only healthy servers • Kubernetes - liveness/readiness • Monitoring - uptime alerts Humare project me: roadmap.

Q21.20. Production me environment variable vs appsettings.json?

Ans. Priority (higher overrides lower): 1. Command line args 2. Environment variables 3. User secrets (dev only) 4. appsettings.{Environment}.json 5. appsettings.json Best practice: • appsettings.json - defaults, non-sensitive • Environment variables - secrets, environment-specific • User secrets - local dev secrets • Vault (Azure/AWS) - production secrets MonsterASP: Panel → Environment Variables.

Final Words - Sabse Important

Bhai, tumne ek complete full-stack .NET project banaya hai. 7 controllers, 5 domain entities, Identity + role hierarchy, medical logic, pledge workflow, caching, REST API, security layers. Yeh average student project nahi hai. Viva me yeh confidence rakhna - tumne kuch meaningful banaya hai.

Last Minute Checklist

1. *Project ek baar local pe run karke dekh lo - sab kaam kar raha hai?*
2. *Admin credentials yaad rakho: admin@bloodconnect.com / Admin@123*
3. *Folder structure ek baar dekh lo - kaha kaunsi file hai*
4. *Agar laptop allowed - battery, charger, internet check*
5. *Print ya digital - yeh guide saath rakho*
6. *Confidence > Perfection. Smile karo. Bolke batao.*

All the best! Tum phodke aao. Every drop counts, every question counts too.